COSC 3320

Exam 1 Survival Book

Algorithms with Dr. Leiss • open-book edition

What is inside

Finder	*"Which page do I go to?"* for every question type (page iii)
Part 0	Foundations from zero: pseudocode, recursion, Big-O, sums, recurrences
Part 1	Q1 type: Towers of Hanoi on a graph
Part 2	Q2 type: counting page transfers (virtual memory, LRU)
Part 3	Q3 type: QuickSort best / worst arrays for any pivot position
Part 4	Q4 type: matrix-chain products: min, **max**, **average**
Part 5	Other topics seen on Leiss exams: Huffman, HeapSort, RSA, Euclid
Part 6	Every uploaded past exam, solved: 2008, 2012, 2014, 2018 (+ 2020/21)
Part 7	Five mock exams in Leiss's format
Part 8	Full answer keys for the five mock exams
Appendix	One-page formula sheet and answer index

Prepared for Erica • October 2026.

Every number in this book was checked by computer: page-transfer counts by simulating LRU memory on the full-size arrays; Hanoi move lists by a simulator that rejects illegal moves; QuickSort arrays by actually running the stated partition; matrix-chain tables against a brute-force list of all parenthesizations.

Read this first

What Leiss's Exam 1 has looked like

Every Exam 1 you uploaded (2008, 2012, 2014, 2018) has **exactly the same four-question template**, each worth 25%, open books and notes, 80–85 minutes:

Q	Type	What changes from year to year
1	Towers of Hanoi on a graph	The arrows between Start, Aux1, Aux2, Dest. (A one-way loop of 4 pegs in 2008/2014/2018; a "lollipop" in 2012.)
2	Page transfers with LRU	Page size, number of page frames, array size, row- vs column-major. The loop is almost always `A[J,I] := A[J,I] * B[I,J]`.
3	QuickSort best/worst arrays	Which position the pivot comes from (first, last, second, next-to-last) and the minimum n.
4	Matrix-chain products	**Maximum** instead of minimum (2012, 2014, 2018, 2020) or **average** (2008), and the five matrix dimensions.

The 2020 exam (online semester) kept Q2–Q4 and added Huffman coding, HeapSort and RSA bit complexity. Those are in Part 5, in case they show up.

Watch out: an honest note

Nobody can guarantee a grade, and Leiss can always change the questions. But the pattern above has held for at least a decade, so if you can do the four recipes in Parts 1–4 you are prepared for the most likely exam. The mock exams in Part 7 use *new* numbers and *new* graph shapes so you practice adapting, not memorizing.

How to use this book during an open-book exam

1. Read the question and match it to a row of the **Finder** on the next page.
2. Jump to the recipe page. Every recipe is a numbered list of steps. Follow them in order.
3. Copy the matching *exam template* (green boxes) and fill in your numbers.
4. **Always show your work.** The graded exams you uploaded show that Leiss's graders give credit for the method (the recurrence, the table, the trace), not just the final number. One 2018 student wrote the right final number on Q4 but had errors in the table and got only 15 of 25.
5. Budget about 20 minutes per question. If you get stuck, write the method (recurrence/recipe) and move on.

If you are studying the night before

Time	Do this
30 min	Part 0 only as needed (skip what you already know). Read every *Key idea* box.
45 min	Part 2 (paging). It is the most mechanical question; aim to get 25/25.
45 min	Part 4 (matrix chain). Also mechanical. Practice one full table by hand.
40 min	Part 3 (QuickSort). Learn the idea; the arrays are already worked out for you.
40 min	Part 1 (Hanoi). Learn to *identify the graph shape*; the algorithms are written out.
85 min	Take Mock Exam 1 under time pressure, with this book open. Then check with Part 8.
More time?	Mock Exams 2–5. Then Part 5.

Finder: which page do I go to?

If the question says…	It is…	Go to page
Towers of Hanoi		
"Towers of Hanoi … graph $G = (V, E)$" — first, **draw the graph**	identify the shape	10
Every peg has exactly one arrow out and one arrow in (a loop through all 4 pegs)	4-peg one-way loop	11
… and Dest is 1 / 2 / 3 arrows after Start	templates	14
Start has an arrow out but no arrow in; the other three form a loop	"tail-in lollipop" (2012)	16
Dest has an arrow in but no arrow out	"tail-out lollipop"	17
Only three pegs, or two-way edges, or extra edges	other shapes	18
"Estimate the time complexity"	solving the recurrences	13
Page transfers		
"how many pages are transferred between disk and main memory"	5-step recipe	24
Same numbers as a past exam?	lookup table	26
Memory might be big enough to keep pages (large frame count)	reuse rule	23
QuickSort		
"construct an assignment of the numbers $1 \ldots n$" — general recipe	recipe	31
pivot = **first** element $A[lo]$ (2008, 2012, 2018)	ready arrays	32
pivot = **last** element $A[hi]$ (2014)	ready arrays	33
pivot = **second** element $A[lo + 1]$ (2020)	ready arrays	34
pivot = **next-to-last** $A[hi - 1]$ (2021)	ready arrays	35
pivot = **middle** element	ready arrays	36
"n at least 16" (or bigger)	$n = 31$ arrays	36
Matrix chain		
"multiplying n rectangular matrices" — how the DP works	DP recipe	39
"**maximum** number of scalar multiplications"	max version	41
"**average** number"	average version	44
"minimum" (the version from class)	min version	43
Other topics		
Huffman code / expected code length		48

Heap / HeapSort, "show the heap after each insertion"	49
RSA, bit complexity, Gauss/Karatsuba multiplication	50
Multiplicative inverse / Euclid's algorithm	51
Solving a recurrence	5
Past exams and mocks	
Past exams solved (2008, 2012, 2014, 2018, 2020/21)	54
Mock exams / answer keys	65 / 72
One-page formula sheet	90

Contents

Read this first i

Finder: which page do I go to? iii

I Foundations from Zero 1

1 The Basics You Need (Explained Slowly) 2

1.1 How to read pseudocode . . . 2
1.2 Recursion: a procedure that calls itself . . . 3
1.3 Time complexity and Big-O . . . 4
1.4 Powers of two you will use constantly . . . 5
1.5 Sums . . . 5
1.6 Recurrences: how to "estimate the time complexity" . . . 5

II Question 1: Towers of Hanoi on a Graph 8

2 Towers of Hanoi on a Graph 9

2.1 What the question is asking . . . 9
2.2 The one principle behind every solution: parking . . . 9
2.3 Identify the shape of your graph (30 seconds) . . . 10
2.4 Warm-up: a one-way loop of 3 pegs . . . 11
2.5 The 4-peg one-way loop (2008, 2014, 2018) . . . 11
2.6 Ready-to-copy answers for each 4-peg loop . . . 14
2.7 Tail-in lollipop (2012) . . . 16
2.8 Tail-out lollipop (Dest is a dead end) . . . 17
2.9 Other shapes . . . 18
2.10 Grading notes and common mistakes . . . 19

III Question 2: Counting Page Transfers 20

3 Virtual Memory, Pages and LRU 21

3.1 The question . . . 21
3.2 What is happening inside the computer . . . 21

3.3 How a 2D array is laid out in memory . . . 22
3.4 Walking versus jumping . . . 23
3.5 The LRU reuse rule . . . 23
3.6 The 5-step recipe . . . 24
3.7 2018 / 2014, fully worked: row-major, 1024 frames . . . 24
3.8 2012 and 2008: smaller numbers, same pattern . . . 25
3.9 2020: column-major (the mirror image) . . . 25
3.10 2008 Exam 2: a different loop body . . . 25
3.11 Lookup table: common variations . . . 26
3.12 Edge cases and how to handle them . . . 26

IV Question 3: QuickSort Best and Worst Inputs 28

4 QuickSort: Building the Best and the Worst Input 29
4.1 The question . . . 29
4.2 QuickSort from zero . . . 29
4.3 Best case vs. worst case . . . 30
4.4 Why we choose $n = 15$ . . . 31
4.5 The construction recipe . . . 31
4.6 Pivot = first element $A[lo]$ (2008, 2012, 2018) . . . 32
4.7 Pivot = last element $A[hi]$ (2014) . . . 33
4.8 Pivot = second element $A[lo+1]$ (2020) . . . 34
4.9 Pivot = next-to-last element $A[hi-1]$ (2021) . . . 35
4.10 Pivot = middle element $A[\lfloor (lo+hi)/2 \rfloor]$ . . . 36
4.11 Arrays for $n = 31$ (if the exam asks for $n \geq 16$) . . . 36
4.12 If your partition procedure is different . . . 37

V Question 4: Matrix-Chain Products 38

5 Matrix Chains: Minimum, Maximum, Average 39
5.1 The question . . . 39
5.2 Matrix multiplication in two minutes . . . 39
5.3 Why the parenthesization matters . . . 39
5.4 The dynamic-programming method . . . 39
5.5 MAXIMUM, fully worked: 2018 . . . 41
5.6 MINIMUM (the version from class) . . . 43

5.7 AVERAGE (2008) 44

VI Other Topics That Have Appeared **47**

6 Huffman, HeapSort, RSA, Euclid **48**

6.1 Huffman codes 48

6.2 Heaps and HeapSort 49

6.3 Fast multiplication, repeated squaring, and RSA 50

6.4 Euclid's algorithm and multiplicative inverses 51

VII Past Exams, Solved **53**

7 Past Exams with Model Answers **54**

7.1 Exam 1, Monday Feb. 25, 2008 54

7.2 Exam 1, Wednesday Sept. 26, 2012 56

7.3 Exam 1, Wednesday Sept. 17, 2014 58

7.4 Exam 1, Wednesday Feb. 21, 2018 59

7.5 2020 and 2021 versions (from your review sheets) 60

VIII Five Mock Exams **64**

8 Mock Exams **65**

8.1 Mock Exam 1 66

8.2 Mock Exam 2 67

8.3 Mock Exam 3 68

8.4 Mock Exam 4 (2020-style, six questions) 69

8.5 Mock Exam 5 70

IX Answer Keys for the Mock Exams **71**

9 Mock Exam Solutions **72**

9.1 Mock Exam 1: solutions 72

9.2 Mock Exam 2: solutions 74

9.3 Mock Exam 3: solutions 78

9.4 Mock Exam 4: solutions 81

9.5 Mock Exam 5: solutions 85

A One-Page Formula Sheet **90**

B Answer Index **91**

PART I

Foundations from Zero

CHAPTER 1

The Basics You Need (Explained Slowly)

This chapter assumes nothing. If a topic is already comfortable, skip it. Everything later in the book points back here when it uses one of these ideas.

1.1 How to read pseudocode

Leiss writes algorithms in *pseudocode*: a programming-like notation that is meant for humans, not computers. You only need a handful of pieces.

Variables and assignment

`x := 5` means "put the value 5 into the box called `x`". The symbol `:=` is *assignment* (in C++ you would write `x = 5;`). `x := x + 1` means "take what is in `x`, add 1, put it back".

Arrays

- `A[1:n]` is an array (a row of boxes) with positions $1, 2, \ldots, n$. Leiss counts from 1, not 0.
- `A[lo:hi]` means "the piece of `A` from position `lo` to position `hi`". QuickSort works on such pieces (called *subarrays*).
- A two-dimensional (2D) array `A[1:2048, 1:2048]` is a grid with 2048 rows and 2048 columns. `A[i,j]` is the box in **row** i, **column** j. The *first* index is always the row, the *second* is always the column.

Loops

```
for I := 1 to 3 do
   for J := 1 to 3 do
      { something with I and J }
```

The **inner** loop (`J`) runs completely for *each* value of the **outer** loop (`I`). So the pairs (I, J) happen in this order:

$$(1,1), (1,2), (1,3),\ (2,1), (2,2), (2,3),\ (3,1), (3,2), (3,3).$$

Key idea

In a double loop, **the inner index changes fastest**. This single fact is the heart of the page-transfer question (Part 2).

Procedures (functions) and `if`

```
procedure Max(a, b)
   if a ≥ b then return a
   else return b
```

A *procedure* is a named recipe with inputs (parameters). `Max(3,7)` runs the recipe with `a=3, b=7` and gives back 7.

1.2 Recursion: a procedure that calls itself

In plain English

Recursion is *delegation*. To solve a big problem, you solve one small piece yourself and hand a *smaller copy of the same problem* to a helper who follows the exact same instructions. The helper does the same thing, and so on, until the problem is so small that it is trivial (the *base case*).

Example: factorial

$n! = n \cdot (n-1) \cdots 2 \cdot 1$. Notice $n! = n \cdot (n-1)!$. So:

```
procedure Fact(n)
   if n = 1 then return 1          // base case: trivially small
   return n * Fact(n-1)            // smaller copy of the same problem
```

Trace of `Fact(4)`:

$$\text{Fact}(4) = 4 \cdot \text{Fact}(3) = 4 \cdot 3 \cdot \text{Fact}(2) = 4 \cdot 3 \cdot 2 \cdot \text{Fact}(1) = 4 \cdot 3 \cdot 2 \cdot 1 = 24.$$

The two rules of every correct recursive procedure

1. **Base case:** there is a smallest input you solve directly (here $n = 1$; in Hanoi, $n = 0$ disks: do nothing).
2. **Progress:** every recursive call is on a strictly smaller input (here $n - 1$), so you must eventually reach the base case.

Key idea: the "leap of faith"

When you *design* a recursive procedure, you do **not** trace every call in your head. You *assume* the procedure already works correctly for smaller inputs, and you only check that the current level uses those answers correctly. This is exactly how proofs by induction work, and it is how every Hanoi algorithm in Part 1 is designed.

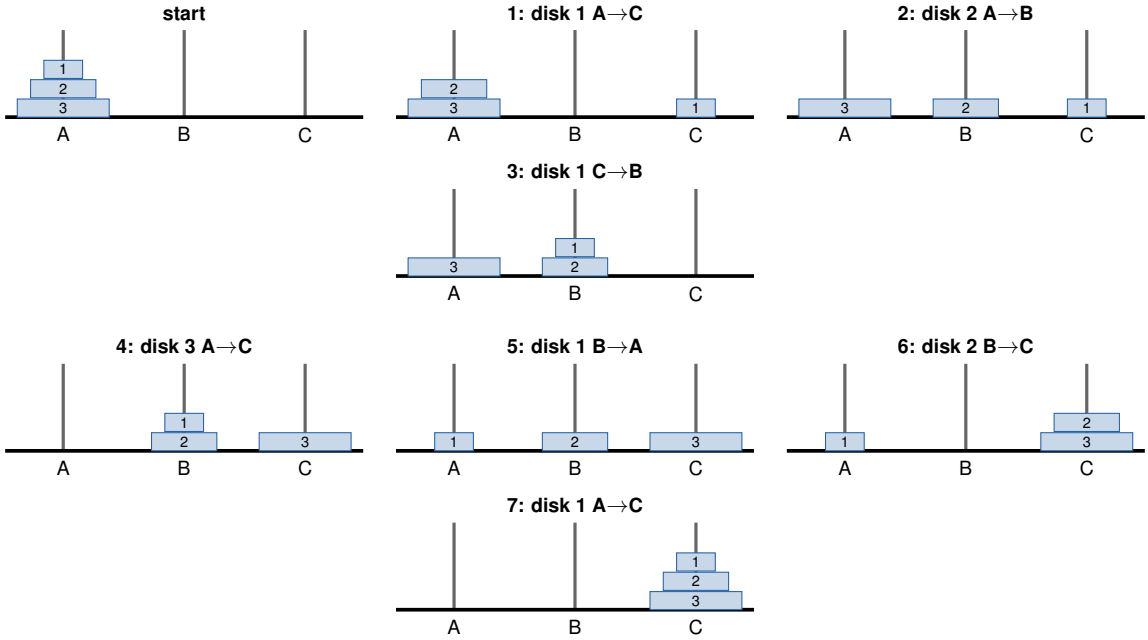
The classic Towers of Hanoi (3 pegs, any move allowed)

Three pegs A, B, C. On A sit n disks, largest at the bottom. Goal: move all of them to C. **Rules:** move one disk at a time (the top disk of a peg); never put a larger disk on a smaller one.

Recursive idea. The largest disk n can only move when it is alone on its peg and the target peg is empty, i.e. all $n - 1$ smaller disks are on the *third* peg. So:

```
procedure Hanoi(k, from, to, other)     // move the top k disks from 'from' to 'to'
   if k = 0 then return                 // nothing to move
   Hanoi(k-1, from, other, to)          // 1. park the k-1 smaller disks on 'other'
   move disk k from 'from' to 'to'      // 2. the big disk moves
   Hanoi(k-1, other, to, from)          // 3. bring the smaller disks on top of it
```

Here is the whole solution for $n = 3$ (7 moves). Disk numbers are sizes.



Moves 1–3 are "park the 2 smaller disks on B", move 4 is "big disk moves", moves 5–7 are "bring them back on top".

Counting moves. Let $T(n)$ = number of moves for n disks. Then

$$T(0) = 0, \qquad T(n) = T(n-1) + 1 + T(n-1) = 2T(n-1) + 1.$$

So $T(1) = 1, T(2) = 3, T(3) = 7, T(4) = 15, \ldots$ and in general $T(n) = 2^n - 1$ (we prove this in Section 1.6).

1.3 Time complexity and Big-O

Time complexity = how the amount of work grows as the input size n grows. We count basic steps (a comparison, a disk move, an arithmetic operation) and express the count as a function of n.

Big-O, Big-Omega, Big-Theta

- $f(n) = \mathcal{O}(g(n))$: "f grows **at most** as fast as g" (up to a constant factor). Formally: there are constants $c > 0, n_0$ with $f(n) \leq c\, g(n)$ for all $n \geq n_0$.
- $f(n) = \Omega(g(n))$: "f grows **at least** as fast as g".
- $f(n) = \Theta(g(n))$: both; "f grows **exactly** like g".

Rules of thumb: drop constant factors and lower-order terms. $3n^2 + 100n + 7 = \Theta(n^2)$; $5 \cdot 2^n - 3 = \Theta(2^n)$.

Watch out

Exponentials with different bases are **different** orders: 3^n is *not* $\mathcal{O}(2^n)$, because $3^n/2^n = (1.5)^n \to \infty$. So in the Hanoi question, the base matters: $\Theta(2^n)$, $\Theta(2.56^n)$, $\Theta(2.73^n)$, $\Theta(3^n)$ are all different answers. Saying $\mathcal{O}(3^n)$ when the truth is $\Theta(2.56^n)$ is still *correct* (it is an upper bound), just less precise.

How fast do things grow?

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n	3^n
10	3.3	10	33	100	1,024	59,049
20	4.3	20	86	400	$\approx 10^6$	$\approx 3.5 \cdot 10^9$
100	6.6	100	664	10,000	$\approx 10^{30}$	$\approx 5 \cdot 10^{47}$

1.4 Powers of two you will use constantly

2^7	128	2^{10}	1,024	2^{20}	1,048,576
2^8	256	2^{11}	2,048	2^{21}	2,097,152
2^9	512	2^{12}	4,096	2^{22}	4,194,304
		2^{13}	8,192	2^{23}	8,388,608

Handy products: $1024 \cdot 1024 = 1{,}048{,}576$; $2048 \cdot 2048 = 4{,}194{,}304$; $2048 \cdot 2 = 4096$; $1024 \cdot 2 = 2048$; $256 \cdot 256 = 65{,}536$. **Logs:** $\log_2 x$ is "which power of 2 gives x": $\log_2 1024 = 10$, $\log_2 2048 = 11$.

1.5 Sums

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} \qquad \text{(arithmetic series; e.g. } 1 + \cdots + 14 = 105\text{)}$$

$$1 + 2 + 4 + \cdots + 2^{k-1} = 2^k - 1 \qquad \text{(geometric series, ratio 2)}$$

$$1 + a + a^2 + \cdots + a^k = \frac{a^{k+1} - 1}{a - 1} = \Theta(a^k) \text{ for } a > 1 \qquad \text{(general geometric series)}$$

Key idea

A geometric sum is dominated by its **last (largest) term**: $1 + 3 + 9 + \cdots + 3^k < 1.5 \cdot 3^k$. This is why "$T(n)$ = (something growing like c^n) + $T(n-1)$" still grows like c^n.

1.6 Recurrences: how to "estimate the time complexity"

A **recurrence** defines $T(n)$ in terms of T of smaller inputs. Recursive algorithms produce recurrences: one term per recursive call plus the non-recursive work.

Method 1: unrolling (works for the classic Hanoi)

$T(n) = 2T(n-1) + 1$. Substitute repeatedly:

$$\begin{aligned} T(n) &= 2T(n-1)+1 = 2\big(2T(n-2)+1\big)+1 = 4T(n-2)+2+1 \\ &= 8T(n-3)+4+2+1 = \cdots = 2^n T(0) + (2^{n-1}+\cdots+2+1) = 0 + 2^n - 1. \end{aligned}$$

Method 2: the pattern table (memorize or look up)

Recurrence	Solution	Where you see it
$T(n) = T(n-1) + c$	$\Theta(n)$	a loop written recursively
$T(n) = T(n-1) + cn$	$\Theta(n^2)$	QuickSort worst case
$T(n) = a\,T(n-1) + c,\ a \geq 2$	$\Theta(a^n)$	Hanoi (with $a = 2, c = 1, T(0) = 0$: exactly $2^n - 1$)
$T(n) = T(n/2) + c$	$\Theta(\log n)$	binary search
$T(n) = 2T(n/2) + cn$	$\Theta(n \log n)$	MergeSort, QuickSort best case
$T(n) = 3T(n/2) + cn$	$\Theta(n^{\log_2 3}) = \Theta(n^{1.585})$	Gauss/Karatsuba multiplication
$T(n) = 7T(n/2) + cn^2$	$\Theta(n^{\log_2 7}) = \Theta(n^{2.81})$	Strassen matrix multiplication

The divide-and-conquer ("master") theorem

If $T(n) = a\,T(n/b) + c\,n^d$ (with $a \geq 1$, $b > 1$), then

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Method 3: recurrences with two earlier terms (needed for Hanoi on graphs)

Some Hanoi algorithms give recurrences like $T(n) = T(n-1) + 4T(n-2) + \text{const}$. Here is the trick:

Recipe: growth rate of a linear recurrence

1. Ignore the constant (it does not change the exponential growth rate).
2. Guess $T(n) \approx x^n$ and plug in: $x^n = x^{n-1} + 4x^{n-2}$.
3. Divide by x^{n-2}: $x^2 = x + 4$, i.e. $x^2 - x - 4 = 0$ (the *characteristic equation*).
4. Solve with the quadratic formula: $x = \frac{1 \pm \sqrt{1+16}}{2}$. The **largest** root $x = \frac{1+\sqrt{17}}{2} \approx 2.56$ is the growth rate.
5. Conclusion: $T(n) = \Theta\big(2.56^n\big)$.

Sanity check with Fibonacci: $F(n) = F(n-1) + F(n-2)$ gives $x^2 = x + 1$, $x = \frac{1+\sqrt{5}}{2} \approx 1.618$, the golden ratio. Correct.

Method 4: quick upper bound (always acceptable for "estimate")

If every recursive step makes at most 3 calls on size $n-1$ plus constant work, then $T(n) \leq 3T(n-1)+c$, and by the table $T(n) = \mathcal{O}(3^n)$. This gives a valid (if slightly loose) answer in 30 seconds. On the exam, give this bound *and* the precise rate if you have time.

Mutually recursive procedures (several procedures calling each other)

When procedures P_1, P_2, P_3 call one another, write one recurrence per procedure, e.g.

$$T_1(n) = T_2(n-1) + T_3(n-1) + 1, \quad T_2(n) = 2T_3(n-1) + 2, \quad T_3(n) = 2T_2(n-1) + T_3(n-1) + 3.$$

Then *substitute* to get a recurrence in one unknown. From the second equation $T_2(n-1) = 2T_3(n-2) + 2$; put it into the third:

$$T_3(n) = 2\big(2T_3(n-2)+2\big) + T_3(n-1) + 3 = T_3(n-1) + 4T_3(n-2) + 7,$$

which is Method 3: growth rate $\frac{1+\sqrt{17}}{2} \approx 2.56$. Then T_2 and T_1 (built from T_3) grow at the same rate.

PART II

Question 1: Towers of Hanoi on a Graph

CHAPTER 2

Towers of Hanoi on a Graph

2.1 What the question is asking

Every year the question reads like this (2018 version):

Exam 1, 2018, Q1 (25%)

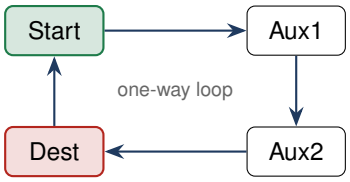
Design a function (algorithm) that solves the Towers of Hanoi game for the following graph $G = (V, E)$ with $V = \{\text{Start}, \text{Aux1}, \text{Aux2}, \text{Dest}\}$ and $E = \{(\text{Start}, \text{Aux1}), (\text{Aux1}, \text{Aux2}), (\text{Aux2}, \text{Dest}), (\text{Dest}, \text{Start})\}$. Estimate the time complexity of your function, in terms of the number n of disks to be moved.

In plain English

It is the usual Towers of Hanoi (Section 1.2): n disks start on peg *Start* and must end on peg *Dest*; one disk at a time; never a larger disk on a smaller one. The **new rule**: a disk may move from peg X to peg Y **only if the pair** (X, Y) **is in** E. Each pair is a **one-way arrow** $X \to Y$. You may *not* move from Y back to X unless (Y, X) is also listed.

Step 0 on the exam: draw the graph

Draw the four pegs as boxes and draw one arrow per pair in E. For 2018:



Here a disk can only travel "clockwise": Start→Aux1→Aux2→Dest→Start. To get from Start to Dest takes 3 arrow-steps; to get from Dest back to Aux2 takes 3 steps as well (Dest→Start→Aux1→Aux2).

2.2 The one principle behind every solution: parking

Key idea: the parking principle

Think about the **largest disk** that has to move, call it disk k. At the moment it moves along an arrow $X \to Y$:

- it must be the *top* disk on X, and
- Y must not contain any smaller disk (or k would land on a smaller disk).

So **all smaller disks** $1, \ldots, k-1$ **must be "parked" on pegs other than** X **and** Y. The simplest way: keep the $k-1$ smaller disks together as one tower and park that tower on a third peg.

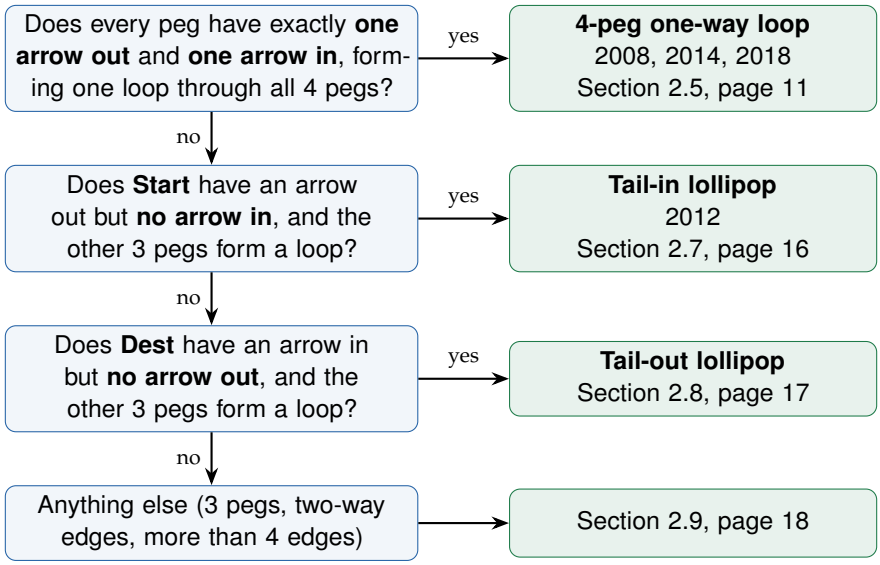
Two more facts make the recursion work:

1. **The big disk never gets in the way.** While the smaller disks move around, disk k sits at the bottom of its peg. Any smaller disk may sit on top of it, so it never blocks anything. Disks larger than k are even further below and are never touched.
2. **So moving a tower of $k-1$ disks is the same problem with $k-1$ disks** (just between different pegs), which is exactly what a recursive call solves.

Recipe: designing a Hanoi-on-a-graph algorithm

1. Draw the graph. Find the **path** the biggest disk must travel (the arrows from its peg to its target).
2. For *each* arrow of that path, decide where the smaller tower is parked (any peg that is neither end of the arrow).
3. Between big-disk moves, move the smaller tower from one parking spot to the next (a recursive call).
4. At the end, move the smaller tower onto the big disk (a recursive call).
5. Every different "from–to" pattern for the tower becomes its own procedure; write one recurrence per procedure; solve.

2.3 Identify the shape of your graph (30 seconds)



On the exam: extra arrows are harmless

If the graph has *more* arrows than one of these shapes, you may simply ignore the extra arrows and use the algorithm for the shape you find inside it. An algorithm that only uses some of the allowed moves is still a correct algorithm. (It may not be the fastest possible, but the question asks for *an* algorithm and its complexity.)

2.4 Warm-up: a one-way loop of 3 pegs

This smaller case is worth understanding first, and the lollipop graphs (2012) use it as a building block. Pegs $Q_0 \to Q_1 \to Q_2 \to Q_0$ (indices taken mod 3, so $Q_3 = Q_0$).

- **Moving a tower 1 step forward** ($Q_p \to Q_{p+1}$): the big disk moves along one arrow. The smaller disks must be on the only other peg, Q_{p+2}. Getting them there from Q_p is "2 steps forward". Afterwards they go from Q_{p+2} to Q_{p+1}, which is again 2 steps forward ($Q_{p+2} \to Q_p \to Q_{p+1}$).
- **Moving a tower 2 steps forward** ($Q_p \to Q_{p+2}$): the big disk must make two moves, $Q_p \to Q_{p+1}$ then $Q_{p+1} \to Q_{p+2}$. For the first, park the smaller tower on Q_{p+2}; for the second, on Q_p.

```
procedure C1(k, p)        // move top k disks from Q[p] to Q[p+1]   (1 step forward)
   if k = 0 then return
   C2(k-1, p)             // park smaller disks on Q[p+2]
   move disk k: Q[p] → Q[p+1]
   C2(k-1, p+2)           // smaller disks Q[p+2] → Q[p+1]   (2 steps forward)

procedure C2(k, p)        // move top k disks from Q[p] to Q[p+2]   (2 steps forward)
   if k = 0 then return
   C2(k-1, p)             // park smaller disks on Q[p+2]
   move disk k: Q[p] → Q[p+1]
   C1(k-1, p+2)           // smaller disks Q[p+2] → Q[p]   (1 step forward)
   move disk k: Q[p+1] → Q[p+2]
   C2(k-1, p)             // smaller disks Q[p] → Q[p+2]
```

Move counts. $C_1(0) = C_2(0) = 0$ and

$$C_1(n) = 2C_2(n-1) + 1, \qquad C_2(n) = 2C_2(n-1) + C_1(n-1) + 2.$$

n	$C_1(n)$	$C_2(n)$
1	1	2
2	5	7
3	15	21
4	43	59
5	119	163
6	327	447
7	895	1223
8	2447	3343
9	6687	9135
10	18271	24959

Growth. Substitute $C_1(n-1) = 2C_2(n-2) + 1$: $C_2(n) = 2C_2(n-1) + 2C_2(n-2) + 3$. Characteristic equation $x^2 = 2x + 2$, largest root $x = 1 + \sqrt{3} \approx 2.732$. So both are $\Theta\big((1+\sqrt{3})^n\big) \approx \Theta(2.73^n)$. (These are known to be the fewest moves possible on a 3-peg one-way loop.)

2.5 The 4-peg one-way loop (2008, 2014, 2018)

Set-up: number the pegs along the arrows

Start at Start and follow the arrows, numbering the pegs P_0, P_1, P_2, P_3. So $P_0 =$ Start and the only arrows are $P_0 \to P_1 \to P_2 \to P_3 \to P_0$. Indices are taken *mod 4*: $P_4 = P_0$, $P_5 = P_1$, etc. Let

$$d = \text{the number of Dest} \in \{1,2,3\} = \text{how many arrow-steps from Start to Dest.}$$

Exam	Arrows	Numbering P_0, P_1, P_2, P_3	d
2008	Start→Dest→Aux1→Aux2→Start	Start, Dest, Aux1, Aux2	1
2014	Start→Aux1→Dest→Aux2→Start	Start, Aux1, Dest, Aux2	2
2018	Start→Aux1→Aux2→Dest→Start	Start, Aux1, Aux2, Dest	3

Three procedures: move a tower 1, 2 or 3 steps forward

Write $\text{Go}_s(k, p)$ for "move the top k disks, which form a tower on P_p, to P_{p+s}". The big disk is disk k; the "smaller tower" is disks $1..k-1$.

```
procedure Go1(k, p)          // top k disks:  P[p] → P[p+1]
   if k = 0 then return
   Go2(k-1, p)               // park smaller tower on P[p+2]
   move disk k: P[p] → P[p+1]
   Go3(k-1, p+2)             // smaller tower P[p+2] → P[p+1] (3 steps)

procedure Go2(k, p)          // top k disks:  P[p] → P[p+2]
   if k = 0 then return
   Go3(k-1, p)               // park smaller tower on P[p+3]
   move disk k: P[p]   → P[p+1]
   move disk k: P[p+1] → P[p+2]
   Go3(k-1, p+3)             // smaller tower P[p+3] → P[p+2] (3 steps)

procedure Go3(k, p)          // top k disks:  P[p] → P[p+3]
   if k = 0 then return
   Go2(k-1, p)               // park smaller tower on P[p+2]
   move disk k: P[p] → P[p+1]
   Go2(k-1, p+2)             // smaller tower P[p+2] → P[p]   (2 steps)
   move disk k: P[p+1] → P[p+2]
   move disk k: P[p+2] → P[p+3]
   Go3(k-1, p)               // smaller tower P[p] → P[p+3]

main:  Go_d(n, 0)            // d = 1, 2 or 3 as read off the graph
```

Why every move is legal (this is your correctness argument)

For each procedure, look at where the smaller tower is *every time the big disk moves*. It must be on neither end of the arrow.

Procedure	big disk k moves along	smaller tower is parked on
Go1	$P_p \to P_{p+1}$	P_{p+2} ✓
Go2	$P_p \to P_{p+1}$, then $P_{p+1} \to P_{p+2}$	P_{p+3} (for both moves) ✓
Go3	$P_p \to P_{p+1}$	P_{p+2} ✓
	$P_{p+1} \to P_{p+2}$, then $P_{p+2} \to P_{p+3}$	P_p (for both moves) ✓

And every tower move is a recursive call on $k-1$ disks between the correct pegs: Go1's second call moves the tower from P_{p+2} forward 3 steps to $P_{p+5} = P_{p+1}$; Go2's second call goes from P_{p+3} forward 3 steps to $P_{p+6} = P_{p+2}$; Go3's middle call goes from P_{p+2} forward 2 steps to $P_{p+4} = P_p$. Every single disk move uses an arrow $P_i \to P_{i+1}$. By induction on k (base case $k = 0$: nothing to do), every procedure is correct.

Picture of Go3 (the hardest one)

Stage	disk k is on	smaller tower is on	what happens next
0	P_p	P_p (on top of k)	Go2$(k-1, p)$: tower $P_p \to P_{p+2}$
1	P_p	P_{p+2}	disk k: $P_p \to P_{p+1}$
2	P_{p+1}	P_{p+2}	Go2$(k-1, p+2)$: tower $P_{p+2} \to P_p$
3	P_{p+1}	P_p	disk k: $P_{p+1} \to P_{p+2} \to P_{p+3}$
4	P_{p+3}	P_p	Go3$(k-1, p)$: tower $P_p \to P_{p+3}$
5	P_{p+3}	P_{p+3} (on top of k)	done

Number of moves

Let $T_s(n)$ be the number of single-disk moves made by $\mathrm{Go}_s(n, \cdot)$. Reading the procedures line by line:

$$T_1(n) = T_2(n-1) + T_3(n-1) + 1, \quad T_2(n) = 2T_3(n-1) + 2, \quad T_3(n) = 2T_2(n-1) + T_3(n-1) + 3$$

with $T_1(0) = T_2(0) = T_3(0) = 0$. Values (computed and confirmed by running the algorithm in a move-checking simulator):

n	$T_1(n)$	$T_2(n)$	$T_3(n)$
1	1	2	3
2	6	8	10
3	19	22	29
4	52	60	76
5	137	154	199
6	354	400	510
7	911	1022	1313
8	2336	2628	3360
9	5989	6722	8619
10	15342	17240	22066

Time complexity, two ways

(a) Quick upper bound. Let $M(n) = \max(T_1(n), T_2(n), T_3(n))$. Each procedure makes at most 3 recursive calls on $n-1$ disks plus at most 3 moves, so $M(n) \le 3M(n-1) + 3$, hence $M(n) \le \frac{3^{n+1}-3}{2} = \mathcal{O}(3^n)$.

(b) Exact growth rate. From $T_2(n-1) = 2T_3(n-2) + 2$:

$$T_3(n) = 2T_2(n-1) + T_3(n-1) + 3 = T_3(n-1) + 4T_3(n-2) + 7.$$

Characteristic equation $x^2 = x + 4$, largest root $x = \frac{1+\sqrt{17}}{2} \approx 2.5616$. Since $T_2(n) = 2T_3(n-1) + 2$ and $T_1(n) = T_2(n-1) + T_3(n-1) + 1$ are built from T_3, all three grow at this rate:

$$T_1(n), T_2(n), T_3(n) = \Theta\left(\left(\tfrac{1+\sqrt{17}}{2}\right)^n\right) \approx \Theta(2.56^n)$$

(Check with the table: $3360/1313 \approx 2.559$.)

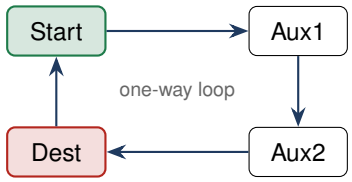
Time vs. moves. Each call does a constant amount of work besides its recursive calls and its moves, and every call with $k \geq 1$ makes at least one move, so the running time is $\Theta(\text{number of moves}) = \Theta(2.56^n)$. The **space** (recursion depth) is $\Theta(n)$.

Watch out: do not overclaim

With 4 pegs, cleverer strategies that split the small tower over two pegs need fewer moves than this (for example, computer search shows the minimum for 2018's graph with 3 disks is 21, while our algorithm uses 29). The exam asks you to *design a function and estimate its complexity*, not to find the minimum. Say "my algorithm makes $T_3(n)$ moves"; do not claim it is optimal.

2.6 Ready-to-copy answers for each 4-peg loop

2.6.1 2018 graph: Dest is 3 steps after Start



On the exam: model answer, 2018

Graph. The arrows form one loop Start→Aux1→Aux2→Dest→Start. Number the pegs along the loop: P_0 =Start, P_1 =Aux1, P_2 =Aux2, P_3 =Dest (indices mod 4). Dest is 3 steps after Start.
Idea. The largest disk can only move when all smaller disks are parked on a peg that is neither end of the arrow it uses. I use three procedures that move a tower of k disks 1, 2, or 3 steps forward along the loop: Go1, Go2, Go3 (copy the pseudocode from page 11). **Call Go3**$(n, 0)$**.**
Correctness. [Copy the "parked on" table.]
Complexity. $T_1(n) = T_2(n-1) + T_3(n-1) + 1$, $T_2(n) = 2T_3(n-1) + 2$, $T_3(n) = 2T_2(n-1) + T_3(n-1) + 3$. Hence $T_3(n) = T_3(n-1) + 4T_3(n-2) + 7$, so $T_3(n) = \Theta(((1+\sqrt{17})/2)^n) \approx \Theta(2.56^n)$; in any case $\mathcal{O}(3^n)$. Time $= \Theta(\#\text{moves})$, space $\Theta(n)$.

Check: all moves for $n = 1$ and $n = 2$ (produced by running the procedures; every move is along an arrow, never a larger disk on a smaller one):

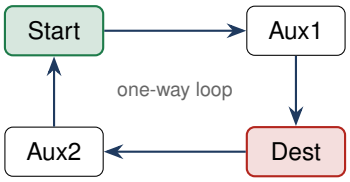
$n = 1$ (3 moves):

1. disk 1: Start→Aux1 2. disk 1: Aux1→Aux2 3. disk 1: Aux2→Dest

$n = 2$ (10 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Aux2
3. disk 2: Start→Aux1
4. disk 1: Aux2→Dest
5. disk 1: Dest→Start
6. disk 2: Aux1→Aux2
7. disk 2: Aux2→Dest
8. disk 1: Start→Aux1
9. disk 1: Aux1→Aux2
10. disk 1: Aux2→Dest

2.6.2 2014 graph: Dest is 2 steps after Start



Numbering: P_0 =Start, P_1 =Aux1, P_2 =Dest, P_3 =Aux2. **Call Go2**$(n,0)$**.** Number of moves $T_2(n)$: $2,8,22,60,154,\ldots$; complexity $\Theta(2.56^n)$, $\mathcal{O}(3^n)$.

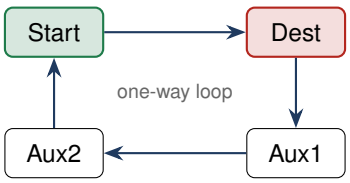
$n=1$ (2 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Dest

$n=2$ (8 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Dest
3. disk 1: Dest→Aux2
4. disk 2: Start→Aux1
5. disk 2: Aux1→Dest
6. disk 1: Aux2→Start
7. disk 1: Start→Aux1
8. disk 1: Aux1→Dest

2.6.3 2008 graph: Dest is 1 step after Start



Numbering: P_0 =Start, P_1 =Dest, P_2 =Aux1, P_3 =Aux2. **Call Go1**$(n,0)$**.** Number of moves $T_1(n)$: $1,6,19,52,137,\ldots$; complexity $\Theta(2.56^n)$, $\mathcal{O}(3^n)$.

$n=1$ (1 move):

1. disk 1: Start→Dest

$n=2$ (6 moves):

1. disk 1: Start→Dest
2. disk 1: Dest→Aux1
3. disk 2: Start→Dest
4. disk 1: Aux1→Aux2
5. disk 1: Aux2→Start
6. disk 1: Start→Dest

$n=3$ (19 moves):

1. disk 1: Start→Dest
2. disk 1: Dest→Aux1
3. disk 1: Aux1→Aux2
4. disk 2: Start→Dest
5. disk 2: Dest→Aux1
6. disk 1: Aux2→Start
7. disk 1: Start→Dest
8. disk 1: Dest→Aux1
9. disk 3: Start→Dest

10. disk 1: Aux1→Aux2
11. disk 1: Aux2→Start
12. disk 2: Aux1→Aux2
13. disk 1: Start→Dest
14. disk 1: Dest→Aux1
15. disk 2: Aux2→Start
16. disk 2: Start→Dest
17. disk 1: Aux1→Aux2
18. disk 1: Aux2→Start
19. disk 1: Start→Dest

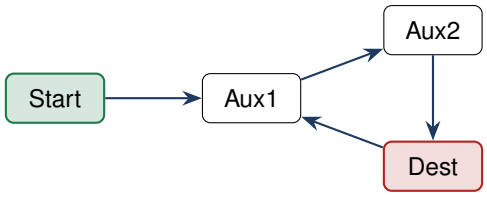
On the exam: a brand-new 4-peg loop

Whatever order the four pegs appear in, the only thing that changes is (1) the names in $P_0..P_3$ and (2) d. Write the numbering, copy the three procedures, call $\text{Go}_d(n, 0)$, copy the complexity paragraph. That is a complete answer.

2.7 Tail-in lollipop (2012)

Exam 1, 2012, Q1

…graph with $V = \{\text{Start}, \text{Aux1}, \text{Dest}, \text{Aux2}\}$ and $E = \{(\text{Start}, \text{Aux1}), (\text{Aux1}, \text{Aux2}), (\text{Aux2}, \text{Dest}), (\text{Dest}, \text{Aux1})\}$…



Shape. No arrow comes *into* Start, so a disk that leaves Start can never come back. The other three pegs form a one-way 3-loop Aux1→Aux2→Dest→Aux1. Aux1 is the *entry* peg; Dest is 2 steps after the entry.

Plan for the largest disk n. It must travel Start→Aux1→Aux2→Dest.

1. Start→Aux1 and Aux1→Aux2: the smaller tower must avoid Start, Aux1, Aux2, so it must sit on **Dest**. (Getting it there from Start is the same problem with $n-1$ disks!)
2. Aux2→Dest: the smaller tower must avoid Aux2 and Dest, and it cannot go back to Start, so it must sit on **Aux1**. Moving it Dest→Aux1 is 1 step on the 3-loop: C_1.
3. Finally move the smaller tower Aux1→Dest: 2 steps on the 3-loop: C_2.

```
// 3-loop  Q[0]=Aux1 → Q[1]=Aux2 → Q[2]=Dest → Q[0];  C1, C2 as on page "Warm-up"
procedure H(k)                // move the top k disks from Start to Dest
   if k = 0 then return
   H(k-1)                     // smaller tower: Start → Dest
   move disk k: Start → Aux1
   move disk k: Aux1  → Aux2
   C1(k-1, 2)                 // smaller tower: Dest → Aux1   (1 step on the loop)
   move disk k: Aux2  → Dest
   C2(k-1, 0)                 // smaller tower: Aux1 → Dest   (2 steps on the loop)
main: H(n)
```

Moves. $H(0) = 0$, $H(n) = H(n-1) + C_1(n-1) + C_2(n-1) + 3$. Values: $3, 9, 24, 63, 168, 453, 1230, \ldots$ (for $n \leq 7$ these are the minimum possible, as confirmed by computer search).

Complexity. $C_1, C_2 = \Theta((1+\sqrt{3})^n)$, and $H(n)$ is a sum of such terms, dominated by the largest (geometric sum), so

$$H(n) = \Theta\big((1+\sqrt{3})^n\big) \approx \Theta(2.73^n) \qquad (\text{and } \mathcal{O}(3^n)).$$

$n = 1$ (3 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Aux2
3. disk 1: Aux2→Dest

$n = 2$ (9 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Aux2
3. disk 1: Aux2→Dest
4. disk 2: Start→Aux1
5. disk 2: Aux1→Aux2
6. disk 1: Dest→Aux1
7. disk 2: Aux2→Dest
8. disk 1: Aux1→Aux2
9. disk 1: Aux2→Dest

Other tail-in lollipops

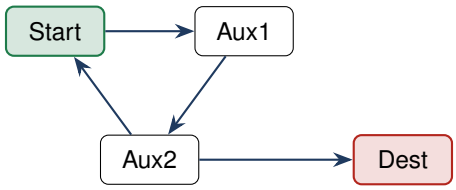
Let E be the entry peg (the one Start points to), and the loop $E \to E^+ \to E^{++} \to E$.

Where is Dest?	Algorithm In(k) (top k disks Start→Dest)	Moves ($n = 1,2,3,4$)
2 steps after E (2012)	as above	3, 9, 24, 63
1 step after E	tower Start→ E^{++} [same as the 2012 problem with $k-1$ disks]; disk k: Start→ E →Dest; tower E^{++} →Dest by C_2	2, 7, 18, 47
Dest $= E$	tower Start→ E^+ [the "1 step after" problem with $k-1$ disks]; disk k: Start→Dest; tower E^+ →Dest by C_2	1, 5, 15, 40

All three are $\Theta((1+\sqrt{3})^n)$.

2.8 Tail-out lollipop (Dest is a dead end)

Example (used in Mock Exam 5): $E = \{(\text{Start}, \text{Aux1}), (\text{Aux1}, \text{Aux2}), (\text{Aux2}, \text{Start}), (\text{Aux2}, \text{Dest})\}$.



Shape. Start, Aux1, Aux2 form a one-way 3-loop; the only way into Dest is from the *exit* peg Aux2, and nothing leaves Dest. Disks that reach Dest stay there forever, so they must arrive **largest first**: disk n, then $n-1, \ldots$, then disk 1.

Plan for disk k (the largest one not yet on Dest; the tower of disks $1..k-1$ is on top of it). Disk k walks along the loop to the exit, then exits. When it moves, the small tower must avoid both ends of the arrow (Dest only holds larger disks, so Dest itself is always fine to land on).

```
// loop Q[0]=Start → Q[1]=Aux1 → Q[2]=Aux2 → Q[0];  exit arrow Aux2 → Dest
procedure Out(k, p)          // top k disks sit on Q[p]; move them all to Dest
   if k = 0 then return
```

```
   case (steps from Q[p] to the exit Aux2) of
     2:  C2(k-1, p)                  // tower Q[p] → Q[p+2] = exit
         move disk k: Q[p] → Q[p+1]
         C1(k-1, p+2)                // tower exit → Q[p]
         move disk k: Q[p+1] → Aux2;  move disk k: Aux2 → Dest
         Out(k-1, p)
     1:  C2(k-1, p)                  // tower Q[p] → Q[p+2]
         move disk k: Q[p] → Aux2;    move disk k: Aux2 → Dest
         Out(k-1, p+2)
     0:  C1(k-1, p)                  // tower Q[p] → Q[p+1]
         move disk k: Aux2 → Dest
         Out(k-1, p+1)
main: Out(n, 0)    // Start = Q[0] is 2 steps from the exit Aux2
```

For this graph only case 2 is ever used (the tower always returns to Start), so $\mathrm{Out}(n) = \mathrm{Out}(n-1) + C_1(n-1) + C_2(n-1) + 3$: $3, 9, 24, 63, \cdots = \Theta((1+\sqrt{3})^n)$. It is the mirror image of the 2012 graph. If the exit is Aux1 or Start itself, the other cases are used; the moves are $2, 7, 18, 47, \ldots$ and $1, 5, 15, 40, \ldots$ respectively, still $\Theta((1+\sqrt{3})^n)$.

$n = 2$ (9 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Aux2
3. disk 2: Start→Aux1
4. disk 1: Aux2→Start
5. disk 2: Aux1→Aux2
6. disk 2: Aux2→Dest
7. disk 1: Start→Aux1
8. disk 1: Aux1→Aux2
9. disk 1: Aux2→Dest

2.9 Other shapes

Only three pegs

Graph	Algorithm	Moves / complexity
All six arrows (classic game)	Section 1.2	$2^n - 1, \Theta(2^n)$
One-way loop Start→Dest→Aux→Start	$C_1(n, \text{Start})$	$\Theta((1+\sqrt{3})^n)$
One-way loop Start→Aux→Dest→Start	$C_2(n, \text{Start})$	$\Theta((1+\sqrt{3})^n)$
Line Start↔Aux↔Dest (two-way, no Start–Dest edge)	see below	$3^n - 1, \Theta(3^n)$

Line graph. Disk k must go Start→Aux→Dest, and at both moves the small tower must be on the third peg:

```
procedure L(k, X, Y)        // X, Y = the two end pegs, M = middle peg
   if k = 0 then return
   L(k-1, X, Y)             // small tower X → Y (out of the way)
   move disk k: X → M
   L(k-1, Y, X)             // small tower Y → X
   move disk k: M → Y
   L(k-1, X, Y)             // small tower X → Y
```

$L(n) = 3L(n-1) + 2$, so $L(n) = 3^n - 1 = \Theta(3^n)$.

Two-way (undirected) edges

An undirected edge $\{X, Y\}$ means arrows both ways. If the 4 pegs form a two-way loop, you can still use the one-way algorithm (ignore one direction): $\Theta(2.56^n)$. If they form a two-way line Start–Aux1–

Aux2–Dest, treat the two middle pegs as a "middle" and adapt the line idea; any correct recursive algorithm with its recurrence earns the points.

Graphs where the game cannot be solved

Leiss has published research papers on Hanoi problems on graphs, and he likes students to notice when a graph does not work. If the disks can never come back to a peg once they leave it (no loop that contains at least 3 pegs that the disks can use), then 2 or more disks generally cannot be moved. Example: a one-way line Start→Aux1→Aux2→Dest. Disk 1 must leave Start first and can never return; for disk 2 to go Start→Aux1 the small disk must be on Aux2 or Dest, and then disk 2 can never pass it. If you get such a graph, say so and explain why.

2.10 Grading notes and common mistakes

- **Never use a move that is not an arrow.** The most common error: moving a disk "directly" from Start to Dest when there is no arrow (Start,Dest).
- **Count every single-disk move.** If the big disk has to cross two arrows, that is two moves.
- **Base case.** For $n = 1$ the disk simply walks along the arrows to Dest; every move must be necessary and along an arrow. (On a graded 2012 exam, 3 points were taken off next to the grader's comment "why twice move?" on the base case.)
- **State the recurrence(s) and solve or bound them.** A full-credit 2018 answer used one recursive function with 3 recursive calls and wrote $F(n) = \mathcal{O}(3^n)$; our answer is more precise and also correct.
- **Say which procedure the main program calls**, with which peg number.

PART III

Question 2: Counting Page Transfers

CHAPTER 3

Virtual Memory, Pages and LRU

3.1 The question

Exam 1, 2018, Q2 (25%) — identical in 2014

Determine for the following code how many pages are transferred between disk and main memory. Assume each page has 1024 words, the active memory set size is 1024 (i.e., at any time no more than 1024 pages may be in main memory), and the replacement strategy is LRU (the Least Recently Used page is always replaced); also assume that all 2D arrays are of size (1:2048, 1:2048), with each array element occupying one word,

```
for I := 1 to 2048 do
   for J := 1 to 2048 do
      { A[J,I] := A[J,I] * B[I,J] }
```

provided the arrays are mapped into the main memory space in **row**-major order.

Answer: 8,392,704 transfers (worked out in Section 3.7). The rest of this chapter explains why, so that you can handle any changed numbers.

3.2 What is happening inside the computer

In plain English

- The program's data lives on the **disk** (huge, slow). Only a limited part of it can be in **main memory** (RAM: small, fast) at any time.
- Data moves between disk and RAM in fixed-size chunks called **pages** (here: 1024 words per page).
- RAM has room for a fixed number of pages, the **page frames**. Leiss calls this number the **active memory set size** (here: 1024 pages).
- When the program touches a word whose page is *not* in RAM, that is a **page fault**: the whole page is **read** from disk into RAM. **That is 1 transfer.**
- If RAM is already full, some page must be thrown out first. **LRU** (Least Recently Used) throws out the page whose *last use* is longest ago.
- If the page being thrown out was **changed** while in RAM (it is "**dirty**"), it must be **written** back to disk: **1 more transfer**. A page that was only read ("clean") is simply dropped: no transfer.

Key idea

$$\text{total transfers} = (\text{page reads}) + (\text{page writes})$$

Page reads: one per page fault. Page writes: one for each time a dirty page leaves memory (evicted,

or saved at the end). An array that appears on the **left** of := is modified, so its pages become dirty and each page-in of it is eventually matched by one page-out. An array that only appears on the right is only read: **no writes**.

Watch out

- One iteration of the loop is *not* "3 transfers". A transfer happens only when a *page* is missing (or a dirty page is evicted). Many iterations can use the same page for free.
- The arithmetic ($*$) happens in the CPU and costs no transfers.
- Leiss's graded answers count the writes too, including the dirty pages still in memory at the end (they must be saved eventually). Do the same.

3.3 How a 2D array is laid out in memory

Memory is one long line of words. A 2D array has to be flattened into that line. There are two ways:

- **Row-major** (C, C++, Pascal): store row 1 completely, then row 2, then row 3, ...
- **Column-major** (Fortran): store column 1 completely, then column 2, ...

Small example: a 4×4 array, pages of 2 words (so each row, or each column, fills 2 pages). Numbers show the storage order; colors show pages.

Row-major

	col 1	col 2	col 3	col 4
row 1	0 pg 0	1 pg 0	2 pg 1	3 pg 1
row 2	4 pg 2	5 pg 2	6 pg 3	7 pg 3
row 3	8 pg 4	9 pg 4	10 pg 5	11 pg 5
row 4	12 pg 6	13 pg 6	14 pg 7	15 pg 7

Column-major

	col 1	col 2	col 3	col 4
row 1	0 pg 0	4 pg 2	8 pg 4	12 pg 6
row 2	1 pg 0	5 pg 2	9 pg 4	13 pg 6
row 3	2 pg 1	6 pg 3	10 pg 5	14 pg 7
row 4	3 pg 1	7 pg 3	11 pg 5	15 pg 7

Address formulas for an $N \times N$ array `A[1:N,1:N]` (offset counted from 0, page size P words):

$$\text{row-major: } \text{offset}(i,j) = (i-1)N + (j-1), \qquad \text{column-major: } \text{offset}(i,j) = (j-1)N + (i-1),$$

$$\text{page number} = \left\lfloor \frac{\text{offset}}{P} \right\rfloor.$$

Key idea: what lies next to what

- **Row-major:** elements of the *same row* are neighbors. Going along a row (changing the **second** index) walks through memory word by word. Going down a column (changing the **first** index) jumps N words each step.
- **Column-major:** the opposite. Changing the **first** index walks; changing the **second** index jumps.

- With $N = 2048$ and $P = 1024$: one row (row-major) or one column (column-major) is $2048/1024 = 2$ pages. Each array has $2048 \times 2 = 4096$ pages in total.

3.4 Walking versus jumping

Look at each array reference in the loop body and ask: **in which position (first or second subscript) is the inner loop variable?**

	inner variable in **second** position, e.g. `B[I,J]`	inner variable in **first** position, e.g. `A[J,I]`
row-major	**walking** along a row	**jumping** down a column
column-major	**jumping** across a row	**walking** down a column

- **Walking** reference: during one full inner loop it uses only N/P pages (2 pages for $N = 2048$, $P = 1024$), each for P consecutive iterations. Each of those pages is read **once** and never needed again later, so over the whole program a walking array costs $N \cdot \frac{N}{P}$ reads = **the number of pages of the array** (4096).
- **Jumping** reference: each iteration of the inner loop lands in a **different page** (a different row, or a different column). During one inner loop it touches N different pages (2048). The question is whether those pages are *still in RAM* when the next outer iteration needs them again (the next column $I + 1$ lies in the same pages as column I, as long as both are in the same half). That is decided by LRU:

3.5 The LRU reuse rule

Key idea: when does LRU keep a page?

Under LRU with F frames, a page that was used at some moment is **still in RAM at its next use** if and only if **fewer than F other distinct pages** were used in between.

For a jumping reference, between two uses of the same page (outer iteration I and $I + 1$) the program touches *all the other pages of the sweep*: about N pages of the jumping array plus a few pages of the other arrays. So:

- If F is **smaller** than the number of pages one inner loop touches (the usual exam case: 2048 pages needed, 1024 or 512 frames), every page is evicted just before it is needed again: **every single access of the jumping array is a page fault**. Reads $= N \cdot N$ (4,194,304 for $N = 2048$). This is the famous bad case of LRU: a cyclic sweep that is larger than memory makes LRU miss *every* time.
- If F is **comfortably larger** than the pages touched in one inner loop (e.g. $F = 4096$), the pages survive and each page of the jumping array is read only once: reads $= N \cdot \frac{N}{P}$, same as walking.

Watch out: do not divide by F

The number of frames is *not* something you divide by. With LRU and a sweep that does not fit, having 1024 frames instead of 512 helps *nothing*: still every access misses. Frames matter only

through the yes/no question "does the sweep fit?".

3.6 The 5-step recipe

Recipe: page transfers

1. **Pages per row/column:** $R = N/P$ (e.g. $2048/1024 = 2$). Pages per array: $N \cdot R$.
2. **Classify every array reference** as walking or jumping using the table above (look at where the *inner* loop variable sits, and at row/column-major).
3. **Pages touched per inner loop:** walking $= R$, jumping $= N$. Add them up over all arrays.
4. **Fit test:** if that sum is $\geq F$ (frames), jumping pages are *not* reused (every jumping access faults). If it is well below F, every page is read only once.
5. **Count:**
   - walking array: $N \cdot R$ reads;
   - jumping array, does not fit: $N \cdot N$ reads; fits: $N \cdot R$ reads;
   - each array that is assigned to (left of :=): writes = its reads;
   - total = all reads + all writes.

3.7 2018 / 2014, fully worked: row-major, 1024 frames

Data: $N = 2048$, $P = 1024$, $F = 1024$, row-major, `A[J,I] := A[J,I] * B[I,J]`, I outer, J inner.

Step 1. A row has 2048 words = 2 pages. Each array: $2048 \cdot 2 = 4096$ pages.

Step 2. `A[J,I]`: inner variable J is in the *first* position, row-major ⇒ **jumping** (one element from each row). `B[I,J]`: J in the second position ⇒ **walking** along row I of B.

Step 3. One inner loop (fixed I) touches 2048 pages of A (the page holding column I of every row J) and 2 pages of B (row I). Total 2050.

Step 4. $2050 \geq 1024$: does not fit. Concretely, take $I = 1$: the program uses $A[1,1], A[2,1], \ldots, A[2048,1]$, i.e. the first page of each of the 2048 rows. When $I = 2$ starts, it needs $A[1,2]$, which is on the *same* page as $A[1,1]$ (the first page of row 1). But since that page was last used, 2047 other A-pages and 2 B-pages were used, more than $F - 1 = 1023$. LRU has thrown it out. The same happens for every page, every time.

Step 5.

Transfers	Count	Why
A reads	$2048 \cdot 2048 = 4{,}194{,}304$	every access to A faults
A writes	$4{,}194{,}304$	every page brought in is modified, written back once
B reads	$2048 \cdot 2 = 4{,}096$	2 pages per row, 2048 rows, each read once
B writes	0	B is never modified
Total	**8,392,704**	$= (2048 + 2048 + 2) \cdot 2048$

This matches the full-credit answer on the graded 2018 exam: "$(2048 + 2048 + 2) \times 2048 = 8{,}392{,}704$".

On the exam: model answer, 2018

Row-major: each row of 2048 words occupies $2048/1024 = 2$ pages. For fixed I, the inner loop accesses $A[1, I], \ldots, A[2048, I]$, one element in each of 2048 different rows, hence 2048 different pages of A; it accesses $B[I, 1], \ldots, B[I, 2048]$, which is row I of B, i.e. 2 pages. Since $2048 + 2 > 1024$ frames, under LRU each A-page is replaced before it is used again in the next iteration of I (2047 other A-pages and 2 B-pages are used in between). So every A access causes a page read, and since A is modified each such page is later written back. Per iteration of I: 2048 reads of A + 2048 writes of A + 2 reads of B. Total: $(2048 + 2048 + 2) \cdot 2048 =$ **8,392,704** page transfers.

3.8 2012 and 2008: smaller numbers, same pattern

2012: pages of 512 words, $F = 256$, arrays 1024×1024, row-major, same loop. $R = 1024/512 = 2$. Per inner loop: 1024 A-pages (jumping) + 2 B-pages; $1026 \geq 256$, does not fit.

$$\text{Total} = (1024 + 1024 + 2) \cdot 1024 = \textbf{2,099,200}.$$

2008: pages of 512 words, $F = 512$, arrays 1024×1024, row-major, same loop. Per inner loop 1026 pages ≥ 512: does not fit. Same total, **2,099,200**. (Both graded answers had exactly this and received full credit. Notice the number of frames did not matter.)

3.9 2020: column-major (the mirror image)

Data: $N = 2048$, $P = 1024$, $F = 512$, **column-major**, same loop.

- `A[J,I]`: J in the first position, column-major $\Rightarrow$ **walking** down column I: 2 pages per inner loop.
- `B[I,J]`: J in the second position, column-major $\Rightarrow$ **jumping**: one element from each of 2048 columns, 2048 pages per inner loop. Does not fit in 512 frames.

A reads	$2048 \cdot 2 = 4{,}096$
A writes	4,096
B reads	$2048 \cdot 2048 = 4{,}194{,}304$
B writes	0
Total	**4,202,496**

It is smaller than the row-major answer because now the *jumping* array is B, which is never written back.

3.10 2008 Exam 2: a different loop body

Data: pages of 128 words, $F = 100$, arrays 256×256, **column-major**:

```
for I := 1 to 256 do
   for J := 1 to 256 do
      { A[I,J] := A[I,J] * B[N-I+1,J] }
```

$R = 256/128 = 2$ pages per column. Inner variable J is in the *second* position of both references; column-major ⇒ both are **jumping** (fixed row, moving across columns). Per inner loop: 256 pages of A + 256 pages of $B = 512 \geq 100$: does not fit. (The row $N - I + 1$ of B runs backwards, but that does not matter: it is still one element per column.)

A reads $256 \cdot 256 = 65{,}536$; A writes 65,536; B reads 65,536; Total $= 3 \cdot 65{,}536 =$ **196,608**.

(The graded answer was exactly $3 \cdot 256 \cdot 256 = 196{,}608$, full credit.)

3.11 Lookup table: common variations

Every row below was produced by simulating LRU on the full-size arrays, counting reads and write-backs (including the final flush). "Total" is the answer. (With 512 frames instead of 1024 the 2048-rows are identical: also simulated.)

N	page	frames	order	loop body (I outer, J inner)	A reads	A writes	B reads	total
2048	1024	1024	row	`A[J,I] := A[J,I] * B[I,J]`	4,194,304	4,194,304	4,096	**8,392,704**
2048	1024	1024	row	`A[I,J] := A[I,J] * B[J,I]`	4,096	4,096	4,194,304	**4,202,496**
2048	1024	1024	row	`A[I,J] := A[I,J] * B[I,J]`	4,096	4,096	4,096	**12,288**
2048	1024	1024	row	`A[J,I] := A[J,I] * B[J,I]`	4,194,304	4,194,304	4,194,304	**12,582,912**
2048	1024	1024	col	`A[J,I] := A[J,I] * B[I,J]`	4,096	4,096	4,194,304	**4,202,496**
2048	1024	1024	col	`A[I,J] := A[I,J] * B[J,I]`	4,194,304	4,194,304	4,096	**8,392,704**
2048	1024	1024	col	`A[I,J] := A[I,J] * B[I,J]`	4,194,304	4,194,304	4,194,304	**12,582,912**
2048	1024	1024	col	`A[J,I] := A[J,I] * B[J,I]`	4,096	4,096	4,096	**12,288**
1024	512	256	row	`A[J,I] := A[J,I] * B[I,J]`	1,048,576	1,048,576	2,048	**2,099,200**
1024	512	256	row	`A[I,J] := A[I,J] * B[J,I]`	2,048	2,048	1,048,576	**1,052,672**
1024	512	256	row	`A[I,J] := A[I,J] * B[I,J]`	2,048	2,048	2,048	**6,144**
1024	512	256	row	`A[J,I] := A[J,I] * B[J,I]`	1,048,576	1,048,576	1,048,576	**3,145,728**
1024	512	256	col	`A[J,I] := A[J,I] * B[I,J]`	2,048	2,048	1,048,576	**1,052,672**
1024	512	256	col	`A[I,J] := A[I,J] * B[J,I]`	1,048,576	1,048,576	2,048	**2,099,200**
1024	512	256	col	`A[I,J] := A[I,J] * B[I,J]`	1,048,576	1,048,576	1,048,576	**3,145,728**
1024	512	256	col	`A[J,I] := A[J,I] * B[J,I]`	2,048	2,048	2,048	**6,144**

Patterns to notice:

- Switching row-major ↔ column-major has the same effect as swapping the two subscripts.
- If both references walk, the answer is tiny (each page once): $3 \cdot 4096 = 12{,}288$.
- The answer is largest when the *modified* array jumps (reads and writes both N^2).

3.12 Edge cases and how to handle them

When memory is big enough

Suppose the frames are many, e.g. $F = 4096$ with $N = 2048$, $P = 1024$, row-major and the 2018 loop. One inner loop touches $2048 + 2 = 2050$ pages, which is less than 4096. Then each A-page used for column I is still in RAM for column $I + 1$. Each A-page is read once (when column 1 or column 1025 first needs it): A reads = 4096, A writes = 4096, B reads = 4096. Total 12,288. (Checked by simulation; this is Mock Exam 2, Q2 in the column-major version.)

Borderline case: the exact rule is "fewer than F distinct other pages between two uses". If the count is right at the boundary, carefully count the pages used between two uses of the same page. Leiss's exams have always had numbers that are far from the boundary.

Page larger than a row

If $P > N$ (e.g. $N = 1024$, $P = 2048$), one page holds P/N whole rows (row-major). A jumping reference down a column then touches only $N/(P/N) = N^2/P$ distinct pages per inner loop, because consecutive rows share pages. Apply the fit test to that number.

Array only written, never read

In `A[I,J] := B[J,I] + C[I,J]`, A is written without being read. A page still has to be in memory to be written into, so count the page-in *and* the write-back, and state this assumption in your answer.

Loops in the other order

If the loop is `for J ... for I ...` (J outer, I inner), then I is the inner variable; redo Step 2 with I. Always find the **inner** variable first.

PART IV

Question 3: QuickSort Best and Worst Inputs

CHAPTER 4

QuickSort: Building the Best and the Worst Input

4.1 The question

Exam 1, 2018, Q3 (25%)

Consider QuickSort on the array $A[1{:}n]$ and assume that the pivot element x (used to split the array $A[lo{:}hi]$ into two portions such that all elements in the left portion $A[lo{:}m]$ are $\leq x$ and all elements in the right portion $A[m{:}hi]$ are $\geq x$) is the **first** element of the array to be split (i.e., $A[lo]$). For a specific value of n at least 13 (**your** choice), construct an assignment of the numbers $1 \ldots n$ to the n array elements that causes QuickSort, with the stated choice of pivot, to (a) execute optimally (that is $A[lo{:}m]$ and $A[m{:}hi]$ are always of equal size) (b) execute in the slowest possible way.

Other years changed only the pivot position: **last** element $A[hi]$ (2014), **second** element $A[lo+1]$ (2020), **next-to-last** $A[hi-1]$ (2021), and the minimum n (11, 13, or 14).

4.2 QuickSort from zero

In plain English

To sort a list: pick one element, the **pivot**. Rearrange the list so that everything smaller than the pivot is on its left and everything larger is on its right; now the pivot is in its final sorted position. Then sort the left part and the right part the same way (recursion). A part of size 0 or 1 is already sorted (base case).

```
procedure QuickSort(A, lo, hi)
   if lo < hi then
      m := Partition(A, lo, hi)      // pivot ends at position m
      QuickSort(A, lo, m-1)          // left part:  everything < pivot
      QuickSort(A, m+1, hi)          // right part: everything > pivot

procedure Partition(A, lo, hi)
   q := position of the pivot       // lo, lo+1, hi, hi-1, ... (as stated)
   swap A[lo] and A[q]               // step 1: bring the pivot to the front
   x := A[lo];  i := lo              // i = end of the "smaller than x" zone
   for j := lo+1 to hi do            // step 2: scan everything else once
      if A[j] < x then
         i := i + 1
         swap A[i] and A[j]          //   grow the "smaller" zone
   swap A[lo] and A[i]               // step 3: put the pivot between the zones
   return i
```

Partition in slow motion

Array $[5,8,2,9,1,7,3]$, pivot = first element $x = 5$. The "smaller zone" is positions 2..i.

j	$A[j]$	< 5?	array afterwards (i = end of smaller zone)
2	8	no	$[5,8,2,9,1,7,3]$, $i = 1$
3	2	yes	$i = 2$, swap $A[2], A[3]$: $[5,2,8,9,1,7,3]$
4	9	no	$[5,2,8,9,1,7,3]$, $i = 2$
5	1	yes	$i = 3$, swap $A[3], A[5]$: $[5,2,1,9,8,7,3]$
6	7	no	$[5,2,1,9,8,7,3]$, $i = 3$
7	3	yes	$i = 4$, swap $A[4], A[7]$: $[5,2,1,3,8,7,9]$
end			swap $A[1], A[4]$: $[3,2,1,\boxed{5},8,7,9]$, return $m = 4$

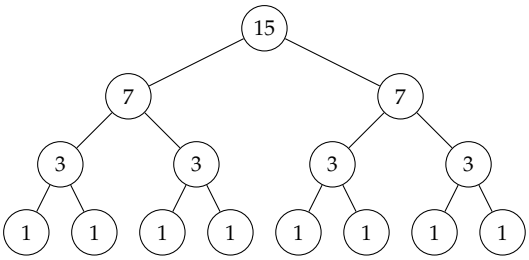
Left part $[3,2,1]$ (all < 5), right part $[8,7,9]$ (all > 5). Partition compared the pivot with each of the other 6 elements exactly once.

Key idea: the cost of QuickSort

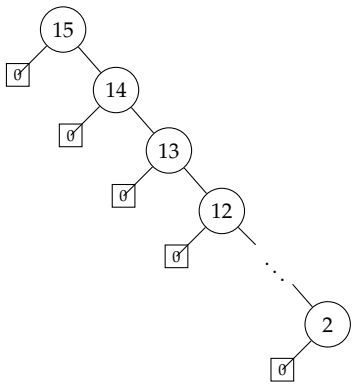
Partitioning a part of size s costs $s-1$ comparisons. The total cost depends entirely on **how evenly the pivot splits** each part.

4.3 Best case vs. worst case

Best: pivot = median every time



Worst: pivot = min (or max)



Best case. If the pivot is always the **median** of its part, a part of size s splits into two parts of size $(s-1)/2$:

$$C(n) = (n-1) + 2\,C\left(\tfrac{n-1}{2}\right), \qquad C(1) = 0.$$

$C(3) = 2$, $C(7) = 6 + 2 \cdot 2 = 10$, $C(15) = 14 + 2 \cdot 10 = 34$, $C(31) = 30 + 2 \cdot 34 = 98$. In general, for $n = 2^k - 1$: $C(n) = (n+1)\log_2(n+1) - 2n = \Theta(n \log n)$. Recursion depth $\log_2(n+1)$.

Worst case. If the pivot is always the **smallest** (or always the largest) element of its part, one side is

empty and the other has $s-1$ elements:

$$C(n) = (n-1) + C(n-1), \quad C(1) = 0 \quad \Longrightarrow \quad C(n) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = \Theta(n^2).$$

For $n = 15$: 105 comparisons (vs. 34 in the best case). Recursion depth $n-1$.

Watch out: "slowest possible" means an EMPTY side every time

A split of 1 vs. $n-2$ is also $\Theta(n^2)$, but it is *not* the slowest possible. The slowest possible execution has the pivot equal to the minimum or maximum of the current part *at every step*. For pivot = first or last element, a sorted array does this; for the second or next-to-last element a sorted array does **not** (the pivot would be the 2nd smallest or 2nd largest).

4.4 Why we choose $n = 15$

"Equal size" at every split means a part of size s splits into two parts of size $(s-1)/2$ (the pivot itself sits between them). Going down: $1 \leftarrow 3 \leftarrow 7 \leftarrow 15 \leftarrow 31 \leftarrow 63$, i.e. $n = 2^k - 1$. So:

exam says "n at least ..."	choose
11, 13, 14, or 15	$n = 15$
16 to 31	$n = 31$ (arrays on page 36)

On the exam

Write one sentence explaining the choice: "For both parts to have equal size at every level, n must be of the form $2^k - 1$; I choose $n = 15$, which splits $15 \to 7+7 \to 3+3 \to 1+1$."

4.5 The construction recipe

Key idea: what Partition really does to a well-prepared part

Suppose a part looks like this *after step 1* (pivot at the front):

$$[\, x,\ \underbrace{s_1, s_2, \ldots, s_k}_{\text{all smaller than } x},\ \underbrace{b_1, \ldots, b_k}_{\text{all larger}}\,].$$

Then in step 2 every swap is an element with itself (nothing moves), and step 3 swaps x with s_k. The result is

$$[\, s_k,\ s_1, \ldots, s_{k-1},\ \boxed{x},\ b_1, \ldots, b_k\,].$$

So the **right part comes out unchanged**, and the **left part comes out with its last element moved to the front**.

Recipe: best-case array (build from the inside out)

Let Good(S) be the array for a set S of $2^k - 1$ consecutive numbers.

1. One number: Good$(\{a\}) = [a]$.

2. Otherwise let x be the **median** of S, L the smaller half, U the larger half. Build $G = \text{Good}(L)$ and $H = \text{Good}(U)$.
3. Because the left part will come out "last element first", write G with its **first element moved to the end**: $G' = [g_2, g_3, \ldots, g_k, g_1]$.
4. Form $[\, x,\ G',\ H\,]$ (this is the array *after* step 1 of Partition).
5. **Undo step 1:** swap the first entry with the entry at the pivot position q (for pivot = first element, $q = lo$ and nothing changes).

Example (pivot = first element):

$$\text{Good}(\{1,2,3\}) = [2,1,3], \qquad \text{Good}(\{5,6,7\}) = [6,5,7],$$
$$\text{Good}(\{1..7\}) = [4,\ \underbrace{1,3,2}_{\text{[2,1,3] rotated}},\ 6,5,7], \qquad \text{Good}(\{9..15\}) = [12,\ 9,11,10,\ 14,13,15],$$
$$\text{Good}(\{1..15\}) = [8,\ \underbrace{1,3,2,6,5,7,4}_{\text{[4,1,3,2,6,5,7] rotated}},\ 12,9,11,10,14,13,15].$$

This is exactly the array on the 2018 and 2008 graded exams that received full credit.

Recipe: worst-case array

Make the element in the pivot position the **minimum** of every part it will be in. With pivot = minimum, Partition changes nothing except (for pivot positions other than the first) the step-1 swap, and the right part is everything else. Build from the outside in:

1. Start with $[\,\text{min},\ \text{Worst}(\text{rest})\,]$ (array after step 1).
2. Undo step 1 by swapping the first entry with the entry at the pivot position.

For pivot = first element this gives simply the sorted array $1, 2, \ldots, n$.

On the exam: what to write for full credit

1. State n and why ($n = 2^k - 1$).
2. State the partition procedure you assume (copy the pseudocode, 6 lines).
3. Give the array for (a). Show the trace: each part, its pivot, and the two parts it splits into (copy the trace table). Conclude $C(n) = 2C((n-1)/2) + n - 1 = \Theta(n \log n)$.
4. Give the array for (b), say the pivot is always the minimum (or maximum), show the first few splits ($14, 13, 12, \ldots$), conclude $C(n) = C(n-1) + n - 1 = n(n-1)/2 = \Theta(n^2)$.

4.6 Pivot = first element $A[lo]$ (2008, 2012, 2018)

Ready answer, pivot = first element, $n = 15$

(a) Optimal: $\langle\, 8, 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15 \,\rangle$ (34 comparisons)
(b) Slowest: $\langle\, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 \,\rangle$ (105 comparisons; the descending array $15, 14, \ldots, 1$ also works)

Trace of (a). Each row is one call of Partition (parts of size 1 need no call).

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	**8** 1 3 2 6 5 7 4 12 9 11 10 14 13 15	8	4 1 3 2 6 5 7 **8** 12 9 11 10 14 13 15	7 / 7
2	1..7	**4** 1 3 2 6 5 7	4	2 1 3 **4** 6 5 7	3 / 3
3	1..3	**2** 1 3	2	1 **2** 3	1 / 1
4	5..7	**6** 5 7	6	5 **6** 7	1 / 1
5	9..15	**12** 9 11 10 14 13 15	12	10 9 11 **12** 14 13 15	3 / 3
6	9..11	**10** 9 11	10	9 **10** 11	1 / 1
7	13..15	**14** 13 15	14	13 **14** 15	1 / 1

Trace of (b). The pivot is always the smallest element; the left part is always empty.

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	**1** 2 3 4 5 6 7 8 9 10 11 12 13 14 15	1	**1** 2 3 4 5 6 7 8 9 10 11 12 13 14 15	0 / 14
2	2..15	**2** 3 4 5 6 7 8 9 10 11 12 13 14 15	2	**2** 3 4 5 6 7 8 9 10 11 12 13 14 15	0 / 13
3	3..15	**3** 4 5 6 7 8 9 10 11 12 13 14 15	3	**3** 4 5 6 7 8 9 10 11 12 13 14 15	0 / 12
4	4..15	**4** 5 6 7 8 9 10 11 12 13 14 15	4	**4** 5 6 7 8 9 10 11 12 13 14 15	0 / 11
5	5..15	**5** 6 7 8 9 10 11 12 13 14 15	5	**5** 6 7 8 9 10 11 12 13 14 15	0 / 10
6	6..15	**6** 7 8 9 10 11 12 13 14 15	6	**6** 7 8 9 10 11 12 13 14 15	0 / 9
7	7..15	**7** 8 9 10 11 12 13 14 15	7	**7** 8 9 10 11 12 13 14 15	0 / 8
8	8..15	**8** 9 10 11 12 13 14 15	8	**8** 9 10 11 12 13 14 15	0 / 7
9	9..15	**9** 10 11 12 13 14 15	9	**9** 10 11 12 13 14 15	0 / 6
10	10..15	**10** 11 12 13 14 15	10	**10** 11 12 13 14 15	0 / 5
11	11..15	**11** 12 13 14 15	11	**11** 12 13 14 15	0 / 4
12	12..15	**12** 13 14 15	12	**12** 13 14 15	0 / 3
13	13..15	**13** 14 15	13	**13** 14 15	0 / 2
14	14..15	**14** 15	14	**14** 15	0 / 1

4.7 Pivot = last element $A[hi]$ (2014)

Ready answer, pivot = last element, $n = 15$ (Partition as above: pivot swapped to the front first)

(a) Optimal: $\langle 12, 1, 2, 3, 7, 5, 4, 6, 14, 9, 10, 11, 15, 13, 8 \rangle$ (34 comparisons)

(b) Slowest: $\langle 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 \rangle$ (105 comparisons; pivot is always the maximum)

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	12 1 2 3 7 5 4 6 14 9 10 11 15 13 **8**	8	6 1 2 3 7 5 4 **8** 14 9 10 11 15 13 12	7 / 7
2	1..7	6 1 2 3 7 5 **4**	4	3 1 2 **4** 7 5 6	3 / 3
3	1..3	3 1 **2**	2	1 **2** 3	1 / 1
4	5..7	7 5 **6**	6	5 **6** 7	1 / 1
5	9..15	14 9 10 11 15 13 **12**	12	11 9 10 **12** 15 13 14	3 / 3
6	9..11	11 9 **10**	10	9 **10** 11	1 / 1
7	13..15	15 13 **14**	14	13 **14** 15	1 / 1

(b): For the sorted array the last element is the maximum of every part; after Partition the left part is again sorted (1..14, then 1..13, ...) and the right part is empty.

Alternative: the textbook (Lomuto/CLRS) partition that keeps the pivot at the end

Many textbooks partition with the pivot staying at the end: `x := A[hi]; i := lo-1; for j := lo to hi-1: if A[j] <= x then i := i+1, swap A[i],A[j]; swap A[i+1],A[hi]`. If that is the version in your notes, use these arrays instead (also checked by running them):

Pivot = last element, textbook partition, $n = 15$

(a) Optimal: $\langle 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15, 8 \rangle$
(b) Slowest: $\langle 1, 2, 3, \ldots, 15 \rangle$ (sorted)

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	1 3 2 6 5 7 4 12 9 11 10 14 13 15 8	8	1 3 2 6 5 7 4 **8** 9 11 10 14 13 15 12	7 / 7
2	1..7	1 3 2 6 5 7 4	4	1 3 2 **4** 5 7 6	3 / 3
3	1..3	1 3 2	2	1 **2** 3	1 / 1
4	5..7	5 7 6	6	5 **6** 7	1 / 1
5	9..15	9 11 10 14 13 15 12	12	9 11 10 **12** 13 15 14	3 / 3
6	9..11	9 11 10	10	9 **10** 11	1 / 1
7	13..15	13 15 14	14	13 **14** 15	1 / 1

4.8 Pivot = second element $A[lo + 1]$ (2020)

Ready answer, pivot = second element, $n = 15$

(a) Optimal: $\langle 4, 8, 3, 1, 5, 6, 7, 2, 10, 12, 11, 9, 13, 14, 15 \rangle$
(b) Slowest: $\langle 15, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \rangle$

Why (b) works: the second position holds 1, the smallest. After Partition the right part is $15, 2, 3, \ldots, 14$, whose second element 2 is again the smallest, and so on. A sorted array would *not* be slowest here (its second element is the 2nd smallest, giving splits 1 vs. $n - 2$).

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	4 8 3 1 5 6 7 2 10 12 11 9 13 14 15	8	2 4 3 1 5 6 7 **8** 10 12 11 9 13 14 15	7 / 7
2	1..7	2 4 3 1 5 6 7	4	1 2 3 **4** 5 6 7	3 / 3
3	1..3	1 2 3	2	1 **2** 3	1 / 1
4	5..7	5 6 7	6	5 **6** 7	1 / 1
5	9..15	10 12 11 9 13 14 15	12	9 10 11 **12** 13 14 15	3 / 3
6	9..11	9 10 11	10	9 **10** 11	1 / 1
7	13..15	13 14 15	14	13 **14** 15	1 / 1

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	15 1 2 3 4 5 6 7 8 9 10 11 12 13 14	1	**1** 15 2 3 4 5 6 7 8 9 10 11 12 13 14	0 / 14
2	2..15	15 2 3 4 5 6 7 8 9 10 11 12 13 14	2	**2** 15 3 4 5 6 7 8 9 10 11 12 13 14	0 / 13
3	3..15	15 3 4 5 6 7 8 9 10 11 12 13 14	3	**3** 15 4 5 6 7 8 9 10 11 12 13 14	0 / 12
4	4..15	15 4 5 6 7 8 9 10 11 12 13 14	4	**4** 15 5 6 7 8 9 10 11 12 13 14	0 / 11
5	5..15	15 5 6 7 8 9 10 11 12 13 14	5	**5** 15 6 7 8 9 10 11 12 13 14	0 / 10
6	6..15	15 6 7 8 9 10 11 12 13 14	6	**6** 15 7 8 9 10 11 12 13 14	0 / 9

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
7	7..15	15 7 8 9 10 11 12 13 14	7	7 15 8 9 10 11 12 13 14	0 / 8
8	8..15	15 8 9 10 11 12 13 14	8	8 15 9 10 11 12 13 14	0 / 7
9	9..15	15 9 10 11 12 13 14	9	9 15 10 11 12 13 14	0 / 6
10	10..15	15 10 11 12 13 14	10	10 15 11 12 13 14	0 / 5
11	11..15	15 11 12 13 14	11	11 15 12 13 14	0 / 4
12	12..15	15 12 13 14	12	12 15 13 14	0 / 3
13	13..15	15 13 14	13	13 15 14	0 / 2
14	14..15	15 14	14	14 15	0 / 1

4.9 Pivot = next-to-last element $A[hi-1]$ (2021)

Ready answer, pivot = next-to-last element, $n = 15$

(a) Optimal: $\langle 12, 2, 3, 1, 5, 4, 7, 6, 14, 10, 11, 9, 13, 8, 15 \rangle$

(b) Slowest: $\langle 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 1, 15 \rangle$

Why (b) works: position 14 (next-to-last) holds 1, the minimum. Partition moves 1 to the front; the right part is $3, 4, \ldots, 14, 2, 15$, whose next-to-last element is 2, the new minimum; and so on.

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	12 2 3 1 5 4 7 6 14 10 11 9 13 8 15	8	6 2 3 1 5 4 7 8 14 10 11 9 13 12 15	7 / 7
2	1..7	6 2 3 1 5 4 7	4	1 2 3 4 5 6 7	3 / 3
3	1..3	1 2 3	2	1 2 3	1 / 1
4	5..7	5 6 7	6	5 6 7	1 / 1
5	9..15	14 10 11 9 13 12 15	12	9 10 11 12 13 14 15	3 / 3
6	9..11	9 10 11	10	9 10 11	1 / 1
7	13..15	13 14 15	14	13 14 15	1 / 1

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	2 3 4 5 6 7 8 9 10 11 12 13 14 1 15	1	1 3 4 5 6 7 8 9 10 11 12 13 14 2 15	0 / 14
2	2..15	3 4 5 6 7 8 9 10 11 12 13 14 2 15	2	2 4 5 6 7 8 9 10 11 12 13 14 3 15	0 / 13
3	3..15	4 5 6 7 8 9 10 11 12 13 14 3 15	3	3 5 6 7 8 9 10 11 12 13 14 4 15	0 / 12
4	4..15	5 6 7 8 9 10 11 12 13 14 4 15	4	4 6 7 8 9 10 11 12 13 14 5 15	0 / 11
5	5..15	6 7 8 9 10 11 12 13 14 5 15	5	5 7 8 9 10 11 12 13 14 6 15	0 / 10
6	6..15	7 8 9 10 11 12 13 14 6 15	6	6 8 9 10 11 12 13 14 7 15	0 / 9
7	7..15	8 9 10 11 12 13 14 7 15	7	7 9 10 11 12 13 14 8 15	0 / 8
8	8..15	9 10 11 12 13 14 8 15	8	8 10 11 12 13 14 9 15	0 / 7
9	9..15	10 11 12 13 14 9 15	9	9 11 12 13 14 10 15	0 / 6
10	10..15	11 12 13 14 10 15	10	10 12 13 14 11 15	0 / 5
11	11..15	12 13 14 11 15	11	11 13 14 12 15	0 / 4
12	12..15	13 14 12 15	12	12 14 13 15	0 / 3
13	13..15	14 13 15	13	13 14 15	0 / 2
14	14..15	14 15	14	14 15	0 / 1

4.10 Pivot = middle element $A[\lfloor (lo+hi)/2 \rfloor]$

Ready answer, pivot = middle element, $n = 15$

(a) Optimal: the **sorted** array $\langle 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 \rangle$ (the middle of a sorted part is its median, and Partition keeps both halves sorted)
(b) Slowest: $\langle 2, 12, 4, 10, 6, 14, 8, 1, 3, 5, 7, 9, 11, 13, 15 \rangle$

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	2 12 4 10 6 14 8 **1** 3 5 7 9 11 13 15	1	**1** 12 4 10 6 14 8 2 3 5 7 9 11 13 15	0 / 14
2	2..15	12 4 10 6 14 8 **2** 3 5 7 9 11 13 15	2	**2** 4 10 6 14 8 12 3 5 7 9 11 13 15	0 / 13
3	3..15	4 10 6 14 8 12 **3** 5 7 9 11 13 15	3	**3** 10 6 14 8 12 4 5 7 9 11 13 15	0 / 12
4	4..15	10 6 14 8 12 **4** 5 7 9 11 13 15	4	**4** 6 14 8 12 10 5 7 9 11 13 15	0 / 11
5	5..15	6 14 8 12 10 **5** 7 9 11 13 15	5	**5** 14 8 12 10 6 7 9 11 13 15	0 / 10
6	6..15	14 8 12 10 **6** 7 9 11 13 15	6	**6** 8 12 10 14 7 9 11 13 15	0 / 9
7	7..15	8 12 10 14 **7** 9 11 13 15	7	**7** 12 10 14 8 9 11 13 15	0 / 8
8	8..15	12 10 14 **8** 9 11 13 15	8	**8** 10 14 12 9 11 13 15	0 / 7
9	9..15	10 14 12 **9** 11 13 15	9	**9** 14 12 10 11 13 15	0 / 6
10	10..15	14 12 **10** 11 13 15	10	**10** 12 14 11 13 15	0 / 5
11	11..15	12 14 **11** 13 15	11	**11** 14 12 13 15	0 / 4
12	12..15	14 **12** 13 15	12	**12** 14 13 15	0 / 3
13	13..15	14 **13** 15	13	**13** 14 15	0 / 2
14	14..15	**14** 15	14	**14** 15	0 / 1

4.11 Arrays for $n = 31$ (if the exam asks for $n \geq 16$)

All verified by running QuickSort with the Partition above (best: every split equal, 98 comparisons; worst: one side empty every time, 465 comparisons).

Pivot	best (a) / worst (b)
first	(a) 16, 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15, 8, 24, 17, 19, 18, 22, 21, 23, 20, 28, 25, 27, 26, 30, 29, 31
	(b) 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31
last	(a) 24, 1, 2, 3, 7, 5, 4, 6, 14, 9, 10, 11, 15, 13, 8, 12, 28, 17, 18, 19, 23, 21, 20, 22, 30, 25, 26, 27, 31, 29, 16
	(b) 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31
last (textbook)	(a) 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15, 8, 24, 17, 19, 18, 22, 21, 23, 20, 28, 25, 27, 26, 30, 29, 31, 16
	(b) sorted 1..31
second	(a) 8, 16, 3, 1, 5, 6, 7, 2, 10, 12, 11, 9, 13, 14, 15, 4, 20, 24, 19, 17, 21, 22, 23, 18, 26, 28, 27, 25, 29, 30, 31
	(b) 31, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

next-to-last	(a) 24, 2, 3, 1, 5, 4, 7, 6, 14, 10, 11, 9, 13, 8, 15, 12, 28, 18, 19, 17, 21, 20, 23, 22, 30, 26, 27, 25, 29, 16, 31
	(b) 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 1, 31
middle	(a) sorted 1..31
	(b) 2, 24, 4, 18, 6, 28, 8, 20, 10, 26, 12, 22, 14, 30, 16, 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31

4.12 If your partition procedure is different

The "smaller to the left, larger to the right" requirement does not fix *where* the other elements end up; that depends on the exact partition code. That is why you must state the partition you use. If your notes use another method (for example Hoare's two-pointer method), the *principle* is unchanged:

- **(a)** the element sitting in the pivot position must be the **median** of the current part, at every level of the recursion;
- **(b)** it must be the **minimum or maximum** of the current part, at every level.

Build the array from the inside out and check it by tracing, exactly as in the tables above. The graded exams show that a clearly stated method with a trace that "works correctly" receives full credit.

PART V

Question 4: Matrix-Chain Products

CHAPTER 5

Matrix Chains: Minimum, Maximum, Average

5.1 The question

Exam 1, 2018, Q4 (25%)

Consider the problem of multiplying n rectangular matrices discussed in class. Assume, in contrast to what we did in class, that we want to determine the **maximum** number of scalar multiplications that one might need (that is, compute the maximum of all possible parenthesizations). Formulate precisely an algorithm that determines this value. Then carry out your method on the following product to show what is the worst-possible parenthesization and how many scalar multiplications are required to carry it out: $M_{9,3} * M_{3,9} * M_{9,1} * M_{1,8} * M_{8,4}$.

$M_{9,3}$ means a matrix with 9 rows and 3 columns. **Answer: 1251**, parenthesization $(((M_1M_2)(M_3M_4))M_5)$. Section 5.5 has the full work. 2008 asked for the **average** instead (Section 5.7).

5.2 Matrix multiplication in two minutes

An $a \times b$ matrix ("a rows, b columns") can be multiplied by a $b \times c$ matrix (the inner numbers must agree). The result is $a \times c$. Each of the $a \cdot c$ result entries is a sum of b products (row times column), so

$$\boxed{(a \times b) \cdot (b \times c) \text{ costs } a \cdot b \cdot c \text{ scalar multiplications and gives an } a \times c \text{ matrix.}}$$

Example: $(9 \times 3) \cdot (3 \times 9)$ costs $9 \cdot 3 \cdot 9 = 243$ and gives a 9×9 matrix.

5.3 Why the parenthesization matters

Matrix multiplication is *associative*: $(M_1M_2)M_3 = M_1(M_2M_3)$, same result. But the cost can be very different. With $M_1 : 10 \times 30$, $M_2 : 30 \times 5$, $M_3 : 5 \times 60$:

$$(M_1M_2)M_3 : \ 10 \cdot 30 \cdot 5 \ [\text{gives } 10 \times 5] \ + \ 10 \cdot 5 \cdot 60 = 1500 + 3000 = 4500,$$
$$M_1(M_2M_3) : \ 30 \cdot 5 \cdot 60 \ [\text{gives } 30 \times 60] \ + \ 10 \cdot 30 \cdot 60 = 9000 + 18000 = 27000.$$

Six times more work, same answer. The order of the matrices never changes; only the brackets do.

How many parenthesizations are there? For n matrices: $1, 1, 2, 5, 14, 42, 132$ for $n = 1, \ldots, 7$ (the Catalan numbers). With 5 matrices there are **14**, few enough to list all of them as a check (we do this below), but the exam wants the *algorithm*.

5.4 The dynamic-programming method

Notation. Matrix M_i has dimensions $p_{i-1} \times p_i$. For $M_{9,3} * M_{3,9} * M_{9,1} * M_{1,8} * M_{8,4}$:

$$p = (p_0, p_1, p_2, p_3, p_4, p_5) = (9, 3, 9, 1, 8, 4).$$

Every consecutive pair of matrices shares a number, so you only write each number once.

Key observation. In any parenthesization of $M_i \cdots M_j$ there is a **last** multiplication. It multiplies the result of $M_i \cdots M_k$ (a $p_{i-1} \times p_k$ matrix) by the result of $M_{k+1} \cdots M_j$ (a $p_k \times p_j$ matrix), for some *split point* k with $i \le k < j$. That last multiplication costs $p_{i-1}\, p_k\, p_j$, no matter how the two halves were bracketed inside.

Consequence (this is why DP works for max as well as min). The cost of a parenthesization with split k is

$$(\text{cost of the left part}) + (\text{cost of the right part}) + p_{i-1}p_k p_j,$$

and the left and right parts can be bracketed *independently*. So to make the total as large as possible, make each part as large as possible. Let $C(i,j)$ be the maximum cost for $M_i \cdots M_j$:

$$\boxed{C(i,i) = 0, \qquad C(i,j) = \max_{i \le k < j} \Big\{ C(i,k) + C(k+1,j) + p_{i-1}\, p_k\, p_j \Big\} \quad (i < j).}$$

For the class version (minimum) replace max by min. Nothing else changes.

The algorithm

```
procedure MaxChain(p[0..n])
   for i := 1 to n do C[i,i] := 0
   for len := 2 to n do                      // chain length: 2, 3, ..., n
      for i := 1 to n-len+1 do
         j := i + len - 1
         C[i,j] := -infinity                 // (for MIN: +infinity)
         for k := i to j-1 do
            v := C[i,k] + C[k+1,j] + p[i-1]*p[k]*p[j]
            if v > C[i,j] then               // (for MIN: v < C[i,j])
               C[i,j] := v;  K[i,j] := k     // remember the best split
   return C[1,n]

procedure Print(i, j)                        // the parenthesization
   if i = j then write "M_i"
   else write "("; Print(i, K[i,j]); Print(K[i,j]+1, j); write ")"
```

Why by length? $C(i,j)$ needs $C(i,k)$ and $C(k+1,j)$, which are shorter chains, so they are already computed. In the table, you fill the diagonal ($C(i,i) = 0$), then the next diagonal ($C(1,2), C(2,3), \ldots$), and so on up to the top-right corner $C(1,n)$.

Complexity. There are $\mathcal{O}(n^2)$ table entries and each tries $\mathcal{O}(n)$ split points: time $\Theta(n^3)$, space $\Theta(n^2)$. (Trying all parenthesizations directly would take exponential time, since the Catalan numbers grow like 4^n.)

Recipe: matrix chain by hand

1. Write $p_0, p_1, \ldots, p_n$ with their indices above them.
2. Length 2: $C(i, i+1) = p_{i-1}p_i p_{i+1}$ (only one way).
3. Length 3, 4, 5: for each cell list *every* k as "left + right + $p_{i-1} \cdot p_k \cdot p_j$ = value"; circle the max (or min); record k.
4. Draw the C table and the K table.
5. Read the parenthesization from K starting at $K(1,n)$.
6. Check: multiply it out step by step (dimension and cost of each product) and confirm the sum equals $C(1,n)$.

Watch out: the index trap

For $C(i,j)$ with split k the new cost is $p_{i-1} \cdot p_k \cdot p_j$: the *rows of* M_i, the *columns of* M_k, the *columns of* M_j. The most common mistake is to use p_i instead of p_{i-1}. Sanity check: these are the outer dimensions of the left result ($p_{i-1} \times p_k$) and the right result ($p_k \times p_j$).

5.5 MAXIMUM, fully worked: 2018

Dimensions: $M_1 : 9 \times 3$, $M_2 : 3 \times 9$, $M_3 : 9 \times 1$, $M_4 : 1 \times 8$, $M_5 : 8 \times 4$. So $p = (9,3,9,1,8,4)$, i.e. $p_0 = 9$, $p_1 = 3$, $p_2 = 9$, $p_3 = 1$, $p_4 = 8$, $p_5 = 4$.

Base case: $C(i,i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1,j) + p_{i-1}\,p_k\,p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1)+C(2,2)+p_0p_1p_2 = 0+0+9\cdot3\cdot9 = 0+0+243$		**243**
		$\Rightarrow\ C(1,2) = 243, \quad K(1,2) = 1$		
$C(2,3)$	$k=2$	$C(2,2)+C(3,3)+p_1p_2p_3 = 0+0+3\cdot9\cdot1 = 0+0+27$		**27**
		$\Rightarrow\ C(2,3) = 27, \quad K(2,3) = 2$		
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4 = 0+0+9\cdot1\cdot8 = 0+0+72$		**72**
		$\Rightarrow\ C(3,4) = 72, \quad K(3,4) = 3$		
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5 = 0+0+1\cdot8\cdot4 = 0+0+32$		**32**
		$\Rightarrow\ C(4,5) = 32, \quad K(4,5) = 4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+27+9\cdot3\cdot1 = 0+27+27$		54
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 243+0+9\cdot9\cdot1 = 243+0+81$	$\leftarrow$ max	**324**
		$\Rightarrow\ C(1,3) = 324, \quad K(1,3) = 2$		
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+72+3\cdot9\cdot8 = 0+72+216$	$\leftarrow$ max	**288**
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 27+0+3\cdot1\cdot8 = 27+0+24$		51
		$\Rightarrow\ C(2,4) = 288, \quad K(2,4) = 2$		
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+32+9\cdot1\cdot4 = 0+32+36$		68
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 72+0+9\cdot8\cdot4 = 72+0+288$	$\leftarrow$ max	**360**
		$\Rightarrow\ C(3,5) = 360, \quad K(3,5) = 4$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+288+9\cdot3\cdot8 = 0+288+216$		504
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 243+72+9\cdot9\cdot8 = 243+72+648$	$\leftarrow$ max	**963**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 324+0+9\cdot1\cdot8 = 324+0+72$		396
		$\Rightarrow\ C(1,4) = 963, \quad K(1,4) = 2$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+360+3\cdot9\cdot4 = 0+360+108$	$\leftarrow$ max	**468**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 27+32+3\cdot1\cdot4 = 27+32+12$		71
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 288+0+3\cdot8\cdot4 = 288+0+96$		384
		$\Rightarrow\ C(2,5) = 468, \quad K(2,5) = 2$		
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+468+9\cdot3\cdot4 = 0+468+108$		576
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 243+360+9\cdot9\cdot4 = 243+360+324$		927
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 324+32+9\cdot1\cdot4 = 324+32+36$		392
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 963+0+9\cdot8\cdot4 = 963+0+288$	$\leftarrow$ max	**1251**
		$\Rightarrow\ C(1,5) = 1251, \quad K(1,5) = 4$		

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	243	324	963	**1251**
2		0	27	288	468
3			0	72	360
4				0	32
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$

Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(9\times 3)\cdot(3\times 9)$ costs $9\cdot 3\cdot 9 = 243$
- $(M_3)\cdot(M_4)$: $(9\times 1)\cdot(1\times 8)$ costs $9\cdot 1\cdot 8 = 72$
- $((M_1\,M_2))\cdot((M_3\,M_4))$: $(9\times 9)\cdot(9\times 8)$ costs $9\cdot 9\cdot 8 = 648$
- $(((M_1\,M_2)\,(M_3\,M_4)))\cdot(M_5)$: $(9\times 8)\cdot(8\times 4)$ costs $9\cdot 8\cdot 4 = 288$
- Total $= 243 + 72 + 648 + 288 = 1251$ ✓

Check against all 14 parenthesizations (computed by brute force; you would not do this on the exam, but it proves the table is right):

#	parenthesization	scalar mults
1	$((M_1\,(M_2\,M_3))\,(M_4\,M_5))$	122 **(MIN)**
2	$(M_1\,((M_2\,M_3)\,(M_4\,M_5)))$	179
3	$(M_1\,(((M_2\,M_3)\,M_4)\,M_5))$	255
4	$(M_1\,(M_2\,(M_3\,(M_4\,M_5))))$	284
5	$(((M_1\,M_2)\,M_3)\,(M_4\,M_5))$	392
6	$(((M_1\,(M_2\,M_3))\,M_4)\,M_5)$	414
7	$(M_1\,((M_2\,(M_3\,M_4))\,M_5))$	492
8	$((M_1\,((M_2\,M_3)\,M_4))\,M_5)$	555
9	$(M_1\,(M_2\,((M_3\,M_4)\,M_5)))$	576
10	$((M_1\,M_2)\,(M_3\,(M_4\,M_5)))$	635
11	$((((M_1\,M_2)\,M_3)\,M_4)\,M_5)$	684
12	$((M_1\,(M_2\,(M_3\,M_4)))\,M_5)$	792
13	$((M_1\,M_2)\,((M_3\,M_4)\,M_5))$	927
14	$(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$	1251 **(MAX)**
	sum of all 14	**7558**
	average	**7558/14 ≈ 539.86**

On the exam: model answer, 2018 Q4

Method. Let M_i be $p_{i-1}\times p_i$; here $p = (9,3,9,1,8,4)$. Let $C(i,j)$ = maximum number of scalar multiplications to compute $M_i\cdots M_j$. The last multiplication splits at some k ($i \le k < j$) and costs $p_{i-1}p_kp_j$ independently of how the two parts are bracketed, so $C(i,i) = 0$, $C(i,j) = \max_{i\le k<j}\{C(i,k) + C(k+1,j) + p_{i-1}p_kp_j\}$. Fill the table by increasing chain length, store the maximizing k in $K(i,j)$, answer $C(1,n)$; time $\Theta(n^3)$, space $\Theta(n^2)$.
Computation. [Copy the cell-by-cell lines and the two tables.]
Answer. Worst parenthesization $(((M_1M_2)(M_3M_4))M_5)$ with $243 + 72 + 648 + 288 = \mathbf{1251}$ scalar multiplications.

5.6 MINIMUM (the version from class)

Same table with min. For the 2018 dimensions: **Dimensions:** $M_1 : 9 \times 3$, $M_2 : 3 \times 9$, $M_3 : 9 \times 1$, $M_4 : 1 \times 8$, $M_5 : 8 \times 4$. So $p = (9,3,9,1,8,4)$, i.e. $p_0 = 9$, $p_1 = 3$, $p_2 = 9$, $p_3 = 1$, $p_4 = 8$, $p_5 = 4$.

Base case: $C(i,i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1,j) + p_{i-1}\,p_k\,p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1)+C(2,2)+p_0p_1p_2 = 0+0+9\cdot3\cdot9 = 0+0+243$		**243**
		$\Rightarrow\ C(1,2) = 243,\quad K(1,2) = 1$		
$C(2,3)$	$k=2$	$C(2,2)+C(3,3)+p_1p_2p_3 = 0+0+3\cdot9\cdot1 = 0+0+27$		**27**
		$\Rightarrow\ C(2,3) = 27,\quad K(2,3) = 2$		
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4 = 0+0+9\cdot1\cdot8 = 0+0+72$		**72**
		$\Rightarrow\ C(3,4) = 72,\quad K(3,4) = 3$		
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5 = 0+0+1\cdot8\cdot4 = 0+0+32$		**32**
		$\Rightarrow\ C(4,5) = 32,\quad K(4,5) = 4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+27+9\cdot3\cdot1 = 0+27+27$	$\leftarrow$ min	**54**
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 243+0+9\cdot9\cdot1 = 243+0+81$		324
		$\Rightarrow\ C(1,3) = 54,\quad K(1,3) = 1$		
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+72+3\cdot9\cdot8 = 0+72+216$		288
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 27+0+3\cdot1\cdot8 = 27+0+24$	$\leftarrow$ min	**51**
		$\Rightarrow\ C(2,4) = 51,\quad K(2,4) = 3$		
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+32+9\cdot1\cdot4 = 0+32+36$	$\leftarrow$ min	**68**
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 72+0+9\cdot8\cdot4 = 72+0+288$		360
		$\Rightarrow\ C(3,5) = 68,\quad K(3,5) = 3$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+51+9\cdot3\cdot8 = 0+51+216$		267
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 243+72+9\cdot9\cdot8 = 243+72+648$		963
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 54+0+9\cdot1\cdot8 = 54+0+72$	$\leftarrow$ min	**126**
		$\Rightarrow\ C(1,4) = 126,\quad K(1,4) = 3$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+68+3\cdot9\cdot4 = 0+68+108$		176
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 27+32+3\cdot1\cdot4 = 27+32+12$	$\leftarrow$ min	**71**
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 51+0+3\cdot8\cdot4 = 51+0+96$		147
		$\Rightarrow\ C(2,5) = 71,\quad K(2,5) = 3$		
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+71+9\cdot3\cdot4 = 0+71+108$		179
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 243+68+9\cdot9\cdot4 = 243+68+324$		635
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 54+32+9\cdot1\cdot4 = 54+32+36$	$\leftarrow$ min	**122**
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 126+0+9\cdot8\cdot4 = 126+0+288$		414
		$\Rightarrow\ C(1,5) = 122,\quad K(1,5) = 3$		

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	243	54	126	**122**
2		0	27	51	71
3			0	72	68
4				0	32
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	1	3	3
2		–	2	3	3
3			–	3	3
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $((M_1\,(M_2\,M_3))\,(M_4\,M_5))$

Check by multiplying it out:

- $(M_2)\cdot(M_3)$: $(3\times 9)\cdot(9\times 1)$ costs $3\cdot 9\cdot 1 = 27$
- $(M_1)\cdot((M_2\,M_3))$: $(9\times 3)\cdot(3\times 1)$ costs $9\cdot 3\cdot 1 = 27$
- $(M_4)\cdot(M_5)$: $(1\times 8)\cdot(8\times 4)$ costs $1\cdot 8\cdot 4 = 32$
- $((M_1\,(M_2\,M_3)))\cdot((M_4\,M_5))$: $(9\times 1)\cdot(1\times 4)$ costs $9\cdot 1\cdot 4 = 36$
- Total $= 27+27+32+36 = 122$ ✓

5.7 AVERAGE (2008)

Exam 1, 2008, Q4

…we want to determine the **average** number of scalar multiplications that one might need (that is, compute the average of all possible parenthesizations). Formulate a method that determines this value. Then carry out your method on the following product to show what is the worst-possible parenthesization and how many scalar multiplications are required to carry it out: $M_{2,4} * M_{4,3} * M_{3,1} * M_{1,4} * M_{4,7}$.

Watch out: the trap that cost a 2008 student 5 points

You **cannot** simply replace max by "average over k". Different split points k have different numbers of parenthesizations behind them, so a plain average over k gives the wrong weights. (The graded 2008 answer averaged over k and lost 3 points on the method and 2 more on the parenthesization, marked "does not match".)

Correct method: keep a count and a sum.

- $N(i,j)$ = **number** of parenthesizations of $M_i\cdots M_j$. $N(i,i)=1$, $N(i,j)=\sum_{k=i}^{j-1} N(i,k)\cdot N(k+1,j)$ (each left bracketing can be combined with each right bracketing).
- $S(i,j)$ = **sum** of the costs of all those parenthesizations. $S(i,i)=0$ and

$$S(i,j)=\sum_{k=i}^{j-1}\Big[\,S(i,k)\cdot N(k+1,j)\;+\;N(i,k)\cdot S(k+1,j)\;+\;N(i,k)\cdot N(k+1,j)\cdot p_{i-1}p_kp_j\,\Big].$$

  Why: with split k there are $N(i,k)\cdot N(k+1,j)$ combinations. Each left bracketing appears $N(k+1,j)$ times (once with each right bracketing), contributing $S(i,k)\cdot N(k+1,j)$ in total; symmetrically for the right part; and every one of the combinations pays the final $p_{i-1}p_kp_j$.
- **Average** $= S(1,n)/N(1,n)$. Same table-filling order as before; time $\Theta(n^3)$.

Equivalently, $\mathrm{Avg}(i,j)=\sum_k w_k\big[\mathrm{Avg}(i,k)+\mathrm{Avg}(k+1,j)+p_{i-1}p_kp_j\big]$ with weights $w_k = N(i,k)N(k+1,j)/N(i,j)$: a *weighted* average.

2008 worked: $p=(2,4,3,1,4,7)$

cell	split	count $N(i,k)N(k{+}1,j)$	sum part $S(i,k)N(k{+}1,j)+N(i,k)S(k{+}1,j)+\text{count}\cdot p_{i-1}p_kp_j$	value
Chains of length 2				

cell	split	count $N(i,k)N(k{+}1,j)$	sum part $S(i,k)N(k{+}1,j)+N(i,k)S(k{+}1,j)+\text{count}\cdot p_{i-1}p_kp_j$	value
$(1,2)$	$k=1$	$1\cdot1=1$	$0\cdot1+1\cdot0+1\cdot(2\cdot4\cdot3)$	24
			$\Rightarrow\ N(1,2)=1,\quad S(1,2)=24,\quad \text{average}=24$	
$(2,3)$	$k=2$	$1\cdot1=1$	$0\cdot1+1\cdot0+1\cdot(4\cdot3\cdot1)$	12
			$\Rightarrow\ N(2,3)=1,\quad S(2,3)=12,\quad \text{average}=12$	
$(3,4)$	$k=3$	$1\cdot1=1$	$0\cdot1+1\cdot0+1\cdot(3\cdot1\cdot4)$	12
			$\Rightarrow\ N(3,4)=1,\quad S(3,4)=12,\quad \text{average}=12$	
$(4,5)$	$k=4$	$1\cdot1=1$	$0\cdot1+1\cdot0+1\cdot(1\cdot4\cdot7)$	28
			$\Rightarrow\ N(4,5)=1,\quad S(4,5)=28,\quad \text{average}=28$	
Chains of length 3				
$(1,3)$	$k=1$	$1\cdot1=1$	$0\cdot1+1\cdot12+1\cdot(2\cdot4\cdot1)$	20
	$k=2$	$1\cdot1=1$	$24\cdot1+1\cdot0+1\cdot(2\cdot3\cdot1)$	30
			$\Rightarrow\ N(1,3)=2,\quad S(1,3)=50,\quad \text{average}=25$	
$(2,4)$	$k=2$	$1\cdot1=1$	$0\cdot1+1\cdot12+1\cdot(4\cdot3\cdot4)$	60
	$k=3$	$1\cdot1=1$	$12\cdot1+1\cdot0+1\cdot(4\cdot1\cdot4)$	28
			$\Rightarrow\ N(2,4)=2,\quad S(2,4)=88,\quad \text{average}=44$	
$(3,5)$	$k=3$	$1\cdot1=1$	$0\cdot1+1\cdot28+1\cdot(3\cdot1\cdot7)$	49
	$k=4$	$1\cdot1=1$	$12\cdot1+1\cdot0+1\cdot(3\cdot4\cdot7)$	96
			$\Rightarrow\ N(3,5)=2,\quad S(3,5)=145,\quad \text{average}=145/2\approx72.50$	
Chains of length 4				
$(1,4)$	$k=1$	$1\cdot2=2$	$0\cdot2+1\cdot88+2\cdot(2\cdot4\cdot4)$	152
	$k=2$	$1\cdot1=1$	$24\cdot1+1\cdot12+1\cdot(2\cdot3\cdot4)$	60
	$k=3$	$2\cdot1=2$	$50\cdot1+2\cdot0+2\cdot(2\cdot1\cdot4)$	66
			$\Rightarrow\ N(1,4)=5,\quad S(1,4)=278,\quad \text{average}=278/5\approx55.60$	
$(2,5)$	$k=2$	$1\cdot2=2$	$0\cdot2+1\cdot145+2\cdot(4\cdot3\cdot7)$	313
	$k=3$	$1\cdot1=1$	$12\cdot1+1\cdot28+1\cdot(4\cdot1\cdot7)$	68
	$k=4$	$2\cdot1=2$	$88\cdot1+2\cdot0+2\cdot(4\cdot4\cdot7)$	312
			$\Rightarrow\ N(2,5)=5,\quad S(2,5)=693,\quad \text{average}=693/5\approx138.60$	
Chains of length 5				
$(1,5)$	$k=1$	$1\cdot5=5$	$0\cdot5+1\cdot693+5\cdot(2\cdot4\cdot7)$	973
	$k=2$	$1\cdot2=2$	$24\cdot2+1\cdot145+2\cdot(2\cdot3\cdot7)$	277
	$k=3$	$2\cdot1=2$	$50\cdot1+2\cdot28+2\cdot(2\cdot1\cdot7)$	134
	$k=4$	$5\cdot1=5$	$278\cdot1+5\cdot0+5\cdot(2\cdot4\cdot7)$	558
			$\Rightarrow\ N(1,5)=14,\quad S(1,5)=1942,\quad \text{average}=1942/14\approx138.71$	

Count table $N(i,j)$

$i\backslash j$	1	2	3	4	5
1	1	1	2	5	14
2		1	1	2	5
3			1	1	2
4				1	1
5					1

Sum table $S(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	24	50	278	1942
2		0	12	88	693
3			0	12	145
4				0	28
5					0

Answer: $\text{average}=\dfrac{S(1,5)}{N(1,5)}=\dfrac{1942}{14}=\dfrac{971}{7}\approx$ **138.71** scalar multiplications.

Brute-force check (all 14):

#	parenthesization	scalar mults
1	$((M_1\,(M_2\,M_3))\,(M_4\,M_5))$	62 **(MIN)**

2	$(((M_1\,M_2)\,M_3)\,(M_4\,M_5))$	72
3	$(((M_1\,(M_2\,M_3))\,M_4)\,M_5)$	84
4	$((((M_1\,M_2)\,M_3)\,M_4)\,M_5)$	94
5	$((M_1\,M_2)\,(M_3\,(M_4\,M_5)))$	115
6	$((M_1\,((M_2\,M_3)\,M_4))\,M_5)$	116
7	$(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$	116
8	$(M_1\,((M_2\,M_3)\,(M_4\,M_5)))$	124
9	$((M_1\,(M_2\,(M_3\,M_4)))\,M_5)$	148
10	$((M_1\,M_2)\,((M_3\,M_4)\,M_5))$	162
11	$(M_1\,(M_2\,(M_3\,(M_4\,M_5))))$	189
12	$(M_1\,(((M_2\,M_3)\,M_4)\,M_5))$	196
13	$(M_1\,((M_2\,(M_3\,M_4))\,M_5))$	228
14	$(M_1\,(M_2\,((M_3\,M_4)\,M_5)))$	236 **(MAX)**
	sum of all 14	**1942**
	average	**1942/14 $\approx$ 138.71**

The 2008 question also asks for "the worst-possible parenthesization". That is the maximum:

$$(M_1(M_2((M_3M_4)M_5))) \text{ with } \mathbf{236} \text{ scalar multiplications}$$

(the max table is in Part 6).

PART VI

Other Topics That Have Appeared

CHAPTER 6

Huffman, HeapSort, RSA, Euclid

These appeared on the 2020 Exam 1 (Huffman, HeapSort, RSA) and on the 2008 second exam (Huffman, HeapSort, Euclid). They are less likely than Questions 1–4, but each one is quick to learn.

6.1 Huffman codes

2008 Exam 2, Q4 (also 2020 Exam 1 with other numbers)

Construct a Huffman code for the symbols a through g, listed below together with their probabilities. Then determine the expected length of your resulting code! a 1/28, b 2/28, c 3/28, d 4/28, e 5/28, f 6/28, g 7/28.

In plain English

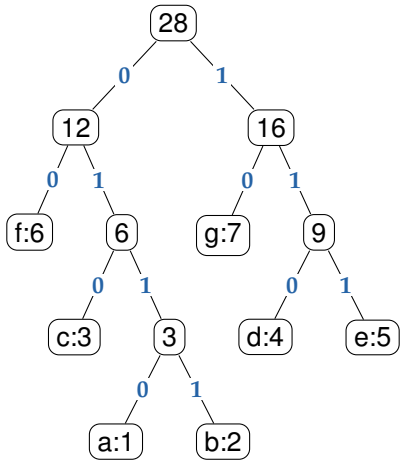
We want to write each symbol as a string of bits so that frequent symbols get *short* strings and the whole message is as short as possible on average. No code word may be the beginning (prefix) of another, so the message can be decoded without separators. Huffman's greedy rule gives the best possible such code.

Recipe: Huffman

1. Write all symbols with their weights (use the numerators: 1, 2, ...).
2. Repeatedly take the **two smallest** weights, join them under a new node whose weight is their **sum**. (Ties: any choice is fine.)
3. Stop when one tree remains. Label every left edge 0 and every right edge 1.
4. Code of a symbol = the 0/1 labels on the path from the root. Length = depth.
5. Expected length $= \sum(\text{probability} \times \text{length})$.

Shortcut check: $\sum(\text{weight} \times \text{length})$ equals the sum of all the merged (internal) weights.

2008 worked. Merges: $a{+}b = 3$; $c{+}(ab) = 6$; $d{+}e = 9$; $f{+}(cab) = 12$; $g{+}(de) = 16$; $12{+}16 = 28$.



symbol	a	b	c	d	e	f	g
weight	1	2	3	4	5	6	7
code	0110	0111	010	110	111	00	10
length	4	4	3	3	3	2	2

$$E[\text{length}] = \frac{1\cdot 4 + 2\cdot 4 + 3\cdot 3 + 4\cdot 3 + 5\cdot 3 + 6\cdot 2 + 7\cdot 2}{28} = \frac{74}{28} = \frac{37}{14} \approx \textbf{2.643} \text{ bits per symbol.}$$

Check: merged weights $3+6+9+12+16+28 = 74$ ✓. (A fixed-length code would need 3 bits.)

2020 version: weights a–h $= 1, 2, 3, 4, 5, 6, 6, 13$ (out of 40). Merges: 1+2 = 3, 3+3 = 6, 4+5 = 9, 6+6 = 12 (f,g), 6+9 = 15, 12+13 = 25, 15+25 = 40. Lengths: a 4, b 4, c 3, d 3, e 3, f 3, g 3, h 2. Expected length $= 110/40 =$ **2.75** (merged weights: $3+6+9+12+15+25+40 = 110$ ✓).

Complexity: with a heap (priority queue) each of the $n-1$ merges costs $\mathcal{O}(\log n)$: total $\mathcal{O}(n \log n)$.

6.2 Heaps and HeapSort

2008 Exam 2, Q5 (2020 Exam 1 had the same with other numbers)

(a) Construct a heap for the following array of numbers: 1 8 2 5 6 3 4 7 9 10. Show the array after the insertion of each element into the heap. (b) Use your heap to sort the array. Show the resulting heap after the extraction of each maximum.

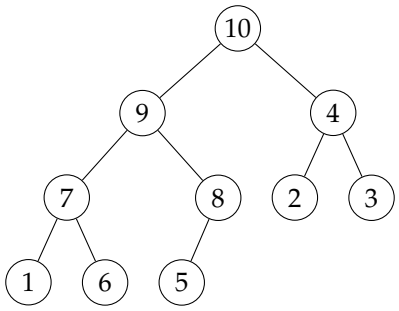
Max-heap: a complete binary tree stored in an array $H[1..m]$ (filled level by level, left to right), such that every parent is $\geq$ its children. Positions: parent of i is $\lfloor i/2 \rfloor$; children of i are $2i$ and $2i+1$. The maximum is at the root $H[1]$.

- **Insert** x: put it at the end ($H[m+1]$); while it is larger than its parent, swap it with the parent ("sift up"). Cost $\mathcal{O}(\log m)$.
- **Extract max:** remove $H[1]$; move the *last* element to the root; while it is smaller than its larger child, swap it with the **larger** child ("sift down"). Cost $\mathcal{O}(\log m)$.
- **HeapSort:** n inserts + n extracts $= \mathcal{O}(n \log n)$. In place: each extracted max goes to the free slot at the end of the array.

2008 worked: insert 1 8 2 5 6 3 4 7 9 10

insert	swaps on the way up	heap array afterwards
1	–	1
8	with 1	8 1
2	–	8 1 2
5	with 1	8 5 2 1
6	with 5	8 6 2 1 5
3	with 2	8 6 3 1 5 2
4	with 3	8 6 4 1 5 2 3
7	with 1, then 6	8 7 4 6 5 2 3 1
9	with 6, then 7, then 8	9 8 4 7 5 2 3 1 6
10	with 5, then 8, then 9	10 9 4 7 8 2 3 1 6 5

Final heap as a tree:



(b) extracting the maximum 10 times

removed	last element moved to root	swaps on the way down	heap afterwards
10	5	with 9, then 8	9 8 4 7 5 2 3 1 6
9	6	with 8, then 7	8 7 4 6 5 2 3 1
8	1	with 7, then 6	7 6 4 1 5 2 3
7	3	with 6, then 5	6 5 4 1 3 2
6	2	with 5, then 3	5 3 4 1 2
5	2	with 4	4 3 2 1
4	1	with 3	3 1 2
3	2	–	2 1
2	1	–	1
1	–	–	(empty)

The maxima come out $10, 9, \ldots, 1$; placed from the back of the array this gives the sorted array $1, 2, \ldots, 10$.

2020 version: insert 8 1 2 3 5 6 4 7 10 9

Arrays after each insertion: 8; 8 1; 8 1 2; 8 3 2 1; 8 5 2 1 3; 8 5 6 1 3 2; 8 5 6 1 3 2 4; 8 7 6 5 3 2 4 1; 10 8 6 7 3 2 4 1 5; **10 9 6 7 8 2 4 1 5 3**.
After each extraction: 9 8 6 7 3 2 4 1 5; 8 7 6 5 3 2 4 1; 7 5 6 1 3 2 4; 6 5 4 1 3 2; 5 3 4 1 2; 4 3 2 1; 3 1 2; 2 1; 1; (empty).

6.3 Fast multiplication, repeated squaring, and RSA

2020 Exam 1

Determine the bit complexity of encryption and decryption in RSA, M^e mod n and C^d mod n, with n having s digits, using the multiplication algorithm covered in class.

Step 1: multiplying two s-bit numbers

School method: $\Theta(s^2)$ bit operations.

Divide and conquer: split $x = x_1 \cdot 2^{s/2} + x_0$, $y = y_1 \cdot 2^{s/2} + y_0$. Then

$$xy = x_1y_1 \cdot 2^s + (x_1y_0 + x_0y_1) \cdot 2^{s/2} + x_0y_0.$$

Computing the four half-size products gives $T(s) = 4T(s/2) + cs = \Theta(s^2)$: no gain.

Gauss's trick (Karatsuba): compute only three products

$$P_1 = x_1y_1, \qquad P_2 = x_0y_0, \qquad P_3 = (x_1 + x_0)(y_1 + y_0), \qquad \text{then} \quad x_1y_0 + x_0y_1 = P_3 - P_1 - P_2.$$

$T(s) = 3T(s/2) + cs$, so by the master theorem ($a = 3 > b^d = 2$): $T(s) = \Theta(s^{\log_2 3}) = \Theta(s^{1.585})$.

Decimal example 23×45: $x_1 = 2, x_0 = 3, y_1 = 4, y_0 = 5$. $P_1 = 8$, $P_2 = 15$, $P_3 = 5 \cdot 9 = 45$, middle $= 45 - 8 - 15 = 22$. Result $8 \cdot 100 + 22 \cdot 10 + 15 = 1035$ ✓.

Step 2: computing $x^e \bmod n$ by repeated squaring

Write e in binary. Square repeatedly ($x, x^2, x^4, x^8, \ldots$, each reduced mod n) and multiply together the powers where e has a 1-bit.

```
procedure ModPow(x, e, n)
   result := 1;  b := x mod n
   while e > 0 do
      if e is odd then result := (result * b) mod n
      b := (b * b) mod n
      e := e div 2
   return result
```

Example $3^{13} \bmod 17$: $13 = 1101_2 = 8 + 4 + 1$. Powers: $3^1 = 3$, $3^2 = 9$, $3^4 = 81 \bmod 17 = 13$, $3^8 = 169 \bmod 17 = 16$. So $3^{13} \equiv 16 \cdot 13 \cdot 3 = 624 \equiv \mathbf{12} \pmod{17}$.

If e has t bits, the loop runs t times: at most $2t$ multiplications and $2t$ reductions mod n, each on numbers of at most $2s$ bits.

Step 3: put it together (the answer)

On the exam: model answer, RSA bit complexity

Encryption $C = M^e \bmod n$ and decryption $M = C^d \bmod n$ are modular exponentiations. The exponents $e, d < n$ have at most s bits, so repeated squaring uses $\mathcal{O}(s)$ multiplications of numbers with at most s bits (each intermediate result is reduced mod n), each followed by a reduction mod n. With the divide-and-conquer multiplication from class, $T(s) = 3T(s/2) + \mathcal{O}(s) = \mathcal{O}(s^{\log_2 3}) = \mathcal{O}(s^{1.59})$ per multiplication. Assuming the reduction mod n is done in the same time as a multiplication, each of encryption and decryption costs

$$\mathcal{O}(s) \cdot \mathcal{O}(s^{1.59}) = \mathcal{O}(\mathbf{s^{2.59}}) \text{ bit operations.}$$

(With schoolbook multiplication/division it would be $\mathcal{O}(s) \cdot \mathcal{O}(s^2) = \mathcal{O}(s^3)$. If the public exponent e is small, e.g. $e = 3$ or 65537, encryption needs only $\mathcal{O}(1)$ multiplications: $\mathcal{O}(s^{1.59})$.)

6.4 Euclid's algorithm and multiplicative inverses

2008 Exam 2, Q7

Compute the multiplicative inverse of 65 536 with respect to 331 679.

Meaning: find y with $65536 \cdot y \equiv 1 \pmod{331679}$. It exists because $\gcd(65536, 331679) = 1$.

Method (extended Euclid as a table). Keep rows $r_i = x_i \cdot m + y_i \cdot a$. Start with $r_0 = m$ $(x,y) = (1,0)$ and $r_1 = a$ $(x,y) = (0,1)$. Each new row: $q = \lfloor r_{i-1}/r_i \rfloor$, and *every* column gets "row$_{i-1}$ minus q·row$_i$". Stop when $r = 1$; then $1 = x \cdot m + y \cdot a$, so y is the inverse (add m if negative).

Small warm-up: inverse of 7 mod 40.

r	q	x	y
40		1	0
7		0	1
5	5	1	−5
2	1	−1	6
1	2	3	−17

$1 = 3 \cdot 40 - 17 \cdot 7$, so the inverse is $-17 \equiv \mathbf{23} \pmod{40}$. Check $7 \cdot 23 = 161 = 4 \cdot 40 + 1$ ✓.

2008 worked:

r	q	x	y
331679		1	0
65536		0	1
3999	5	1	−5
1552	16	−16	81
895	2	33	−167
657	1	−49	248
238	1	82	−415
181	2	−213	1078
57	1	295	−1493
10	3	−1098	5557
7	5	5785	−29278
3	1	−6883	34835
1	2	19551	−98948

$1 = 19551 \cdot 331679 - 98948 \cdot 65536$, so the inverse is $-98948 + 331679 = \mathbf{232\,731}$. (Check: $65536 \cdot 232731 \bmod 331679 = 1$.)

Watch out

The graded 2008 answer stopped at −98948 and lost 2 points. An inverse mod m must be given as a number between 0 and $m - 1$: add m to a negative result.

Complexity: the remainders shrink at least by half every two steps, so Euclid needs $\mathcal{O}(\log m) = \mathcal{O}(s)$ iterations for s-bit numbers.

PART VII

Past Exams, Solved

CHAPTER 7

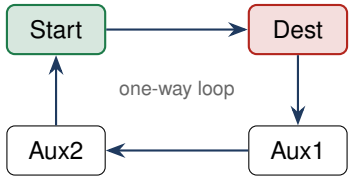
Past Exams with Model Answers

Each exam below is given question by question with a complete answer you could hand in. Long derivations that are identical to the teaching chapters are referenced instead of repeated; all numbers are restated.

	Q1 Hanoi	Q2 pages	Q3 QuickSort	Q4 matrix chain
2008	loop, $d = 1$: Go1	2,099,200	first, $n \geq 11$	**average** ≈ 138.71 (worst 236)
2012	tail-in lollipop	2,099,200	first, $n \geq 11$	max **816**
2014	loop, $d = 2$: Go2	8,392,704	**last**, $n \geq 11$	max **1088**
2018	loop, $d = 3$: Go3	8,392,704	first, $n \geq 13$	max **1251**
2020	–	4,202,496 (col) / 8,392,704 (row)	**second**, $n \geq 14$	max **1312** / **1161**
2021	–	–	**next-to-last**, $n \geq 13$	–

7.1 Exam 1, Monday Feb. 25, 2008

Q1 (25%) Hanoi: $E = \{(\textbf{Start}, \textbf{Dest}), (\textbf{Dest}, \textbf{Aux1}), (\textbf{Aux1}, \textbf{Aux2}), (\textbf{Aux2}, \textbf{Start})\}$



One-way loop. Numbering P_0 =Start, P_1 =Dest, P_2 =Aux1, P_3 =Aux2; Dest is $d = 1$ step after Start. Algorithm: procedures Go1, Go2, Go3 (page 11); **main: Go1**$(n, 0)$. Moves $T_1(n) = T_2(n-1) + T_3(n-1) + 1$ with T_2, T_3 as on page 13: $1, 6, 19, 52, 137, \ldots$ Complexity $\Theta\big(((1+\sqrt{17})/2)^n\big) \approx \Theta(2.56^n)$, in particular $\mathcal{O}(3^n)$. Move list for $n = 2$:

1. disk 1: Start→Dest
2. disk 1: Dest→Aux1
3. disk 2: Start→Dest
4. disk 1: Aux1→Aux2
5. disk 1: Aux2→Start
6. disk 1: Start→Dest

Q2 (25%) pages: 512 words/page, 512 frames, 1024×1024, row-major, `A[J,I]:=A[J,I]*B[I,J]`

Row $= 1024/512 = 2$ pages. Inner loop: $A[J, I]$ jumps (1024 pages), $B[I, J]$ walks (2 pages); $1026 > 512$, so every A access faults. Per I: 1024 reads + 1024 writes of A, 2 reads of B. **Total** $= (1024 + 1024 + 2) \cdot 1024 = 2{,}099{,}200$.

Q3 (25%) QuickSort, pivot = first element, $n \geq 11$

$n = 15$ (must be $2^k - 1$). (a) $\langle 8,1,3,2,6,5,7,4,12,9,11,10,14,13,15\rangle$; (b) $\langle 1,2,\ldots,15\rangle$. Partition procedure and traces: page 32. Best $\Theta(n \log n)$ (34 comparisons), worst $\Theta(n^2)$ (105 comparisons).

Q4 (25%) average for $M_{2,4} * M_{4,3} * M_{3,1} * M_{1,4} * M_{4,7}$, and the worst parenthesization

Method and full count/sum tables: page 44. **Average** $= 1942/14 = 971/7 \approx 138.71$. The worst-possible parenthesization (maximum): **Dimensions:** $M_1 : 2 \times 4$, $M_2 : 4 \times 3$, $M_3 : 3 \times 1$, $M_4 : 1 \times 4$, $M_5 : 4 \times 7$. So $p = (2,4,3,1,4,7)$, i.e. $p_0 = 2$, $p_1 = 4$, $p_2 = 3$, $p_3 = 1$, $p_4 = 4$, $p_5 = 7$.

Base case: $C(i,i) = 0$ for $i = 1,\ldots,5$.

cell	split	$C(i,k) + C(k{+}1,j) + p_{i-1}\,p_k\,p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1)+C(2,2)+p_0p_1p_2 = 0+0+2\cdot4\cdot3 = 0+0+24$		**24**
		$\Rightarrow\ C(1,2) = 24,\quad K(1,2) = 1$		
$C(2,3)$	$k=2$	$C(2,2)+C(3,3)+p_1p_2p_3 = 0+0+4\cdot3\cdot1 = 0+0+12$		**12**
		$\Rightarrow\ C(2,3) = 12,\quad K(2,3) = 2$		
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4 = 0+0+3\cdot1\cdot4 = 0+0+12$		**12**
		$\Rightarrow\ C(3,4) = 12,\quad K(3,4) = 3$		
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5 = 0+0+1\cdot4\cdot7 = 0+0+28$		**28**
		$\Rightarrow\ C(4,5) = 28,\quad K(4,5) = 4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+12+2\cdot4\cdot1 = 0+12+8$		20
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 24+0+2\cdot3\cdot1 = 24+0+6$	$\leftarrow$ max	**30**
		$\Rightarrow\ C(1,3) = 30,\quad K(1,3) = 2$		
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+12+4\cdot3\cdot4 = 0+12+48$	$\leftarrow$ max	**60**
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 12+0+4\cdot1\cdot4 = 12+0+16$		28
		$\Rightarrow\ C(2,4) = 60,\quad K(2,4) = 2$		
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+28+3\cdot1\cdot7 = 0+28+21$		49
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 12+0+3\cdot4\cdot7 = 12+0+84$	$\leftarrow$ max	**96**
		$\Rightarrow\ C(3,5) = 96,\quad K(3,5) = 4$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+60+2\cdot4\cdot4 = 0+60+32$	$\leftarrow$ max	**92**
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 24+12+2\cdot3\cdot4 = 24+12+24$		60
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 30+0+2\cdot1\cdot4 = 30+0+8$		38
		$\Rightarrow\ C(1,4) = 92,\quad K(1,4) = 1$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+96+4\cdot3\cdot7 = 0+96+84$	$\leftarrow$ max	**180**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 12+28+4\cdot1\cdot7 = 12+28+28$		68
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 60+0+4\cdot4\cdot7 = 60+0+112$		172
		$\Rightarrow\ C(2,5) = 180,\quad K(2,5) = 2$		
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+180+2\cdot4\cdot7 = 0+180+56$	$\leftarrow$ max	**236**
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 24+96+2\cdot3\cdot7 = 24+96+42$		162
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 30+28+2\cdot1\cdot7 = 30+28+14$		72
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 92+0+2\cdot4\cdot7 = 92+0+56$		148
		$\Rightarrow\ C(1,5) = 236,\quad K(1,5) = 1$		

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	24	30	92	**236**
2		0	12	60	180
3			0	12	96
4				0	28
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	1	1
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(M_1\,(M_2\,((M_3\,M_4)\,M_5)))$

Check by multiplying it out:

- $(M_3)\cdot(M_4)$: $(3\times 1)\cdot(1\times 4)$ costs $3\cdot 1\cdot 4 = 12$
- $((M_3\,M_4))\cdot(M_5)$: $(3\times 4)\cdot(4\times 7)$ costs $3\cdot 4\cdot 7 = 84$
- $(M_2)\cdot(((M_3\,M_4)\,M_5))$: $(4\times 3)\cdot(3\times 7)$ costs $4\cdot 3\cdot 7 = 84$
- $(M_1)\cdot((M_2\,((M_3\,M_4)\,M_5)))$: $(2\times 4)\cdot(4\times 7)$ costs $2\cdot 4\cdot 7 = 56$
- Total $= 12+84+84+56 = 236$ ✓

Worst: $(M_1(M_2((M_3M_4)M_5)))$ with 236 scalar multiplications.

7.2 Exam 1, Wednesday Sept. 26, 2012

Q1 (25%) Hanoi: $E = \{(\text{Start}, \text{Aux1}), (\text{Aux1}, \text{Aux2}), (\text{Aux2}, \text{Dest}), (\text{Dest}, \text{Aux1})\}$

Tail-in lollipop; full solution on page 16. **Answer:** procedure $H(k)$: $H(k-1)$; disk k Start→Aux1→Aux2; $C_1(k-1)$: Dest→Aux1; disk k Aux2→Dest; $C_2(k-1)$: Aux1→Dest. Moves $H(n) = H(n-1) + C_1(n-1) + C_2(n-1) + 3$: $3, 9, 24, 63, 168, \ldots$; complexity $\Theta((1+\sqrt{3})^n) \approx \Theta(2.73^n)$, i.e. $\mathcal{O}(3^n)$.

Q2 (25%) pages: 512 words/page, 256 frames, 1024×1024, row-major, same loop

Same structure as 2008 (frames do not matter because $1026 > 256$): $(1024 + 1024 + 2)\cdot 1024 = 2{,}099{,}200$.

Q3 (25%) QuickSort, pivot = first, $n \geq 11$

Same as 2008: $n = 15$, arrays on page 32.

Q4 (25%) maximum for $M_{6,2} * M_{2,6} * M_{6,1} * M_{1,7} * M_{7,9}$

Dimensions: $M_1 : 6\times 2$, $M_2 : 2\times 6$, $M_3 : 6\times 1$, $M_4 : 1\times 7$, $M_5 : 7\times 9$. So $p = (6,2,6,1,7,9)$, i.e. $p_0 = 6$, $p_1 = 2$, $p_2 = 6$, $p_3 = 1$, $p_4 = 7$, $p_5 = 9$.

Base case: $C(i,i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1,j) + p_{i-1}\,p_k\,p_j$	value
Chains of length 2			
$C(1,2)$	$k=1$	$C(1,1) + C(2,2) + p_0p_1p_2 = 0+0+6\cdot 2\cdot 6 = 0+0+72$	**72**
		$\Rightarrow\ C(1,2) = 72,\quad K(1,2) = 1$	
$C(2,3)$	$k=2$	$C(2,2) + C(3,3) + p_1p_2p_3 = 0+0+2\cdot 6\cdot 1 = 0+0+12$	**12**
		$\Rightarrow\ C(2,3) = 12,\quad K(2,3) = 2$	

cell	split	$C(i,k)+C(k+1,j)+p_{i-1}\,p_k\,p_j$		value
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4 = 0+0+6\cdot1\cdot7 = 0+0+42$		**42**
		$\Rightarrow\ C(3,4)=42,\quad K(3,4)=3$		
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5 = 0+0+1\cdot7\cdot9 = 0+0+63$		**63**
		$\Rightarrow\ C(4,5)=63,\quad K(4,5)=4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+12+6\cdot2\cdot1 = 0+12+12$		24
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 72+0+6\cdot6\cdot1 = 72+0+36$	$\leftarrow$ max	**108**
		$\Rightarrow\ C(1,3)=108,\quad K(1,3)=2$		
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+42+2\cdot6\cdot7 = 0+42+84$	$\leftarrow$ max	**126**
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 12+0+2\cdot1\cdot7 = 12+0+14$		26
		$\Rightarrow\ C(2,4)=126,\quad K(2,4)=2$		
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+63+6\cdot1\cdot9 = 0+63+54$		117
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 42+0+6\cdot7\cdot9 = 42+0+378$	$\leftarrow$ max	**420**
		$\Rightarrow\ C(3,5)=420,\quad K(3,5)=4$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+126+6\cdot2\cdot7 = 0+126+84$		210
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 72+42+6\cdot6\cdot7 = 72+42+252$	$\leftarrow$ max	**366**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 108+0+6\cdot1\cdot7 = 108+0+42$		150
		$\Rightarrow\ C(1,4)=366,\quad K(1,4)=2$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+420+2\cdot6\cdot9 = 0+420+108$	$\leftarrow$ max	**528**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 12+63+2\cdot1\cdot9 = 12+63+18$		93
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 126+0+2\cdot7\cdot9 = 126+0+126$		252
		$\Rightarrow\ C(2,5)=528,\quad K(2,5)=2$		
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+528+6\cdot2\cdot9 = 0+528+108$		636
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 72+420+6\cdot6\cdot9 = 72+420+324$	$\leftarrow$ max	**816**
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 108+63+6\cdot1\cdot9 = 108+63+54$		225
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 366+0+6\cdot7\cdot9 = 366+0+378$		744
		$\Rightarrow\ C(1,5)=816,\quad K(1,5)=2$		

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	72	108	366	**816**
2		0	12	126	528
3			0	42	420
4				0	63
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	2
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $((M_1\,M_2)\,((M_3\,M_4)\,M_5))$

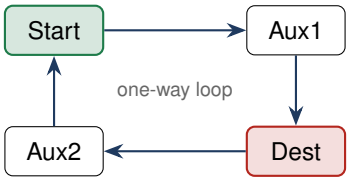
Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(6\times2)\cdot(2\times6)$ costs $6\cdot2\cdot6=72$
- $(M_3)\cdot(M_4)$: $(6\times1)\cdot(1\times7)$ costs $6\cdot1\cdot7=42$
- $((M_3\,M_4))\cdot(M_5)$: $(6\times7)\cdot(7\times9)$ costs $6\cdot7\cdot9=378$
- $((M_1\,M_2))\cdot(((M_3\,M_4)\,M_5))$: $(6\times6)\cdot(6\times9)$ costs $6\cdot6\cdot9=324$
- Total $=72+42+378+324=816$ ✓

Maximum 816, worst parenthesization $((M_1M_2)((M_3M_4)M_5))$**.** (The graded answer had 816 and full credit.)

7.3 Exam 1, Wednesday Sept. 17, 2014

Q1 (25%) Hanoi: $E = \{(\textbf{Start}, \textbf{Aux1}), (\textbf{Aux1}, \textbf{Dest}), (\textbf{Dest}, \textbf{Aux2}), (\textbf{Aux2}, \textbf{Start})\}$



One-way loop; P_0 =Start, P_1 =Aux1, P_2 =Dest, P_3 =Aux2; $d = 2$. **Main: Go2**$(n, 0)$. $T_2(n) = 2T_3(n-1) + 2$: $2, 8, 22, 60, 154, \ldots$; $\Theta(2.56^n)$, $\mathcal{O}(3^n)$. Move list for $n = 2$:

1. disk 1: Start→Aux1
2. disk 1: Aux1→Dest
3. disk 1: Dest→Aux2
4. disk 2: Start→Aux1
5. disk 2: Aux1→Dest
6. disk 1: Aux2→Start
7. disk 1: Start→Aux1
8. disk 1: Aux1→Dest

Q2 (25%) pages: 1024 words/page, 1024 frames, 2048×2048, row-major, same loop

Identical to 2018: **8,392,704** (page 24).

Q3 (25%) QuickSort, pivot = last element $A[hi]$, $n \geq 11$

$n = 15$. With the Partition that first swaps the pivot to the front (page 33):

- (a) $\langle 12, 1, 2, 3, 7, 5, 4, 6, 14, 9, 10, 11, 15, 13, 8 \rangle$; (b) $\langle 1, 2, \ldots, 15 \rangle$.

With the textbook partition that keeps the pivot at the end:

- (a) $\langle 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15, 8 \rangle$; (b) $\langle 1, 2, \ldots, 15 \rangle$.

State which partition you use.

Q4 (25%) maximum for $M_{6,2} * M_{2,8} * M_{8,1} * M_{1,7} * M_{7,9}$

Dimensions: $M_1 : 6 \times 2$, $M_2 : 2 \times 8$, $M_3 : 8 \times 1$, $M_4 : 1 \times 7$, $M_5 : 7 \times 9$. So $p = (6, 2, 8, 1, 7, 9)$, i.e. $p_0 = 6$, $p_1 = 2$, $p_2 = 8$, $p_3 = 1$, $p_4 = 7$, $p_5 = 9$.

Base case: $C(i, i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1, j) + p_{i-1}\, p_k\, p_j$	value
Chains of length 2			
$C(1,2)$	$k = 1$	$C(1,1) + C(2,2) + p_0 p_1 p_2 = 0 + 0 + 6 \cdot 2 \cdot 8 = 0 + 0 + 96$	**96**
		$\Rightarrow\ C(1,2) = 96, \quad K(1,2) = 1$	
$C(2,3)$	$k = 2$	$C(2,2) + C(3,3) + p_1 p_2 p_3 = 0 + 0 + 2 \cdot 8 \cdot 1 = 0 + 0 + 16$	**16**
		$\Rightarrow\ C(2,3) = 16, \quad K(2,3) = 2$	
$C(3,4)$	$k = 3$	$C(3,3) + C(4,4) + p_2 p_3 p_4 = 0 + 0 + 8 \cdot 1 \cdot 7 = 0 + 0 + 56$	**56**
		$\Rightarrow\ C(3,4) = 56, \quad K(3,4) = 3$	
$C(4,5)$	$k = 4$	$C(4,4) + C(5,5) + p_3 p_4 p_5 = 0 + 0 + 1 \cdot 7 \cdot 9 = 0 + 0 + 63$	**63**
		$\Rightarrow\ C(4,5) = 63, \quad K(4,5) = 4$	
Chains of length 3			

cell	split	$C(i,k)+C(k+1,j)+p_{i-1}\,p_k\,p_j$		value
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+16+6\cdot2\cdot1 = 0+16+12$		28
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 96+0+6\cdot8\cdot1 = 96+0+48$	$\leftarrow$ max	**144**
		$\Rightarrow\ C(1,3)=144,\quad K(1,3)=2$		
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+56+2\cdot8\cdot7 = 0+56+112$	$\leftarrow$ max	**168**
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 16+0+2\cdot1\cdot7 = 16+0+14$		30
		$\Rightarrow\ C(2,4)=168,\quad K(2,4)=2$		
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+63+8\cdot1\cdot9 = 0+63+72$		135
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 56+0+8\cdot7\cdot9 = 56+0+504$	$\leftarrow$ max	**560**
		$\Rightarrow\ C(3,5)=560,\quad K(3,5)=4$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+168+6\cdot2\cdot7 = 0+168+84$		252
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 96+56+6\cdot8\cdot7 = 96+56+336$	$\leftarrow$ max	**488**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 144+0+6\cdot1\cdot7 = 144+0+42$		186
		$\Rightarrow\ C(1,4)=488,\quad K(1,4)=2$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+560+2\cdot8\cdot9 = 0+560+144$	$\leftarrow$ max	**704**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 16+63+2\cdot1\cdot9 = 16+63+18$		97
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 168+0+2\cdot7\cdot9 = 168+0+126$		294
		$\Rightarrow\ C(2,5)=704,\quad K(2,5)=2$		
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+704+6\cdot2\cdot9 = 0+704+108$		812
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 96+560+6\cdot8\cdot9 = 96+560+432$	$\leftarrow$ max	**1088**
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 144+63+6\cdot1\cdot9 = 144+63+54$		261
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 488+0+6\cdot7\cdot9 = 488+0+378$		866
		$\Rightarrow\ C(1,5)=1088,\quad K(1,5)=2$		

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	96	144	488	**1088**
2		0	16	168	704
3			0	56	560
4				0	63
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	2
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $((M_1\,M_2)\,((M_3\,M_4)\,M_5))$

Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(6\times2)\cdot(2\times8)$ costs $6\cdot2\cdot8=96$
- $(M_3)\cdot(M_4)$: $(8\times1)\cdot(1\times7)$ costs $8\cdot1\cdot7=56$
- $((M_3\,M_4))\cdot(M_5)$: $(8\times7)\cdot(7\times9)$ costs $8\cdot7\cdot9=504$
- $((M_1\,M_2))\cdot(((M_3\,M_4)\,M_5))$: $(6\times8)\cdot(8\times9)$ costs $6\cdot8\cdot9=432$
- Total $=96+56+504+432=1088$ ✓

Maximum 1088, worst parenthesization $((M_1M_2)((M_3M_4)M_5))$.

7.4 Exam 1, Wednesday Feb. 21, 2018

Q1 (25%) Hanoi: $E = \{(\textbf{Start}, \textbf{Aux1}), (\textbf{Aux1}, \textbf{Aux2}), (\textbf{Aux2}, \textbf{Dest}), (\textbf{Dest}, \textbf{Start})\}$

Model answer on page 14: $d = 3$, **main Go3**$(n, 0)$, $T_3(n) = 2T_2(n-1) + T_3(n-1) + 3$: $3, 10, 29, 76, \ldots$; $\Theta(2.56^n)$, $\mathcal{O}(3^n)$.

Q2 (25%) pages

8,392,704; model answer on page 24.

Q3 (25%) QuickSort, pivot = first, $n \geq 13$

$n = 15$; same arrays as 2008 (page 32).

Q4 (25%) maximum for $M_{9,3} * M_{3,9} * M_{9,1} * M_{1,8} * M_{8,4}$

1251, $(((M_1 M_2)(M_3 M_4))M_5)$; full table on page 41.

7.5 2020 and 2021 versions (from your review sheets)

Pages

2020 review: column-major, 512 frames, 2048×2048, 1024-word pages, same loop: **4,202,496** (page 26).
2020 original: row-major, 512 frames: **8,392,704**.

QuickSort

2020: pivot $= A[lo+1]$ (second element), $n \geq 14$: page 34, $n = 15$: (a) $\langle 4, 8, 3, 1, 5, 6, 7, 2, 10, 12, 11, 9, 13, 14, 15 \rangle$; (b) $\langle 15, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \rangle$.
2021: pivot $= A[hi-1]$ (next-to-last), $n \geq 13$: page 35, $n = 15$: (a) $\langle 12, 2, 3, 1, 5, 4, 7, 6, 14, 10, 11, 9, 13, 8, 15 \rangle$; (b) $\langle 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 1, 15 \rangle$.

Matrix chain, maximum

2020 review, $p = (9, 1, 8, 2, 10, 4)$: **Dimensions:** $M_1 : 9 \times 1$, $M_2 : 1 \times 8$, $M_3 : 8 \times 2$, $M_4 : 2 \times 10$, $M_5 : 10 \times 4$. So $p = (9, 1, 8, 2, 10, 4)$, i.e. $p_0 = 9$, $p_1 = 1$, $p_2 = 8$, $p_3 = 2$, $p_4 = 10$, $p_5 = 4$.

Base case: $C(i, i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1, j) + p_{i-1}\, p_k\, p_j$	value
Chains of length 2			
$C(1,2)$	$k=1$	$C(1,1) + C(2,2) + p_0 p_1 p_2 = 0 + 0 + 9 \cdot 1 \cdot 8 = 0 + 0 + 72$	**72**
		$\Rightarrow\ C(1,2) = 72, \quad K(1,2) = 1$	
$C(2,3)$	$k=2$	$C(2,2) + C(3,3) + p_1 p_2 p_3 = 0 + 0 + 1 \cdot 8 \cdot 2 = 0 + 0 + 16$	**16**
		$\Rightarrow\ C(2,3) = 16, \quad K(2,3) = 2$	
$C(3,4)$	$k=3$	$C(3,3) + C(4,4) + p_2 p_3 p_4 = 0 + 0 + 8 \cdot 2 \cdot 10 = 0 + 0 + 160$	**160**
		$\Rightarrow\ C(3,4) = 160, \quad K(3,4) = 3$	

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5 = 0+0+2\cdot10\cdot4 = 0+0+80$		**80**
			$\Rightarrow\ C(4,5)=80,\quad K(4,5)=4$	
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+16+9\cdot1\cdot2 = 0+16+18$		34
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 72+0+9\cdot8\cdot2 = 72+0+144$	$\leftarrow$ max	**216**
			$\Rightarrow\ C(1,3)=216,\quad K(1,3)=2$	
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+160+1\cdot8\cdot10 = 0+160+80$	$\leftarrow$ max	**240**
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 16+0+1\cdot2\cdot10 = 16+0+20$		36
			$\Rightarrow\ C(2,4)=240,\quad K(2,4)=2$	
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+80+8\cdot2\cdot4 = 0+80+64$		144
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 160+0+8\cdot10\cdot4 = 160+0+320$	$\leftarrow$ max	**480**
			$\Rightarrow\ C(3,5)=480,\quad K(3,5)=4$	
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+240+9\cdot1\cdot10 = 0+240+90$		330
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 72+160+9\cdot8\cdot10 = 72+160+720$	$\leftarrow$ max	**952**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 216+0+9\cdot2\cdot10 = 216+0+180$		396
			$\Rightarrow\ C(1,4)=952,\quad K(1,4)=2$	
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+480+1\cdot8\cdot4 = 0+480+32$	$\leftarrow$ max	**512**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 16+80+1\cdot2\cdot4 = 16+80+8$		104
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 240+0+1\cdot10\cdot4 = 240+0+40$		280
			$\Rightarrow\ C(2,5)=512,\quad K(2,5)=2$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+512+9\cdot1\cdot4 = 0+512+36$		548
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 72+480+9\cdot8\cdot4 = 72+480+288$		840
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 216+80+9\cdot2\cdot4 = 216+80+72$		368
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 952+0+9\cdot10\cdot4 = 952+0+360$	$\leftarrow$ max	**1312**
			$\Rightarrow\ C(1,5)=1312,\quad K(1,5)=4$	

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	72	216	952	**1312**
2		0	16	240	512
3			0	160	480
4				0	80
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$

Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(9\times1)\cdot(1\times8)$ costs $9\cdot1\cdot8=72$
- $(M_3)\cdot(M_4)$: $(8\times2)\cdot(2\times10)$ costs $8\cdot2\cdot10=160$
- $((M_1\,M_2))\cdot((M_3\,M_4))$: $(9\times8)\cdot(8\times10)$ costs $9\cdot8\cdot10=720$
- $(((M_1\,M_2)\,(M_3\,M_4)))\cdot(M_5)$: $(9\times10)\cdot(10\times4)$ costs $9\cdot10\cdot4=360$
- Total $=72+160+720+360=1312$ ✓

1312, $(((M_1M_2)(M_3M_4))M_5)$.

2020 original, $p=(9,1,9,2,8,4)$: **Dimensions:** $M_1:9\times1$, $M_2:1\times9$, $M_3:9\times2$, $M_4:2\times8$, $M_5:8\times4$. So $p=(9,1,9,2,8,4)$, i.e. $p_0=9$, $p_1=1$, $p_2=9$, $p_3=2$, $p_4=8$, $p_5=4$.

Base case: $C(i,i)=0$ for $i=1,\ldots,5$.

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1)+C(2,2)+p_0p_1p_2 = 0+0+9\cdot1\cdot9 = 0+0+81$		**81**
			$\Rightarrow\ C(1,2)=81,\quad K(1,2)=1$	
$C(2,3)$	$k=2$	$C(2,2)+C(3,3)+p_1p_2p_3 = 0+0+1\cdot9\cdot2 = 0+0+18$		**18**
			$\Rightarrow\ C(2,3)=18,\quad K(2,3)=2$	
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4 = 0+0+9\cdot2\cdot8 = 0+0+144$		**144**
			$\Rightarrow\ C(3,4)=144,\quad K(3,4)=3$	
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5 = 0+0+2\cdot8\cdot4 = 0+0+64$		**64**
			$\Rightarrow\ C(4,5)=64,\quad K(4,5)=4$	
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3 = 0+18+9\cdot1\cdot2 = 0+18+18$		36
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3 = 81+0+9\cdot9\cdot2 = 81+0+162$	$\leftarrow$ max	**243**
			$\Rightarrow\ C(1,3)=243,\quad K(1,3)=2$	
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4 = 0+144+1\cdot9\cdot8 = 0+144+72$	$\leftarrow$ max	**216**
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4 = 18+0+1\cdot2\cdot8 = 18+0+16$		34
			$\Rightarrow\ C(2,4)=216,\quad K(2,4)=2$	
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+64+9\cdot2\cdot4 = 0+64+72$		136
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 144+0+9\cdot8\cdot4 = 144+0+288$	$\leftarrow$ max	**432**
			$\Rightarrow\ C(3,5)=432,\quad K(3,5)=4$	
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+216+9\cdot1\cdot8 = 0+216+72$		288
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 81+144+9\cdot9\cdot8 = 81+144+648$	$\leftarrow$ max	**873**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 243+0+9\cdot2\cdot8 = 243+0+144$		387
			$\Rightarrow\ C(1,4)=873,\quad K(1,4)=2$	
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+432+1\cdot9\cdot4 = 0+432+36$	$\leftarrow$ max	**468**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 18+64+1\cdot2\cdot4 = 18+64+8$		90
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 216+0+1\cdot8\cdot4 = 216+0+32$		248
			$\Rightarrow\ C(2,5)=468,\quad K(2,5)=2$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+468+9\cdot1\cdot4 = 0+468+36$		504
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 81+432+9\cdot9\cdot4 = 81+432+324$		837
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 243+64+9\cdot2\cdot4 = 243+64+72$		379
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 873+0+9\cdot8\cdot4 = 873+0+288$	$\leftarrow$ max	**1161**
			$\Rightarrow\ C(1,5)=1161,\quad K(1,5)=4$	

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	81	243	873	**1161**
2		0	18	216	468
3			0	144	432
4				0	64
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$

Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(9\times1)\cdot(1\times9)$ costs $9\cdot1\cdot9=81$
- $(M_3)\cdot(M_4)$: $(9\times2)\cdot(2\times8)$ costs $9\cdot2\cdot8=144$
- $((M_1\,M_2))\cdot((M_3\,M_4))$: $(9\times9)\cdot(9\times8)$ costs $9\cdot9\cdot8=648$
- $(((M_1\,M_2)\,(M_3\,M_4)))\cdot(M_5)$: $(9\times8)\cdot(8\times4)$ costs $9\cdot8\cdot4=288$

- Total $= 81 + 144 + 648 + 288 = 1161$ ✓

1161, $(((M_1M_2)(M_3M_4))M_5)$**.**

Huffman, HeapSort, RSA (2020)

Huffman: expected length **2.75** (page 48). HeapSort: final heap **10 9 6 7 8 2 4 1 5 3** (page 49). RSA: $\mathcal{O}(s^{2.59})$ per encryption/decryption with Gauss multiplication (page 50).

PART VIII

Five Mock Exams

CHAPTER 8

Mock Exams

How to use these. Set a timer for 85 minutes, keep this book open (it is an open-book exam), and write full answers on paper. Then grade yourself with Part 8. Mocks 1–3 and 5 follow the classic four-question template with new numbers and new graph shapes; Mock 4 imitates the broader 2020 exam.

8.1 Mock Exam 1

COSC 3320 **Mock Exam 1**

Open Books and Notes 85 minutes

1. Design a function (algorithm) that solves the Towers of Hanoi game for the following graph $G = (V, E)$ with $V = \{\text{Start, Dest, Aux1, Aux2}\}$ and $E = \{(\text{Start, Aux2}), (\text{Aux2, Dest}), (\text{Dest, Aux1}), (\text{Aux1, Start})\}$. Estimate the time complexity of your function, in terms of the number n of disks to be moved.

2. Determine for the following code how many pages are transferred between disk and main memory. Assume each page has 512 words, the active memory set size is 512 (i.e., at any time no more than 512 pages may be in main memory), and the replacement strategy is LRU; also assume that all 2D arrays are of size (1:1024, 1:1024), with each array element occupying one word,

```
for I := 1 to 1024 do
   for J := 1 to 1024 do
      { A[I,J] := A[I,J] * B[J,I] }
```

provided the arrays are mapped into the main memory space in **row**-major order.

3. Consider QuickSort on the array $A[1{:}n]$ and assume that the pivot element x (used to split the array $A[lo{:}hi]$ into two portions such that all elements in the left portion $A[lo{:}m]$ are $\leq x$ and all elements in the right portion $A[m{:}hi]$ are $\geq x$) is the **last** element of the array to be split (i.e., $A[hi]$). For a specific value of n at least 13 (your choice), construct an assignment of the numbers $1 \ldots n$ to the n array elements that causes QuickSort, with the stated choice of pivot, to (a) execute optimally (that is $A[lo{:}m]$ and $A[m{:}hi]$ are always of equal size); (b) execute in the slowest possible way.

4. Consider the problem of multiplying n rectangular matrices discussed in class. Assume that we want to determine the **maximum** number of scalar multiplications that one might need. Formulate precisely an algorithm that determines this value. Then carry out your method on the following product to show what is the worst-possible parenthesization and how many scalar multiplications are required to carry it out: $M_{5,3} * M_{3,7} * M_{7,2} * M_{2,9} * M_{9,4}$.

Percentage points: 1: 25% 2: 25% 3: 25% 4: 25%

8.2 Mock Exam 2

COSC 3320 **Mock Exam 2**

Open Books and Notes 85 minutes

1. Design a function (algorithm) that solves the Towers of Hanoi game for the following graph $G = (V, E)$ with $V = \{\text{Start, Dest, Aux1, Aux2}\}$ and $E = \{(\text{Start, Aux2}), (\text{Aux2, Dest}), (\text{Dest, Aux1}), (\text{Aux1, Aux2})\}$. Estimate the time complexity of your function, in terms of the number n of disks to be moved.

2. Determine for the following code how many pages are transferred between disk and main memory. Assume each page has 1024 words, the active memory set size is 4096, and the replacement strategy is LRU; all 2D arrays are of size (1:2048, 1:2048), each element one word,

```
for I := 1 to 2048 do
   for J := 1 to 2048 do
      { A[I,J] := A[I,J] + B[J,I] }
```

provided the arrays are mapped into the main memory space in **column**-major order.

3. QuickSort as in class, but the pivot element x is the **second** element of the array to be split (i.e., $A[lo + 1]$). For a specific value of n at least 14 (your choice), construct an assignment of the numbers $1 \ldots n$ that causes QuickSort to (a) execute optimally (both portions always of equal size); (b) execute in the slowest possible way.

4. Consider the problem of multiplying n rectangular matrices discussed in class. Assume that we want to determine the **average** number of scalar multiplications that one might need (that is, compute the average of all possible parenthesizations). Formulate a method that determines this value. Then carry out your method on the following product; also show what is the worst-possible parenthesization and how many scalar multiplications it requires: $M_{3,5} * M_{5,2} * M_{2,6} * M_{6,4} * M_{4,3}$.

Percentage points: 1: 25% 2: 25% 3: 25% 4: 25%

8.3 Mock Exam 3

COSC 3320 **Mock Exam 3**

Open Books and Notes 85 minutes

1. Design a function (algorithm) that solves the Towers of Hanoi game for the graph with $V = \{\text{Start, Dest, Aux1, Aux2}\}$ and $E = \{(\text{Start, Aux2}), (\text{Aux2, Aux1}), (\text{Aux1, Dest}), (\text{Dest, Start})\}$. Estimate the time complexity of your function in terms of n.

2. Determine how many pages are transferred between disk and main memory. Each page has 256 words, LRU replacement, all 2D arrays are of size (1:512, 1:512), one word per element, **row**-major order:

```
for I := 1 to 512 do
   for J := 1 to 512 do
      { A[I,J] := A[I,J] + B[J,I] * C[I,J] }
```

(a) when the active memory set size is 2048 pages; (b) when it is 256 pages.

3. QuickSort as in class, with pivot x the **middle** element $A[\lfloor (lo + hi)/2 \rfloor]$ of the array to be split. For a specific n at least 15 (your choice), construct an assignment of $1 \ldots n$ that makes QuickSort (a) execute optimally; (b) execute in the slowest possible way.

4. For the product $M_{4,10} * M_{10,3} * M_{3,12} * M_{12,5} * M_{5,50}$: (a) how many different parenthesizations are there? (b) Using the dynamic-programming method from class, determine the **minimum** number of scalar multiplications and a parenthesization achieving it. (c) Explain the time and space complexity of the method for n matrices.

Percentage points: 1: 25% 2: 25% 3: 25% 4: 25%

8.4 Mock Exam 4 (2020-style, six questions)

COSC 3320 **Mock Exam 4**

Open Books and Notes 85 minutes

1. Determine how many pages are transferred between disk and main memory. Each page has **2048** words, the active memory set size is 1024, LRU, arrays (1:2048, 1:2048), one word per element, **row**-major:
`for I := 1 to 2048 do for J := 1 to 2048 do { A[J,I] := A[J,I] * B[I,J] }`

2. QuickSort with pivot the **next-to-last** element $A[hi-1]$. For a specific $n \geq 13$, construct (a) an optimal input and (b) a slowest input.

3. Determine the **maximum** number of scalar multiplications and the worst parenthesization for $M_{8,2} * M_{2,9} * M_{9,3} * M_{3,7} * M_{7,5}$. State your algorithm precisely.

4. Construct a Huffman code for the symbols a–h with probabilities a 2/60, b 3/60, c 5/60, d 7/60, e 8/60, f 10/60, g 12/60, h 13/60. Determine the expected length of your code.

5. (a) Construct a heap for the sequence 4 9 2 7 1 8 3 10 6 5, inserting one element at a time; show the array after each insertion. (b) Use the heap to sort; show the heap after each extraction of the maximum.

6. (a) Compute the multiplicative inverse of 17 with respect to 3120. (b) In RSA with an s-bit modulus n, what is the bit complexity of encryption $M^e \bmod n$ if $e = 17$, and of decryption $C^d \bmod n$ if d has s bits, using the multiplication algorithm from class?

Percentage points: 1: 20% 2: 15% 3: 20% 4: 15% 5: 15% 6: 15%

8.5 Mock Exam 5

COSC 3320 **Mock Exam 5**

Open Books and Notes 85 minutes

1. Design a function (algorithm) that solves the Towers of Hanoi game for the graph with $V = \{\text{Start, Dest, Aux1, Aux2}\}$ and $E = \{(\text{Start, Aux1}), (\text{Aux1, Aux2}), (\text{Aux2, Start}), (\text{Aux2, Dest})\}$. Estimate the time complexity of your function in terms of n.

2. Pages of 512 words, LRU, arrays (1:1024, 1:1024), one word per element, **row**-major:

```
for I := 1 to 1024 do
   for J := 1 to 1024 do
      { A[J,I] := A[J,I] + B[J,I] }
```

How many pages are transferred (a) with an active memory set size of 1024 pages; (b) with 4096 pages?

3. QuickSort with pivot the **first** element $A[lo]$. For a specific value of n **at least 20** (your choice), construct (a) an assignment of $1 \ldots n$ that makes QuickSort execute optimally and (b) one that makes it execute in the slowest possible way. How many comparisons does each make?

4. For $M_{2,7} * M_{7,3} * M_{3,6} * M_{6,10} * M_{10,4}$ determine both the **maximum** and the **minimum** number of scalar multiplications over all parenthesizations, with the corresponding parenthesizations. Formulate your method precisely.

Percentage points: 1: 25% 2: 25% 3: 25% 4: 25%

PART IX

Answer Keys for the Mock Exams

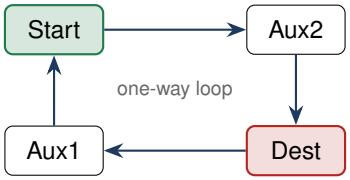
CHAPTER 9

Mock Exam Solutions

Grade yourself generously on presentation but strictly on method: for each question, did you (1) state the method, (2) show the work in a checkable form, (3) give the final answer with the complexity where asked?

9.1 Mock Exam 1: solutions

1. Hanoi on $E = \{(\textbf{Start}, \textbf{Aux2}), (\textbf{Aux2}, \textbf{Dest}), (\textbf{Dest}, \textbf{Aux1}), (\textbf{Aux1}, \textbf{Start})\}$



Every peg has one arrow in and one out: a one-way 4-loop Start→Aux2→Dest→Aux1→Start. Numbering: P_0 =Start, P_1 =Aux2, P_2 =Dest, P_3 =Aux1. Dest is $d = 2$ steps after Start.

Algorithm: Go1, Go2, Go3 exactly as on page 11; **main: Go2**$(n, 0)$. Correctness: the "parked on" table (when disk k moves along $P_i \to P_{i+1}$, the smaller tower is on a peg that is neither P_i nor P_{i+1}), plus induction on k.

Complexity: $T_2(n) = 2T_3(n-1) + 2$, $T_3(n) = 2T_2(n-1) + T_3(n-1) + 3$ (and T_1 not needed here but part of the system). Values T_2: $2, 8, 22, 60, 154, 400$. Substituting, $T_3(n) = T_3(n-1) + 4T_3(n-2) + 7$, characteristic equation $x^2 = x + 4$, so $T_2(n) = \Theta\big(((1+\sqrt{17})/2)^n\big) \approx \Theta(2.56^n)$; simple bound $\mathcal{O}(3^n)$. Time $\Theta(\#\text{moves})$, space $\Theta(n)$.

Check, $n = 2$ (8 moves):

1. disk 1: Start→Aux2
2. disk 1: Aux2→Dest
3. disk 1: Dest→Aux1
4. disk 2: Start→Aux2
5. disk 2: Aux2→Dest
6. disk 1: Aux1→Start
7. disk 1: Start→Aux2
8. disk 1: Aux2→Dest

2. Pages (row-major, 512 words, 512 frames, 1024×1024, `A[I,J] := A[I,J] * B[J,I]`)

1. Row $= 1024/512 = 2$ pages; each array 2048 pages.
2. `A[I,J]`: inner J in second position, row-major: **walking** along row I. `B[J,I]`: inner J in first position: **jumping** down column I.
3. Per inner loop: 2 pages of A, 1024 pages of B: 1026.
4. $1026 > 512$: the B pages are evicted before reuse; every B access faults.
5. A reads $1024 \cdot 2 = 2048$; A writes 2048; B reads $1024 \cdot 1024 = 1{,}048{,}576$; B writes 0.

Total $= 2048 + 2048 + 1{,}048{,}576 = 1{,}052{,}672$ **page transfers.** (Simulated: identical.)

3. QuickSort, pivot = last element, $n \geq 13$

Choose $n = 15 = 2^4 - 1$ so that every split can be exactly equal ($15 \to 7+7 \to 3+3 \to 1+1$). State the partition (pseudocode page 31: swap the pivot to the front, scan, swap it into place).

Answer

(a) $\langle\, 12, 1, 2, 3, 7, 5, 4, 6, 14, 9, 10, 11, 15, 13, 8 \,\rangle$: every pivot is the median of its part; $C(15) = 34$ comparisons, $C(n) = 2C(\frac{n-1}{2}) + n - 1 = \Theta(n \log n)$.
(b) $\langle\, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 \,\rangle$: the last element is always the maximum; $C(n) = C(n-1) + n - 1 = \frac{n(n-1)}{2} = 105$, $\Theta(n^2)$.

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	12 1 2 3 7 5 4 6 14 9 10 11 15 13 **8**	8	6 1 2 3 7 5 4 **8** 14 9 10 11 15 13 12	7 / 7
2	1..7	6 1 2 3 7 5 **4**	4	3 1 2 **4** 7 5 6	3 / 3
3	1..3	3 1 **2**	2	1 **2** 3	1 / 1
4	5..7	7 5 **6**	6	5 **6** 7	1 / 1
5	9..15	14 9 10 11 15 13 **12**	12	11 9 10 **12** 15 13 14	3 / 3
6	9..11	11 9 **10**	10	9 **10** 11	1 / 1
7	13..15	15 13 **14**	14	13 **14** 15	1 / 1

(If you used the textbook partition that keeps the pivot at the end: (a) $\langle\, 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15, 8 \,\rangle$, (b) sorted. Either is correct when the partition is stated.)

4. Maximum for $M_{5,3} * M_{3,7} * M_{7,2} * M_{2,9} * M_{9,4}$

Method: $C(i,i) = 0$, $C(i,j) = \max_{i \leq k < j}\{C(i,k) + C(k+1,j) + p_{i-1}p_k p_j\}$, filled by increasing length, $K(i,j)$ stores the best k; $\Theta(n^3)$ time, $\Theta(n^2)$ space. **Dimensions:** $M_1 : 5 \times 3$, $M_2 : 3 \times 7$, $M_3 : 7 \times 2$, $M_4 : 2 \times 9$, $M_5 : 9 \times 4$. So $p = (5,3,7,2,9,4)$, i.e. $p_0 = 5$, $p_1 = 3$, $p_2 = 7$, $p_3 = 2$, $p_4 = 9$, $p_5 = 4$.

Base case: $C(i,i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1,j) + p_{i-1}\, p_k\, p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1) + C(2,2) + p_0p_1p_2 = 0 + 0 + 5 \cdot 3 \cdot 7 = 0 + 0 + 105$		**105**
		$\Rightarrow\ C(1,2) = 105, \quad K(1,2) = 1$		
$C(2,3)$	$k=2$	$C(2,2) + C(3,3) + p_1p_2p_3 = 0 + 0 + 3 \cdot 7 \cdot 2 = 0 + 0 + 42$		**42**
		$\Rightarrow\ C(2,3) = 42, \quad K(2,3) = 2$		
$C(3,4)$	$k=3$	$C(3,3) + C(4,4) + p_2p_3p_4 = 0 + 0 + 7 \cdot 2 \cdot 9 = 0 + 0 + 126$		**126**
		$\Rightarrow\ C(3,4) = 126, \quad K(3,4) = 3$		
$C(4,5)$	$k=4$	$C(4,4) + C(5,5) + p_3p_4p_5 = 0 + 0 + 2 \cdot 9 \cdot 4 = 0 + 0 + 72$		**72**
		$\Rightarrow\ C(4,5) = 72, \quad K(4,5) = 4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1) + C(2,3) + p_0p_1p_3 = 0 + 42 + 5 \cdot 3 \cdot 2 = 0 + 42 + 30$		72
	$k=2$	$C(1,2) + C(3,3) + p_0p_2p_3 = 105 + 0 + 5 \cdot 7 \cdot 2 = 105 + 0 + 70$	$\leftarrow$ max	**175**
		$\Rightarrow\ C(1,3) = 175, \quad K(1,3) = 2$		
$C(2,4)$	$k=2$	$C(2,2) + C(3,4) + p_1p_2p_4 = 0 + 126 + 3 \cdot 7 \cdot 9 = 0 + 126 + 189$	$\leftarrow$ max	**315**
	$k=3$	$C(2,3) + C(4,4) + p_1p_3p_4 = 42 + 0 + 3 \cdot 2 \cdot 9 = 42 + 0 + 54$		96
		$\Rightarrow\ C(2,4) = 315, \quad K(2,4) = 2$		

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5 = 0+72+7\cdot2\cdot4 = 0+72+56$		128
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 126+0+7\cdot9\cdot4 = 126+0+252$	$\leftarrow$ max	**378**
		$\Rightarrow\ C(3,5)=378,\quad K(3,5)=4$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+315+5\cdot3\cdot9 = 0+315+135$		450
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 105+126+5\cdot7\cdot9 = 105+126+315$	$\leftarrow$ max	**546**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 175+0+5\cdot2\cdot9 = 175+0+90$		265
		$\Rightarrow\ C(1,4)=546,\quad K(1,4)=2$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+378+3\cdot7\cdot4 = 0+378+84$	$\leftarrow$ max	**462**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 42+72+3\cdot2\cdot4 = 42+72+24$		138
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 315+0+3\cdot9\cdot4 = 315+0+108$		423
		$\Rightarrow\ C(2,5)=462,\quad K(2,5)=2$		
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+462+5\cdot3\cdot4 = 0+462+60$		522
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 105+378+5\cdot7\cdot4 = 105+378+140$		623
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 175+72+5\cdot2\cdot4 = 175+72+40$		287
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 546+0+5\cdot9\cdot4 = 546+0+180$	$\leftarrow$ max	**726**
		$\Rightarrow\ C(1,5)=726,\quad K(1,5)=4$		

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	105	175	546	**726**
2		0	42	315	462
3			0	126	378
4				0	72
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$

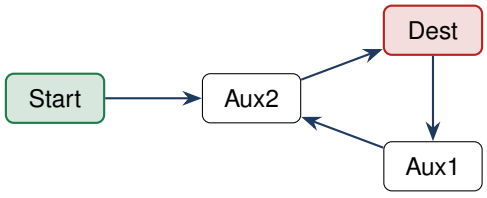
Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(5\times3)\cdot(3\times7)$ costs $5\cdot3\cdot7=105$
- $(M_3)\cdot(M_4)$: $(7\times2)\cdot(2\times9)$ costs $7\cdot2\cdot9=126$
- $((M_1\,M_2))\cdot((M_3\,M_4))$: $(5\times7)\cdot(7\times9)$ costs $5\cdot7\cdot9=315$
- $(((M_1\,M_2)\,(M_3\,M_4)))\cdot(M_5)$: $(5\times9)\cdot(9\times4)$ costs $5\cdot9\cdot4=180$
- Total $=105+126+315+180=726$ ✓

Maximum 726 with $(((M_1M_2)(M_3M_4))M_5)$. (Brute force over all 14 parenthesizations confirms; the minimum would be 184.)

9.2 Mock Exam 2: solutions

1. Hanoi on $E = \{$(Start, Aux2), (Aux2, Dest), (Dest, Aux1), (Aux1, Aux2)$\}$



Shape: nothing points into Start: a tail-in lollipop. Entry peg Aux2; loop Aux2→Dest→Aux1→Aux2; Dest is **1 step** after the entry.

Plan for the largest disk n: it travels Start→Aux2→Dest. For both moves the smaller tower must avoid Start, Aux2, Dest, so it must be on **Aux1**. Afterwards the tower goes Aux1→Aux2→Dest (2 steps on the loop: C_2). Getting the $n-1$ disks from Start to Aux1 (2 steps after the entry) is the 2012-type problem:

```
// loop Q[0]=Aux2 → Q[1]=Dest → Q[2]=Aux1 → Q[0];  C1, C2 = 3-loop procedures
procedure ToAux1(k)           // top k disks: Start → Aux1
   if k = 0 then return
   ToAux1(k-1)                // smaller tower Start → Aux1
   move disk k: Start → Aux2
   move disk k: Aux2 → Dest
   C1(k-1, 2)                 // smaller tower Aux1 → Aux2   (1 step)
   move disk k: Dest → Aux1
   C2(k-1, 0)                 // smaller tower Aux2 → Aux1   (2 steps)

procedure Solve(n)            // all n disks: Start → Dest
   if n = 0 then return
   ToAux1(n-1)
   move disk n: Start → Aux2
   move disk n: Aux2 → Dest
   C2(n-1, 2)                 // smaller tower Aux1 → Dest  (Aux1 → Aux2 → Dest)
```

Check legality: in ToAux1, when disk k goes Start→Aux2→Dest the smaller tower is on Aux1 ✓; when it goes Dest→Aux1 the tower is on Aux2 ✓.

Moves: $X(k) = X(k-1) + C_1(k-1) + C_2(k-1) + 3$ $(3, 9, 24, 63, \dots)$ and $\text{Solve}(n) = X(n-1) + 2 + C_2(n-1)$: $2, 7, 18, 47, 124, \dots$ **Complexity:** $C_1, C_2 = \Theta((1+\sqrt{3})^n)$ (from $x^2 = 2x + 2$), so $\Theta((1+\sqrt{3})^n) \approx \Theta(2.73^n)$; bound $\mathcal{O}(3^n)$.

Check, $n = 2$ (7 moves):

1. disk 1: Start→Aux2
2. disk 1: Aux2→Dest
3. disk 1: Dest→Aux1
4. disk 2: Start→Aux2
5. disk 2: Aux2→Dest
6. disk 1: Aux1→Aux2
7. disk 1: Aux2→Dest

2. Pages (column-major, 1024 words, 4096 frames, 2048×2048, `A[I,J] := A[I,J] + B[J,I]`)

1. Column $= 2048/1024 = 2$ pages; each array 4096 pages.
2. `A[I,J]`: J in second position, column-major: **jumping** across row I (one element per column). `B[J,I]`: J in first position, column-major: **walking** down column I.
3. Per inner loop: 2048 pages of A + 2 of $B = 2050$.
4. $2050 < 4096$: **it fits**. Between two uses of an A page (same column J, rows I and $I+1$ in the same half) only about 2050 other pages are used, fewer than 4096, so LRU keeps it.
5. Each A page is read once: A reads $= 4096$, writes $= 4096$; B reads $= 4096$.

Total $= 12{,}288$. (Simulated: identical. With 1024 frames instead, it would not fit and the answer would be 8,392,704.)

3. QuickSort, pivot = second element, $n \geq 14$

$n = 15$.

Answer

(a) $\langle\, 4, 8, 3, 1, 5, 6, 7, 2, 10, 12, 11, 9, 13, 14, 15 \,\rangle$ (34 comparisons, $\Theta(n \log n)$)
(b) $\langle\, 15, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \,\rangle$ (105 comparisons, $\Theta(n^2)$: the second position always holds the minimum)

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	4 8 3 1 5 6 7 2 10 12 11 9 13 14 15	8	2 4 3 1 5 6 7 **8** 10 12 11 9 13 14 15	7 / 7
2	1..7	2 4 3 1 5 6 7	4	1 2 3 **4** 5 6 7	3 / 3
3	1..3	1 2 3	2	1 **2** 3	1 / 1
4	5..7	5 6 7	6	5 **6** 7	1 / 1
5	9..15	10 12 11 9 13 14 15	12	9 10 11 **12** 13 14 15	3 / 3
6	9..11	9 10 11	10	9 **10** 11	1 / 1
7	13..15	13 14 15	14	13 **14** 15	1 / 1

4. Average for $M_{3,5} * M_{5,2} * M_{2,6} * M_{6,4} * M_{4,3}$ and the worst parenthesization

Method (page 44): count table N, sum table S, average $= S(1,5)/N(1,5)$.

cell	split	count $N(i,k)N(k{+}1,j)$	sum part $S(i,k)N(k{+}1,j) + N(i,k)S(k{+}1,j) + \text{count} \cdot p_{i-1}p_kp_j$	value
Chains of length 2				
$(1,2)$	$k=1$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 0 + 1\cdot(3\cdot 5\cdot 2)$	30
			$\Rightarrow\ N(1,2)=1,\quad S(1,2)=30,\quad \text{average}=30$	
$(2,3)$	$k=2$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 0 + 1\cdot(5\cdot 2\cdot 6)$	60
			$\Rightarrow\ N(2,3)=1,\quad S(2,3)=60,\quad \text{average}=60$	
$(3,4)$	$k=3$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 0 + 1\cdot(2\cdot 6\cdot 4)$	48
			$\Rightarrow\ N(3,4)=1,\quad S(3,4)=48,\quad \text{average}=48$	
$(4,5)$	$k=4$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 0 + 1\cdot(6\cdot 4\cdot 3)$	72
			$\Rightarrow\ N(4,5)=1,\quad S(4,5)=72,\quad \text{average}=72$	
Chains of length 3				
$(1,3)$	$k=1$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 60 + 1\cdot(3\cdot 5\cdot 6)$	150
	$k=2$	$1\cdot 1 = 1$	$30\cdot 1 + 1\cdot 0 + 1\cdot(3\cdot 2\cdot 6)$	66
			$\Rightarrow\ N(1,3)=2,\quad S(1,3)=216,\quad \text{average}=108$	
$(2,4)$	$k=2$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 48 + 1\cdot(5\cdot 2\cdot 4)$	88
	$k=3$	$1\cdot 1 = 1$	$60\cdot 1 + 1\cdot 0 + 1\cdot(5\cdot 6\cdot 4)$	180
			$\Rightarrow\ N(2,4)=2,\quad S(2,4)=268,\quad \text{average}=134$	
$(3,5)$	$k=3$	$1\cdot 1 = 1$	$0\cdot 1 + 1\cdot 72 + 1\cdot(2\cdot 6\cdot 3)$	108
	$k=4$	$1\cdot 1 = 1$	$48\cdot 1 + 1\cdot 0 + 1\cdot(2\cdot 4\cdot 3)$	72
			$\Rightarrow\ N(3,5)=2,\quad S(3,5)=180,\quad \text{average}=90$	
Chains of length 4				
$(1,4)$	$k=1$	$1\cdot 2 = 2$	$0\cdot 2 + 1\cdot 268 + 2\cdot(3\cdot 5\cdot 4)$	388
	$k=2$	$1\cdot 1 = 1$	$30\cdot 1 + 1\cdot 48 + 1\cdot(3\cdot 2\cdot 4)$	102
	$k=3$	$2\cdot 1 = 2$	$216\cdot 1 + 2\cdot 0 + 2\cdot(3\cdot 6\cdot 4)$	360
			$\Rightarrow\ N(1,4)=5,\quad S(1,4)=850,\quad \text{average}=170$	
$(2,5)$	$k=2$	$1\cdot 2 = 2$	$0\cdot 2 + 1\cdot 180 + 2\cdot(5\cdot 2\cdot 3)$	240
	$k=3$	$1\cdot 1 = 1$	$60\cdot 1 + 1\cdot 72 + 1\cdot(5\cdot 6\cdot 3)$	222
	$k=4$	$2\cdot 1 = 2$	$268\cdot 1 + 2\cdot 0 + 2\cdot(5\cdot 4\cdot 3)$	388
			$\Rightarrow\ N(2,5)=5,\quad S(2,5)=850,\quad \text{average}=170$	
Chains of length 5				
$(1,5)$	$k=1$	$1\cdot 5 = 5$	$0\cdot 5 + 1\cdot 850 + 5\cdot(3\cdot 5\cdot 3)$	1075

cell	split	count $N(i,k)N(k{+}1,j)$	sum part $S(i,k)N(k{+}1,j)+N(i,k)S(k{+}1,j)+\text{count}\cdot p_{i-1}p_kp_j$	value
	$k=2$	$1\cdot 2=2$	$30\cdot 2+1\cdot 180+2\cdot(3\cdot 2\cdot 3)$	276
	$k=3$	$2\cdot 1=2$	$216\cdot 1+2\cdot 72+2\cdot(3\cdot 6\cdot 3)$	468
	$k=4$	$5\cdot 1=5$	$850\cdot 1+5\cdot 0+5\cdot(3\cdot 4\cdot 3)$	1030
			$\Rightarrow\ N(1,5)=14,\quad S(1,5)=2849,\quad \text{average}=2849/14\approx 203.50$	

Count table $N(i,j)$

$i\backslash j$	1	2	3	4	5
1	1	1	2	5	14
2		1	1	2	5
3			1	1	2
4				1	1
5					1

Sum table $S(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	30	216	850	2849
2		0	60	268	850
3			0	48	180
4				0	72
5					0

Answer: average $=\dfrac{S(1,5)}{N(1,5)}=\dfrac{2849}{14}=\dfrac{407}{2}\approx$ **203.50** scalar multiplications.

Worst parenthesization (maximum): Dimensions: $M_1:3\times 5,\quad M_2:5\times 2,\quad M_3:2\times 6,\quad M_4:6\times 4,\quad M_5:4\times 3.\quad$ So $p=(3,5,2,6,4,3)$, i.e. $p_0=3$, $p_1=5$, $p_2=2$, $p_3=6$, $p_4=4$, $p_5=3$.

Base case: $C(i,i)=0$ for $i=1,\ldots,5$.

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1)+C(2,2)+p_0p_1p_2=0+0+3\cdot 5\cdot 2=0+0+30$		**30**
		$\Rightarrow\ C(1,2)=30,\quad K(1,2)=1$		
$C(2,3)$	$k=2$	$C(2,2)+C(3,3)+p_1p_2p_3=0+0+5\cdot 2\cdot 6=0+0+60$		**60**
		$\Rightarrow\ C(2,3)=60,\quad K(2,3)=2$		
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4=0+0+2\cdot 6\cdot 4=0+0+48$		**48**
		$\Rightarrow\ C(3,4)=48,\quad K(3,4)=3$		
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5=0+0+6\cdot 4\cdot 3=0+0+72$		**72**
		$\Rightarrow\ C(4,5)=72,\quad K(4,5)=4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3=0+60+3\cdot 5\cdot 6=0+60+90$	$\leftarrow$ max	**150**
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3=30+0+3\cdot 2\cdot 6=30+0+36$		66
		$\Rightarrow\ C(1,3)=150,\quad K(1,3)=1$		
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4=0+48+5\cdot 2\cdot 4=0+48+40$		88
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4=60+0+5\cdot 6\cdot 4=60+0+120$	$\leftarrow$ max	**180**
		$\Rightarrow\ C(2,4)=180,\quad K(2,4)=3$		
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5=0+72+2\cdot 6\cdot 3=0+72+36$	$\leftarrow$ max	**108**
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5=48+0+2\cdot 4\cdot 3=48+0+24$		72
		$\Rightarrow\ C(3,5)=108,\quad K(3,5)=3$		
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4=0+180+3\cdot 5\cdot 4=0+180+60$	$\leftarrow$ max	**240**
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4=30+48+3\cdot 2\cdot 4=30+48+24$		102
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4=150+0+3\cdot 6\cdot 4=150+0+72$		222
		$\Rightarrow\ C(1,4)=240,\quad K(1,4)=1$		
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5=0+108+5\cdot 2\cdot 3=0+108+30$		138
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5=60+72+5\cdot 6\cdot 3=60+72+90$		222
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5=180+0+5\cdot 4\cdot 3=180+0+60$	$\leftarrow$ max	**240**

cell	split	$C(i,k)+C(k+1,j)+p_{i-1}\,p_k\,p_j$		value
			$\Rightarrow\ C(2,5)=240,\quad K(2,5)=4$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5=0+240+3\cdot5\cdot3=0+240+45$	$\leftarrow$ max	**285**
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5=30+108+3\cdot2\cdot3=30+108+18$		156
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5=150+72+3\cdot6\cdot3=150+72+54$		276
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5=240+0+3\cdot4\cdot3=240+0+36$		276
			$\Rightarrow\ C(1,5)=285,\quad K(1,5)=1$	

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	30	150	240	**285**
2		0	60	180	240
3			0	48	108
4				0	72
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	1	1	1
2		–	2	3	4
3			–	3	3
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(M_1\,(((M_2\,M_3)\,M_4)\,M_5))$

Check by multiplying it out:

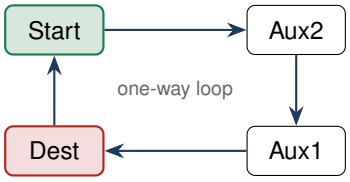
- $(M_2)\cdot(M_3)$: $(5\times2)\cdot(2\times6)$ costs $5\cdot2\cdot6=60$
- $((M_2\,M_3))\cdot(M_4)$: $(5\times6)\cdot(6\times4)$ costs $5\cdot6\cdot4=120$
- $(((M_2\,M_3)\,M_4))\cdot(M_5)$: $(5\times4)\cdot(4\times3)$ costs $5\cdot4\cdot3=60$
- $(M_1)\cdot((((M_2\,M_3)\,M_4)\,M_5))$: $(3\times5)\cdot(5\times3)$ costs $3\cdot5\cdot3=45$
- Total $=60+120+60+45=285$ ✓

Average $=2849/14=203.5$**; worst** $(M_1(((M_2M_3)M_4)M_5))$ **with 285.**

#	parenthesization	scalar mults
1	$((M_1\,M_2)\,((M_3\,M_4)\,M_5))$	120 **(MIN)**
2	$(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$	138
3	$(M_1\,(M_2\,((M_3\,M_4)\,M_5)))$	147
4	$((M_1\,M_2)\,(M_3\,(M_4\,M_5)))$	156
5	$((((M_1\,M_2)\,M_3)\,M_4)\,M_5)$	174
6	$(M_1\,(M_2\,(M_3\,(M_4\,M_5))))$	183
7	$((M_1\,(M_2\,(M_3\,M_4)))\,M_5)$	184
8	$(((M_1\,M_2)\,M_3)\,(M_4\,M_5))$	192
9	$(M_1\,((M_2\,(M_3\,M_4))\,M_5))$	193
10	$(((M_1\,(M_2\,M_3))\,M_4)\,M_5)$	258
11	$(M_1\,((M_2\,M_3)\,(M_4\,M_5)))$	267
12	$((M_1\,(M_2\,M_3))\,(M_4\,M_5))$	276
13	$((M_1\,((M_2\,M_3)\,M_4))\,M_5)$	276
14	$(M_1\,(((M_2\,M_3)\,M_4)\,M_5))$	285 **(MAX)**
	sum of all 14	**2849**
	average	**2849/14 ≈ 203.50**

9.3 Mock Exam 3: solutions

1. Hanoi on $E = \{(\textbf{Start}, \textbf{Aux2}), (\textbf{Aux2}, \textbf{Aux1}), (\textbf{Aux1}, \textbf{Dest}), (\textbf{Dest}, \textbf{Start})\}$



One-way 4-loop; P_0 =Start, P_1 =Aux2, P_2 =Aux1, P_3 =Dest; $d = 3$. **Main: Go3**$(n, 0)$. T_3: $3, 10, 29, 76, 199, \ldots$; $\Theta(2.56^n)$, $\mathcal{O}(3^n)$ (same derivation as 2018).

Check, $n = 2$ (10 moves):

1. disk 1: Start→Aux2
2. disk 1: Aux2→Aux1
3. disk 2: Start→Aux2
4. disk 1: Aux1→Dest
5. disk 1: Dest→Start
6. disk 2: Aux2→Aux1
7. disk 2: Aux1→Dest
8. disk 1: Start→Aux2
9. disk 1: Aux2→Aux1
10. disk 1: Aux1→Dest

2. Pages (row-major, 256 words, 512×512, `A[I,J] := A[I,J] + B[J,I] * C[I,J]`)

Row $= 512/256 = 2$ pages; each array 1024 pages. `A[I,J]` and `C[I,J]` walk (2 pages each per inner loop); `B[J,I]` jumps (512 pages per inner loop). Per inner loop: $2 + 512 + 2 = 516$ pages.

(a) 2048 frames: $516 < 2048$, fits. Every page of every array is read once. A: 1024 reads + 1024 writes; B: 1024 reads; C: 1024 reads. **Total 4,096.**

(b) 256 frames: $516 > 256$, does not fit: every B access faults: $512 \cdot 512 = 262{,}144$ reads. A: $1024 + 1024$, C: 1024. **Total** $1024 + 1024 + 262{,}144 + 1024 = 265{,}216$. (Both simulated.)

3. QuickSort, pivot = middle element, $n \geq 15$

$n = 15$.

Answer

(a) the sorted array $\langle 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 \rangle$: the middle of a sorted part is its median, and (with our Partition) both halves stay sorted.
(b) $\langle 2, 12, 4, 10, 6, 14, 8, 1, 3, 5, 7, 9, 11, 13, 15 \rangle$: the middle position always holds the minimum of the part.

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	1 2 3 4 5 6 7 **8** 9 10 11 12 13 14 15	8	1 2 3 4 5 6 7 **8** 9 10 11 12 13 14 15	7 / 7
2	1..7	1 2 3 **4** 5 6 7	4	1 2 3 **4** 5 6 7	3 / 3
3	1..3	1 **2** 3	2	1 **2** 3	1 / 1
4	5..7	5 **6** 7	6	5 **6** 7	1 / 1
5	9..15	9 10 11 **12** 13 14 15	12	9 10 11 **12** 13 14 15	3 / 3
6	9..11	9 **10** 11	10	9 **10** 11	1 / 1
7	13..15	13 **14** 15	14	13 **14** 15	1 / 1

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	2 12 4 10 6 14 8 **1** 3 5 7 9 11 13 15	1	**1** 12 4 10 6 14 8 2 3 5 7 9 11 13 15	0 / 14
2	2..15	12 4 10 6 14 8 **2** 3 5 7 9 11 13 15	2	**2** 4 10 6 14 8 12 3 5 7 9 11 13 15	0 / 13
3	3..15	4 10 6 14 8 12 **3** 5 7 9 11 13 15	3	**3** 10 6 14 8 12 4 5 7 9 11 13 15	0 / 12
4	4..15	10 6 14 8 12 **4** 5 7 9 11 13 15	4	**4** 6 14 8 12 10 5 7 9 11 13 15	0 / 11
5	5..15	6 14 8 12 10 **5** 7 9 11 13 15	5	**5** 14 8 12 10 6 7 9 11 13 15	0 / 10
6	6..15	14 8 12 10 **6** 7 9 11 13 15	6	**6** 8 12 10 14 7 9 11 13 15	0 / 9
7	7..15	8 12 10 14 **7** 9 11 13 15	7	**7** 12 10 14 8 9 11 13 15	0 / 8
8	8..15	12 10 14 **8** 9 11 13 15	8	**8** 10 14 12 9 11 13 15	0 / 7
9	9..15	10 14 12 **9** 11 13 15	9	**9** 14 12 10 11 13 15	0 / 6
10	10..15	14 12 **10** 11 13 15	10	**10** 12 14 11 13 15	0 / 5
11	11..15	12 14 **11** 13 15	11	**11** 14 12 13 15	0 / 4
12	12..15	14 **12** 13 15	12	**12** 14 13 15	0 / 3
13	13..15	14 **13** 15	13	**13** 14 15	0 / 2
14	14..15	**14** 15	14	**14** 15	0 / 1

4. Minimum for $M_{4,10} * M_{10,3} * M_{3,12} * M_{12,5} * M_{5,50}$

(a) The number of parenthesizations of 5 matrices is the Catalan number $N(5) = 14$ (from $N(1) = 1$, $N(n) = \sum_{k=1}^{n-1} N(k)N(n-k)$: $1, 1, 2, 5, 14$).

(b) **Dimensions:** $M_1 : 4 \times 10$, $M_2 : 10 \times 3$, $M_3 : 3 \times 12$, $M_4 : 12 \times 5$, $M_5 : 5 \times 50$. So $p = (4, 10, 3, 12, 5, 50)$, i.e. $p_0 = 4$, $p_1 = 10$, $p_2 = 3$, $p_3 = 12$, $p_4 = 5$, $p_5 = 50$.

Base case: $C(i, i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1, j) + p_{i-1}\, p_k\, p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1) + C(2,2) + p_0 p_1 p_2 = 0 + 0 + 4 \cdot 10 \cdot 3 = 0 + 0 + 120$		**120**
		$\Rightarrow\ C(1,2) = 120,$	$K(1,2) = 1$	
$C(2,3)$	$k=2$	$C(2,2) + C(3,3) + p_1 p_2 p_3 = 0 + 0 + 10 \cdot 3 \cdot 12 = 0 + 0 + 360$		**360**
		$\Rightarrow\ C(2,3) = 360,$	$K(2,3) = 2$	
$C(3,4)$	$k=3$	$C(3,3) + C(4,4) + p_2 p_3 p_4 = 0 + 0 + 3 \cdot 12 \cdot 5 = 0 + 0 + 180$		**180**
		$\Rightarrow\ C(3,4) = 180,$	$K(3,4) = 3$	
$C(4,5)$	$k=4$	$C(4,4) + C(5,5) + p_3 p_4 p_5 = 0 + 0 + 12 \cdot 5 \cdot 50 = 0 + 0 + 3000$		**3000**
		$\Rightarrow\ C(4,5) = 3000,$	$K(4,5) = 4$	
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1) + C(2,3) + p_0 p_1 p_3 = 0 + 360 + 4 \cdot 10 \cdot 12 = 0 + 360 + 480$		840
	$k=2$	$C(1,2) + C(3,3) + p_0 p_2 p_3 = 120 + 0 + 4 \cdot 3 \cdot 12 = 120 + 0 + 144$	$\leftarrow$ min	**264**
		$\Rightarrow\ C(1,3) = 264,$	$K(1,3) = 2$	
$C(2,4)$	$k=2$	$C(2,2) + C(3,4) + p_1 p_2 p_4 = 0 + 180 + 10 \cdot 3 \cdot 5 = 0 + 180 + 150$	$\leftarrow$ min	**330**
	$k=3$	$C(2,3) + C(4,4) + p_1 p_3 p_4 = 360 + 0 + 10 \cdot 12 \cdot 5 = 360 + 0 + 600$		960
		$\Rightarrow\ C(2,4) = 330,$	$K(2,4) = 2$	
$C(3,5)$	$k=3$	$C(3,3) + C(4,5) + p_2 p_3 p_5 = 0 + 3000 + 3 \cdot 12 \cdot 50 = 0 + 3000 + 1800$		4800
	$k=4$	$C(3,4) + C(5,5) + p_2 p_4 p_5 = 180 + 0 + 3 \cdot 5 \cdot 50 = 180 + 0 + 750$	$\leftarrow$ min	**930**
		$\Rightarrow\ C(3,5) = 930,$	$K(3,5) = 4$	
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1) + C(2,4) + p_0 p_1 p_4 = 0 + 330 + 4 \cdot 10 \cdot 5 = 0 + 330 + 200$		530
	$k=2$	$C(1,2) + C(3,4) + p_0 p_2 p_4 = 120 + 180 + 4 \cdot 3 \cdot 5 = 120 + 180 + 60$	$\leftarrow$ min	**360**
	$k=3$	$C(1,3) + C(4,4) + p_0 p_3 p_4 = 264 + 0 + 4 \cdot 12 \cdot 5 = 264 + 0 + 240$		504
		$\Rightarrow\ C(1,4) = 360,$	$K(1,4) = 2$	

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+930+10\cdot3\cdot50 = 0+930+1500$	$\leftarrow$ min	**2430**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 360+3000+10\cdot12\cdot50 = 360+3000+6000$		9360
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 330+0+10\cdot5\cdot50 = 330+0+2500$		2830
			$\Rightarrow\ C(2,5)=2430,\quad K(2,5)=2$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+2430+4\cdot10\cdot50 = 0+2430+2000$		4430
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 120+930+4\cdot3\cdot50 = 120+930+600$		1650
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 264+3000+4\cdot12\cdot50 = 264+3000+2400$		5664
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 360+0+4\cdot5\cdot50 = 360+0+1000$	$\leftarrow$ min	**1360**
			$\Rightarrow\ C(1,5)=1360,\quad K(1,5)=4$	

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	120	264	360	**1360**
2		0	360	330	2430
3			0	180	930
4				0	3000
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$

Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(4\times10)\cdot(10\times3)$ costs $4\cdot10\cdot3=120$
- $(M_3)\cdot(M_4)$: $(3\times12)\cdot(12\times5)$ costs $3\cdot12\cdot5=180$
- $((M_1\,M_2))\cdot((M_3\,M_4))$: $(4\times3)\cdot(3\times5)$ costs $4\cdot3\cdot5=60$
- $(((M_1\,M_2)\,(M_3\,M_4)))\cdot(M_5)$: $(4\times5)\cdot(5\times50)$ costs $4\cdot5\cdot50=1000$
- Total $=120+180+60+1000=1360$ ✓

Minimum 1360 with $(((M_1M_2)(M_3M_4))M_5)$. (For comparison the maximum is 11,360.)

(c) $\binom{n}{2}+n=\Theta(n^2)$ table cells, each trying up to $n-1$ split points: $\Theta(n^3)$ time, $\Theta(n^2)$ space.

9.4 Mock Exam 4: solutions

1. Pages (row-major, 2048-word pages, 1024 frames, `A[J,I] := A[J,I] * B[I,J]`)

Now a whole row (2048 words) is exactly **one** page. `A[J,I]` jumps: one page per row, 2048 pages per inner loop >1024: every access faults. `B[I,J]` walks: 1 page per inner loop. *A*: $2048\cdot2048=4{,}194{,}304$ **reads** $+$ 4,194,304 **writes;** *B*: 2048 **reads. Total** 8,390,656. (Simulated.)

2. QuickSort, pivot = next-to-last, $n\geq13$

$n=15$: (a) $\langle\,12,2,3,1,5,4,7,6,14,10,11,9,13,8,15\,\rangle$; (b) $\langle\,2,3,4,5,6,7,8,9,10,11,12,13,14,1,15\,\rangle$. Traces:

#	positions	subarray *before* partition (pivot in color)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	12 2 3 1 5 4 7 6 14 10 11 9 13 8 15	8	6 2 3 1 5 4 7 **8** 14 10 11 9 13 12 15	7 / 7

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
2	1..7	6 2 3 1 5 **4** 7	4	1 2 3 **4** 5 6 7	3 / 3
3	1..3	1 **2** 3	2	1 **2** 3	1 / 1
4	5..7	5 **6** 7	6	5 **6** 7	1 / 1
5	9..15	14 10 11 9 13 **12** 15	12	9 10 11 **12** 13 14 15	3 / 3
6	9..11	9 **10** 11	10	9 **10** 11	1 / 1
7	13..15	13 **14** 15	14	13 **14** 15	1 / 1

#	positions	subarray *before* partition (pivot in **color**)	pivot	subarray *after* partition (pivot boxed)	left/right
1	1..15	2 3 4 5 6 7 8 9 10 11 12 13 14 **1** 15	1	**1** 3 4 5 6 7 8 9 10 11 12 13 14 2 15	0 / 14
2	2..15	3 4 5 6 7 8 9 10 11 12 13 14 **2** 15	2	**2** 4 5 6 7 8 9 10 11 12 13 14 3 15	0 / 13
3	3..15	4 5 6 7 8 9 10 11 12 13 14 **3** 15	3	**3** 5 6 7 8 9 10 11 12 13 14 4 15	0 / 12
4	4..15	5 6 7 8 9 10 11 12 13 14 **4** 15	4	**4** 6 7 8 9 10 11 12 13 14 5 15	0 / 11
5	5..15	6 7 8 9 10 11 12 13 14 **5** 15	5	**5** 7 8 9 10 11 12 13 14 6 15	0 / 10
6	6..15	7 8 9 10 11 12 13 14 **6** 15	6	**6** 8 9 10 11 12 13 14 7 15	0 / 9
7	7..15	8 9 10 11 12 13 14 **7** 15	7	**7** 9 10 11 12 13 14 8 15	0 / 8
8	8..15	9 10 11 12 13 14 **8** 15	8	**8** 10 11 12 13 14 9 15	0 / 7
9	9..15	10 11 12 13 14 **9** 15	9	**9** 11 12 13 14 10 15	0 / 6
10	10..15	11 12 13 14 **10** 15	10	**10** 12 13 14 11 15	0 / 5
11	11..15	12 13 14 **11** 15	11	**11** 13 14 12 15	0 / 4
12	12..15	13 14 **12** 15	12	**12** 14 13 15	0 / 3
13	13..15	14 **13** 15	13	**13** 14 15	0 / 2
14	14..15	**14** 15	14	**14** 15	0 / 1

3. Maximum for $M_{8,2} * M_{2,9} * M_{9,3} * M_{3,7} * M_{7,5}$

Dimensions: $M_1 : 8 \times 2$, $M_2 : 2 \times 9$, $M_3 : 9 \times 3$, $M_4 : 3 \times 7$, $M_5 : 7 \times 5$. So $p = (8,2,9,3,7,5)$, i.e. $p_0 = 8$, $p_1 = 2$, $p_2 = 9$, $p_3 = 3$, $p_4 = 7$, $p_5 = 5$.

Base case: $C(i,i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k+1,j) + p_{i-1}\, p_k\, p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1) + C(2,2) + p_0 p_1 p_2 = 0 + 0 + 8 \cdot 2 \cdot 9 = 0 + 0 + 144$		**144**
		$\Rightarrow\ C(1,2) = 144, \quad K(1,2) = 1$		
$C(2,3)$	$k=2$	$C(2,2) + C(3,3) + p_1 p_2 p_3 = 0 + 0 + 2 \cdot 9 \cdot 3 = 0 + 0 + 54$		**54**
		$\Rightarrow\ C(2,3) = 54, \quad K(2,3) = 2$		
$C(3,4)$	$k=3$	$C(3,3) + C(4,4) + p_2 p_3 p_4 = 0 + 0 + 9 \cdot 3 \cdot 7 = 0 + 0 + 189$		**189**
		$\Rightarrow\ C(3,4) = 189, \quad K(3,4) = 3$		
$C(4,5)$	$k=4$	$C(4,4) + C(5,5) + p_3 p_4 p_5 = 0 + 0 + 3 \cdot 7 \cdot 5 = 0 + 0 + 105$		**105**
		$\Rightarrow\ C(4,5) = 105, \quad K(4,5) = 4$		
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1) + C(2,3) + p_0 p_1 p_3 = 0 + 54 + 8 \cdot 2 \cdot 3 = 0 + 54 + 48$		102
	$k=2$	$C(1,2) + C(3,3) + p_0 p_2 p_3 = 144 + 0 + 8 \cdot 9 \cdot 3 = 144 + 0 + 216$	$\leftarrow$ max	**360**
		$\Rightarrow\ C(1,3) = 360, \quad K(1,3) = 2$		
$C(2,4)$	$k=2$	$C(2,2) + C(3,4) + p_1 p_2 p_4 = 0 + 189 + 2 \cdot 9 \cdot 7 = 0 + 189 + 126$	$\leftarrow$ max	**315**
	$k=3$	$C(2,3) + C(4,4) + p_1 p_3 p_4 = 54 + 0 + 2 \cdot 3 \cdot 7 = 54 + 0 + 42$		96
		$\Rightarrow\ C(2,4) = 315, \quad K(2,4) = 2$		
$C(3,5)$	$k=3$	$C(3,3) + C(4,5) + p_2 p_3 p_5 = 0 + 105 + 9 \cdot 3 \cdot 5 = 0 + 105 + 135$		240

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5 = 189+0+9\cdot7\cdot5 = 189+0+315$	$\leftarrow$ max	**504**
			$\Rightarrow\ C(3,5)=504,\quad K(3,5)=4$	
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4 = 0+315+8\cdot2\cdot7 = 0+315+112$		427
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4 = 144+189+8\cdot9\cdot7 = 144+189+504$	$\leftarrow$ max	**837**
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4 = 360+0+8\cdot3\cdot7 = 360+0+168$		528
			$\Rightarrow\ C(1,4)=837,\quad K(1,4)=2$	
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5 = 0+504+2\cdot9\cdot5 = 0+504+90$	$\leftarrow$ max	**594**
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5 = 54+105+2\cdot3\cdot5 = 54+105+30$		189
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5 = 315+0+2\cdot7\cdot5 = 315+0+70$		385
			$\Rightarrow\ C(2,5)=594,\quad K(2,5)=2$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5 = 0+594+8\cdot2\cdot5 = 0+594+80$		674
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5 = 144+504+8\cdot9\cdot5 = 144+504+360$		1008
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5 = 360+105+8\cdot3\cdot5 = 360+105+120$		585
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5 = 837+0+8\cdot7\cdot5 = 837+0+280$	$\leftarrow$ max	**1117**
			$\Rightarrow\ C(1,5)=1117,\quad K(1,5)=4$	

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	144	360	837	**1117**
2		0	54	315	594
3			0	189	504
4				0	105
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	2	2	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(((M_1\,M_2)\,(M_3\,M_4))\,M_5)$

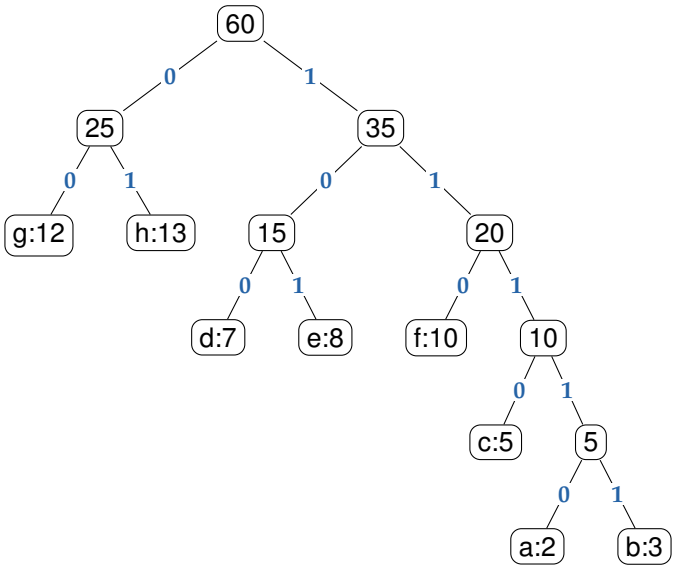
Check by multiplying it out:

- $(M_1)\cdot(M_2)$: $(8\times2)\cdot(2\times9)$ costs $8\cdot2\cdot9=144$
- $(M_3)\cdot(M_4)$: $(9\times3)\cdot(3\times7)$ costs $9\cdot3\cdot7=189$
- $((M_1\,M_2))\cdot((M_3\,M_4))$: $(8\times9)\cdot(9\times7)$ costs $8\cdot9\cdot7=504$
- $(((M_1\,M_2)\,(M_3\,M_4)))\cdot(M_5)$: $(8\times7)\cdot(7\times5)$ costs $8\cdot7\cdot5=280$
- Total $=144+189+504+280=1117$ ✓

Maximum 1117 with $(((M_1M_2)(M_3M_4))M_5)$.

4. Huffman for weights 2, 3, 5, 7, 8, 10, 12, 13 (out of 60)

Merges: $a{+}b=5$; $c{+}(ab)=10$; $d{+}e=15$; $f{+}(cab)=20$; $g{+}h=25$; $(de){+}(fcab)=35$; $25{+}35=60$.



Codes: a 11110, b 11111, c 1110, d 100, e 101, f 110, g 00, h 01. Lengths $5,5,4,3,3,3,2,2$.

$$E = \frac{2\cdot 5 + 3\cdot 5 + 5\cdot 4 + 7\cdot 3 + 8\cdot 3 + 10\cdot 3 + 12\cdot 2 + 13\cdot 2}{60} = \frac{170}{60} = \frac{17}{6} \approx \mathbf{2.83}\ \text{bits}.$$

Check: merged weights $5+10+15+20+25+35+60 = 170$ ✓.

5. Heap for 4 9 2 7 1 8 3 10 6 5

insert	swaps up	heap	removed	last→root	swaps down	heap after
4	–	4	10	1	9, 7, 6	9 7 8 6 5 2 3 4 1
9	4	9 4	9	1	8, 3	8 7 3 6 5 2 1 4
2	–	9 4 2	8	4	7, 6	7 6 3 4 5 2 1
7	4	9 7 2 4	7	1	6, 5	6 5 3 4 1 2
1	–	9 7 2 4 1	6	2	5, 4	5 4 3 2 1
8	2	9 7 8 4 1 2	5	1	4, 2	4 2 3 1
3	–	9 7 8 4 1 2 3	4	1	3	3 2 1
10	4, 7, 9	10 9 8 7 1 2 3 4	3	1	2	2 1
6	–	10 9 8 7 1 2 3 4 6	2	1	–	1
5	1	10 9 8 7 5 2 3 4 6 1	1	–	–	(empty)

Sorted output (maxima placed from the back): $1, 2, \ldots, 10$. Each operation $\mathcal{O}(\log n)$, HeapSort $\mathcal{O}(n \log n)$.

6. (a) Inverse of 17 mod 3120; (b) RSA

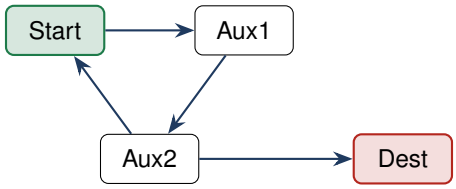
r	q	x	y
3120		1	0
17		0	1
9	183	1	-183
8	1	-1	184
1	1	2	-367

$1 = 2\cdot 3120 - 367\cdot 17$, so $17^{-1} \equiv -367 \equiv 2753 \pmod{3120}$ (check: $17\cdot 2753 = 46801 = 15\cdot 3120 + 1$).

(b) Repeated squaring with exponent $e = 17 = 10001_2$ uses 4 squarings and 1 extra multiplication: a **constant** number of multiplications of s-bit numbers, each $\mathcal{O}(s^{\log_2 3}) = \mathcal{O}(s^{1.59})$ with Gauss's method (and reductions mod n of comparable cost). Encryption: $\mathcal{O}(s^{1.59})$. Decryption with an s-bit d: $\mathcal{O}(s)$ multiplications: $\mathcal{O}(s \cdot s^{1.59}) = \mathcal{O}(s^{2.59})$.

9.5 Mock Exam 5: solutions

1. Hanoi on $E = \{(\textbf{Start}, \textbf{Aux1}), (\textbf{Aux1}, \textbf{Aux2}), (\textbf{Aux2}, \textbf{Start}), (\textbf{Aux2}, \textbf{Dest})\}$



Shape: Dest has no outgoing arrow: tail-out lollipop; loop Start→Aux1→Aux2→Start, exit Aux2→Dest. Disks must enter Dest largest first. Algorithm Out on page 17 (only case "2 steps to the exit" is used): for disk k on Start with the smaller tower on top: $C_2(k-1)$ tower Start→Aux2; disk k Start→Aux1; $C_1(k-1)$ tower Aux2→Start; disk k Aux1→Aux2→Dest; repeat with $k-1$.

Moves: $\text{Out}(n) = \text{Out}(n-1) + C_1(n-1) + C_2(n-1) + 3$: $3, 9, 24, 63, \ldots$; **complexity** $\Theta((1+\sqrt{3})^n)$, bound $\mathcal{O}(3^n)$.

Check, $n = 2$ (9 moves):

1. disk 1: Start→Aux1
2. disk 1: Aux1→Aux2
3. disk 2: Start→Aux1
4. disk 1: Aux2→Start
5. disk 2: Aux1→Aux2
6. disk 2: Aux2→Dest
7. disk 1: Start→Aux1
8. disk 1: Aux1→Aux2
9. disk 1: Aux2→Dest

2. Pages (row-major, 512 words, 1024×1024, `A[J,I] := A[J,I] + B[J,I]`)

Both references have J in the first position: both **jump** (1024 pages each per inner loop, 2048 in total).

(a) 1024 frames: $2048 > 1024$, every access to A and B faults. A: 1,048,576 **reads** + 1,048,576 **writes;** B: 1,048,576 **reads; total** 3,145,728.

(b) 4096 frames: $2048 < 4096$, fits; each page once. A: $2048 + 2048$; B: 2048; **total** 6,144. (Both simulated.)

3. QuickSort, pivot = first, $n \geq 20$

$n = 31 = 2^5 - 1$ (splits $31 \to 15+15 \to 7+7 \to 3+3 \to 1+1$).

Answer

(a) $\langle\, 16, 1, 3, 2, 6, 5, 7, 4, 12, 9, 11, 10, 14, 13, 15, 8, 24, 17, 19, 18, 22, 21, 23, 20, 28, 25, 27, 26, 30, 29, 31 \,\rangle$ comparisons $C(31) = 30 + 2 \cdot 34 = 98$.
(b) $\langle 1, 2, \ldots, 31 \rangle$: comparisons $30 + 29 + \cdots + 1 = 465$.

Trace of (a):

#	positions
1	1..31 pivot 16 split 15 / 15 before: **16** 1 3 2 6 5 7 4 12 9 11 10 14 13 15 8 24 17 19 18 22 21 23 20 28 25 27 26 30 29 31 after: 8 1 3 2 6 5 7 4 12 9 11 10 14 13 15 **16** 24 17 19 18 22 21 23 20 28 25 27 26 30 29 31
2	1..15 pivot 8 split 7 / 7 before: **8** 1 3 2 6 5 7 4 12 9 11 10 14 13 15 after: 4 1 3 2 6 5 7 **8** 12 9 11 10 14 13 15
3	1..7 pivot 4 split 3 / 3 before: **4** 1 3 2 6 5 7 after: 2 1 3 **4** 6 5 7
4	1..3 pivot 2 split 1 / 1 before: **2** 1 3 after: 1 **2** 3
5	5..7 pivot 6 split 1 / 1 before: **6** 5 7 after: 5 **6** 7
6	9..15 pivot 12 split 3 / 3 before: **12** 9 11 10 14 13 15 after: 10 9 11 **12** 14 13 15
7	9..11 pivot 10 split 1 / 1 before: **10** 9 11 after: 9 **10** 11
8	13..15 pivot 14 split 1 / 1 before: **14** 13 15 after: 13 **14** 15
9	17..31 pivot 24 split 7 / 7 before: **24** 17 19 18 22 21 23 20 28 25 27 26 30 29 31 after: 20 17 19 18 22 21 23 **24** 28 25 27 26 30 29 31
10	17..23 pivot 20 split 3 / 3 before: **20** 17 19 18 22 21 23 after: 18 17 19 **20** 22 21 23
11	17..19 pivot 18 split 1 / 1 before: **18** 17 19 after: 17 **18** 19
12	21..23 pivot 22 split 1 / 1 before: **22** 21 23 after: 21 **22** 23
13	25..31 pivot 28 split 3 / 3 before: **28** 25 27 26 30 29 31 after: 26 25 27 **28** 30 29 31
14	25..27 pivot 26 split 1 / 1 before: **26** 25 27 after: 25 **26** 27
15	29..31 pivot 30 split 1 / 1 before: **30** 29 31 after: 29 **30** 31

4. Maximum and minimum for $M_{2,7} * M_{7,3} * M_{3,6} * M_{6,10} * M_{10,4}$

Maximum: Dimensions: $M_1 : 2 \times 7$, $M_2 : 7 \times 3$, $M_3 : 3 \times 6$, $M_4 : 6 \times 10$, $M_5 : 10 \times 4$. So $p = (2, 7, 3, 6, 10, 4)$, i.e. $p_0 = 2$, $p_1 = 7$, $p_2 = 3$, $p_3 = 6$, $p_4 = 10$, $p_5 = 4$.

Base case: $C(i, i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k+1,j) + p_{i-1}\, p_k\, p_j$	value
Chains of length 2			
$C(1,2)$	$k = 1$	$C(1,1) + C(2,2) + p_0 p_1 p_2 = 0 + 0 + 2 \cdot 7 \cdot 3 = 0 + 0 + 42$	**42**

cell	split	$C(i,k)+C(k{+}1,j)+p_{i-1}\,p_k\,p_j$		value
			$\Rightarrow\ C(1,2)=42,\quad K(1,2)=1$	
$C(2,3)$	$k=2$	$C(2,2)+C(3,3)+p_1p_2p_3=0+0+7\cdot3\cdot6=0+0+126$		**126**
			$\Rightarrow\ C(2,3)=126,\quad K(2,3)=2$	
$C(3,4)$	$k=3$	$C(3,3)+C(4,4)+p_2p_3p_4=0+0+3\cdot6\cdot10=0+0+180$		**180**
			$\Rightarrow\ C(3,4)=180,\quad K(3,4)=3$	
$C(4,5)$	$k=4$	$C(4,4)+C(5,5)+p_3p_4p_5=0+0+6\cdot10\cdot4=0+0+240$		**240**
			$\Rightarrow\ C(4,5)=240,\quad K(4,5)=4$	
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1)+C(2,3)+p_0p_1p_3=0+126+2\cdot7\cdot6=0+126+84$	$\leftarrow$ max	**210**
	$k=2$	$C(1,2)+C(3,3)+p_0p_2p_3=42+0+2\cdot3\cdot6=42+0+36$		78
			$\Rightarrow\ C(1,3)=210,\quad K(1,3)=1$	
$C(2,4)$	$k=2$	$C(2,2)+C(3,4)+p_1p_2p_4=0+180+7\cdot3\cdot10=0+180+210$		390
	$k=3$	$C(2,3)+C(4,4)+p_1p_3p_4=126+0+7\cdot6\cdot10=126+0+420$	$\leftarrow$ max	**546**
			$\Rightarrow\ C(2,4)=546,\quad K(2,4)=3$	
$C(3,5)$	$k=3$	$C(3,3)+C(4,5)+p_2p_3p_5=0+240+3\cdot6\cdot4=0+240+72$	$\leftarrow$ max	**312**
	$k=4$	$C(3,4)+C(5,5)+p_2p_4p_5=180+0+3\cdot10\cdot4=180+0+120$		300
			$\Rightarrow\ C(3,5)=312,\quad K(3,5)=3$	
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1)+C(2,4)+p_0p_1p_4=0+546+2\cdot7\cdot10=0+546+140$	$\leftarrow$ max	**686**
	$k=2$	$C(1,2)+C(3,4)+p_0p_2p_4=42+180+2\cdot3\cdot10=42+180+60$		282
	$k=3$	$C(1,3)+C(4,4)+p_0p_3p_4=210+0+2\cdot6\cdot10=210+0+120$		330
			$\Rightarrow\ C(1,4)=686,\quad K(1,4)=1$	
$C(2,5)$	$k=2$	$C(2,2)+C(3,5)+p_1p_2p_5=0+312+7\cdot3\cdot4=0+312+84$		396
	$k=3$	$C(2,3)+C(4,5)+p_1p_3p_5=126+240+7\cdot6\cdot4=126+240+168$		534
	$k=4$	$C(2,4)+C(5,5)+p_1p_4p_5=546+0+7\cdot10\cdot4=546+0+280$	$\leftarrow$ max	**826**
			$\Rightarrow\ C(2,5)=826,\quad K(2,5)=4$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1)+C(2,5)+p_0p_1p_5=0+826+2\cdot7\cdot4=0+826+56$	$\leftarrow$ max	**882**
	$k=2$	$C(1,2)+C(3,5)+p_0p_2p_5=42+312+2\cdot3\cdot4=42+312+24$		378
	$k=3$	$C(1,3)+C(4,5)+p_0p_3p_5=210+240+2\cdot6\cdot4=210+240+48$		498
	$k=4$	$C(1,4)+C(5,5)+p_0p_4p_5=686+0+2\cdot10\cdot4=686+0+80$		766
			$\Rightarrow\ C(1,5)=882,\quad K(1,5)=1$	

Cost table $C(i,j)$

$i\backslash j$	1	2	3	4	5
1	0	42	210	686	**882**
2		0	126	546	826
3			0	180	312
4				0	240
5					0

Best split table $K(i,j)$

$i\backslash j$	1	2	3	4	5
1	–	1	1	1	1
2		–	2	3	4
3			–	3	3
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $(M_1\,(((M_2\,M_3)\,M_4)\,M_5))$

Check by multiplying it out:

- $(M_2)\cdot(M_3)$: $(7\times3)\cdot(3\times6)$ costs $7\cdot3\cdot6=126$
- $((M_2\,M_3))\cdot(M_4)$: $(7\times6)\cdot(6\times10)$ costs $7\cdot6\cdot10=420$
- $(((M_2\,M_3)\,M_4))\cdot(M_5)$: $(7\times10)\cdot(10\times4)$ costs $7\cdot10\cdot4=280$
- $(M_1)\cdot((((M_2\,M_3)\,M_4)\,M_5))$: $(2\times7)\cdot(7\times4)$ costs $2\cdot7\cdot4=56$
- Total $=126+420+280+56=882$ ✓

Minimum: Dimensions: $M_1 : 2 \times 7$, $M_2 : 7 \times 3$, $M_3 : 3 \times 6$, $M_4 : 6 \times 10$, $M_5 : 10 \times 4$. So $p = (2, 7, 3, 6, 10, 4)$, i.e. $p_0 = 2$, $p_1 = 7$, $p_2 = 3$, $p_3 = 6$, $p_4 = 10$, $p_5 = 4$.

Base case: $C(i, i) = 0$ for $i = 1, \ldots, 5$.

cell	split	$C(i,k) + C(k{+}1, j) + p_{i-1}\, p_k\, p_j$		value
Chains of length 2				
$C(1,2)$	$k=1$	$C(1,1) + C(2,2) + p_0 p_1 p_2 = 0 + 0 + 2 \cdot 7 \cdot 3 = 0 + 0 + 42$		**42**
			$\Rightarrow\ C(1,2) = 42,\quad K(1,2) = 1$	
$C(2,3)$	$k=2$	$C(2,2) + C(3,3) + p_1 p_2 p_3 = 0 + 0 + 7 \cdot 3 \cdot 6 = 0 + 0 + 126$		**126**
			$\Rightarrow\ C(2,3) = 126,\quad K(2,3) = 2$	
$C(3,4)$	$k=3$	$C(3,3) + C(4,4) + p_2 p_3 p_4 = 0 + 0 + 3 \cdot 6 \cdot 10 = 0 + 0 + 180$		**180**
			$\Rightarrow\ C(3,4) = 180,\quad K(3,4) = 3$	
$C(4,5)$	$k=4$	$C(4,4) + C(5,5) + p_3 p_4 p_5 = 0 + 0 + 6 \cdot 10 \cdot 4 = 0 + 0 + 240$		**240**
			$\Rightarrow\ C(4,5) = 240,\quad K(4,5) = 4$	
Chains of length 3				
$C(1,3)$	$k=1$	$C(1,1) + C(2,3) + p_0 p_1 p_3 = 0 + 126 + 2 \cdot 7 \cdot 6 = 0 + 126 + 84$		210
	$k=2$	$C(1,2) + C(3,3) + p_0 p_2 p_3 = 42 + 0 + 2 \cdot 3 \cdot 6 = 42 + 0 + 36$	$\leftarrow$ min	**78**
			$\Rightarrow\ C(1,3) = 78,\quad K(1,3) = 2$	
$C(2,4)$	$k=2$	$C(2,2) + C(3,4) + p_1 p_2 p_4 = 0 + 180 + 7 \cdot 3 \cdot 10 = 0 + 180 + 210$	$\leftarrow$ min	**390**
	$k=3$	$C(2,3) + C(4,4) + p_1 p_3 p_4 = 126 + 0 + 7 \cdot 6 \cdot 10 = 126 + 0 + 420$		546
			$\Rightarrow\ C(2,4) = 390,\quad K(2,4) = 2$	
$C(3,5)$	$k=3$	$C(3,3) + C(4,5) + p_2 p_3 p_5 = 0 + 240 + 3 \cdot 6 \cdot 4 = 0 + 240 + 72$		312
	$k=4$	$C(3,4) + C(5,5) + p_2 p_4 p_5 = 180 + 0 + 3 \cdot 10 \cdot 4 = 180 + 0 + 120$	$\leftarrow$ min	**300**
			$\Rightarrow\ C(3,5) = 300,\quad K(3,5) = 4$	
Chains of length 4				
$C(1,4)$	$k=1$	$C(1,1) + C(2,4) + p_0 p_1 p_4 = 0 + 390 + 2 \cdot 7 \cdot 10 = 0 + 390 + 140$		530
	$k=2$	$C(1,2) + C(3,4) + p_0 p_2 p_4 = 42 + 180 + 2 \cdot 3 \cdot 10 = 42 + 180 + 60$		282
	$k=3$	$C(1,3) + C(4,4) + p_0 p_3 p_4 = 78 + 0 + 2 \cdot 6 \cdot 10 = 78 + 0 + 120$	$\leftarrow$ min	**198**
			$\Rightarrow\ C(1,4) = 198,\quad K(1,4) = 3$	
$C(2,5)$	$k=2$	$C(2,2) + C(3,5) + p_1 p_2 p_5 = 0 + 300 + 7 \cdot 3 \cdot 4 = 0 + 300 + 84$	$\leftarrow$ min	**384**
	$k=3$	$C(2,3) + C(4,5) + p_1 p_3 p_5 = 126 + 240 + 7 \cdot 6 \cdot 4 = 126 + 240 + 168$		534
	$k=4$	$C(2,4) + C(5,5) + p_1 p_4 p_5 = 390 + 0 + 7 \cdot 10 \cdot 4 = 390 + 0 + 280$		670
			$\Rightarrow\ C(2,5) = 384,\quad K(2,5) = 2$	
Chains of length 5				
$C(1,5)$	$k=1$	$C(1,1) + C(2,5) + p_0 p_1 p_5 = 0 + 384 + 2 \cdot 7 \cdot 4 = 0 + 384 + 56$		440
	$k=2$	$C(1,2) + C(3,5) + p_0 p_2 p_5 = 42 + 300 + 2 \cdot 3 \cdot 4 = 42 + 300 + 24$		366
	$k=3$	$C(1,3) + C(4,5) + p_0 p_3 p_5 = 78 + 240 + 2 \cdot 6 \cdot 4 = 78 + 240 + 48$		366
	$k=4$	$C(1,4) + C(5,5) + p_0 p_4 p_5 = 198 + 0 + 2 \cdot 10 \cdot 4 = 198 + 0 + 80$	$\leftarrow$ min	**278**
			$\Rightarrow\ C(1,5) = 278,\quad K(1,5) = 4$	

Cost table $C(i,j)$

$i \backslash j$	1	2	3	4	5
1	0	42	78	198	278
2		0	126	390	384
3			0	180	300
4				0	240
5					0

Best split table $K(i,j)$

$i \backslash j$	1	2	3	4	5
1	–	1	2	3	4
2		–	2	2	2
3			–	3	4
4				–	4
5					–

Reading off the parenthesization (start at $K(1,5)$ and recurse): $((((M_1\, M_2)\, M_3)\, M_4)\, M_5)$

Check by multiplying it out:

- $(M_1) \cdot (M_2)$: $(2 \times 7) \cdot (7 \times 3)$ costs $2 \cdot 7 \cdot 3 = 42$
- $((M_1\, M_2)) \cdot (M_3)$: $(2 \times 3) \cdot (3 \times 6)$ costs $2 \cdot 3 \cdot 6 = 36$
- $(((M_1\, M_2)\, M_3)) \cdot (M_4)$: $(2 \times 6) \cdot (6 \times 10)$ costs $2 \cdot 6 \cdot 10 = 120$
- $((((M_1\, M_2)\, M_3)\, M_4)) \cdot (M_5)$: $(2 \times 10) \cdot (10 \times 4)$ costs $2 \cdot 10 \cdot 4 = 80$
- Total $= 42 + 36 + 120 + 80 = 278$ ✓

Maximum 882 with $(M_1(((M_2M_3)M_4)M_5))$; minimum 278 with $((((M_1M_2)M_3)M_4)M_5)$.

APPENDIX A

One-Page Formula Sheet

Q1 Hanoi on a graph

- Draw the graph. Parking principle: when disk k moves $X \to Y$, disks $1..k-1$ are on a peg $\neq X, Y$.
- **4-loop** P_0=Start$\to P_1 \to P_2 \to P_3 \to P_0$, Dest $= P_d$: call $\text{Go}_d(n, 0)$.
- Go1: Go2(k-1,p); k: $p \to p{+}1$; Go3(k-1,p+2).
- Go2: Go3(k-1,p); k: $p \to p{+}1 \to p{+}2$; Go3(k-1,p+3).
- Go3: Go2(k-1,p); k: $p \to p{+}1$; Go2(k-1,p+2); k: $p{+}1 \to p{+}2 \to p{+}3$; Go3(k-1,p).
- $T_1 = T_2' + T_3' + 1$, $T_2 = 2T_3' + 2$, $T_3 = 2T_2' + T_3' + 3$ ($'$ = at $n-1$). $x^2 = x + 4 \Rightarrow \Theta(2.56^n)$; $\mathcal{O}(3^n)$.
- **3-loop**: $C_1 = 2C_2' + 1$, $C_2 = 2C_2' + C_1' + 2$; $\Theta((1+\sqrt{3})^n) = \Theta(2.73^n)$.
- **Tail-in (2012)**: $H = H' + C_1' + C_2' + 3$: 3, 9, 24, 63.
- Classic: $2^n - 1$. Two-way line of 3: $3^n - 1$.

Q2 Page transfers

- Find the **inner** loop variable. Row-major: 2nd subscript = walk, 1st = jump. Column-major: the reverse.
- $R = N/P$ pages per row/column. Walk: R pages per inner loop. Jump: N.
- Fits if pages per inner loop $< F$ (frames).
- Reads: walk NR; jump-no-fit N^2; jump-fits NR.
- Writes = reads, for arrays on the left of $:=$.
- 2018/2014: **8,392,704**. 2008/2012: **2,099,200**. 2020 col: **4,202,496**.

Q3 QuickSort

- $n = 2^k - 1$: 15 (or 31). Best $C(n) = 2C(\frac{n-1}{2}) + n - 1$: $C(15) = 34$, $\Theta(n \log n)$. Worst $\frac{n(n-1)}{2} = 105$, $\Theta(n^2)$.
- First: (a) 8 1 3 2 6 5 7 4 12 9 11 10 14 13 15; (b) sorted.
- Last: (a) 12 1 2 3 7 5 4 6 14 9 10 11 15 13 8; (b) sorted. (Textbook partition: (a) 1 3 2 6 5 7 4 12 9 11 10 14 13 15 8.)
- Second: (a) 4 8 3 1 5 6 7 2 10 12 11 9 13 14 15; (b) 15 1 2 ... 14.
- Next-to-last: (a) 12 2 3 1 5 4 7 6 14 10 11 9 13 8 15; (b) 2 3 ... 14 1 15.
- Middle: (a) sorted; (b) 2 12 4 10 6 14 8 1 3 5 7 9 11 13 15.

Q4 Matrix chain

- M_i is $p_{i-1} \times p_i$. $(a \times b)(b \times c)$ costs abc.
- $C(i,i) = 0$, $C(i,j) =$ $\max$ / $\min_{i \le k < j}\{C(i,k) + C(k{+}1,j) + p_{i-1}p_k p_j\}$.
- Fill by length; $K(i,j)$ = best k; $\Theta(n^3)$ time, $\Theta(n^2)$ space.
- Average: $N(i,j) = \sum N(i,k)N(k{+}1,j)$; $S(i,j) = \sum[S(i,k)N(k{+}1,j) + N(i,k)S(k{+}1,j) + N(i,k)N(k{+}1,j)p_{i-1}p_k p_j]$; avg $= S/N$.
- Catalan: 1, 1, 2, 5, 14, 42.
- Max: 2018 **1251**, 2012 **816**, 2014 **1088**, 2020 **1312/1161**. Avg 2008 **138.71**.

Recurrences

- $T = aT(n-1) + c \Rightarrow \Theta(a^n)$; $T = T(n-1) + cn \Rightarrow \Theta(n^2)$.
- $T = aT(n/b) + cn^d$: $a < b^d$: n^d; $=$: $n^d \log n$; $>$: $n^{\log_b a}$.
- $T = T(n{-}1) + cT(n{-}2)$: largest root of $x^2 = x + c$.

Other

- Huffman: merge two smallest; $E = \sum p_i \ell_i$ = (sum of merged weights)/total.
- Heap: parent $\lfloor i/2 \rfloor$, children $2i, 2i{+}1$; insert sift up; extract: last to root, sift down with larger child.
- Gauss: $T(s) = 3T(s/2) + cs = \mathcal{O}(s^{1.59})$. RSA: $\mathcal{O}(s)$ mults $\Rightarrow \mathcal{O}(s^{2.59})$.
- Inverse: extended Euclid table; make it positive (add m).

APPENDIX B

Answer Index

Question	Answer	page
2008 Q1 Hanoi (loop, $d = 1$)	Go1$(n, 0)$, $\Theta(2.56^n)$	14
2008 Q2 pages	2,099,200	24
2008 Q4 average	$971/7 \approx 138.71$ (worst 236)	44
2012 Q1 Hanoi (tail-in)	$H(n)$, $\Theta(2.73^n)$	16
2012 Q2 pages	2,099,200	24
2012 Q4 max	816	56
2014 Q1 Hanoi (loop, $d = 2$)	Go2$(n, 0)$	14
2014 Q2 pages	8,392,704	24
2014 Q3 QuickSort, last	12 1 2 3 7 5 4 6 14 9 10 11 15 13 8 / sorted	33
2014 Q4 max	1088	58
2018 Q1 Hanoi (loop, $d = 3$)	Go3$(n, 0)$	14
2018 Q2 pages	8,392,704	24
2018 Q3 QuickSort, first	8 1 3 2 6 5 7 4 12 9 11 10 14 13 15 / sorted	32
2018 Q4 max	1251	41
2020 pages col-major / row-major	4,202,496 / 8,392,704	26
2020 matrix chain max	1312 (review) / 1161 (original)	60
2020 Huffman	2.75	48
2008 E2 Huffman	$37/14 \approx 2.643$	48
2008 E2 inverse of 65536 mod 331679	232,731	51
2008 E2 pages	196,608	25
Mock 1	Go2; 1,052,672; last-pivot arrays; max 726	72
Mock 2	tail-in $2, 7, 18, 47$; 12,288; second-pivot; avg 203.5, worst 285	74
Mock 3	Go3; 4,096 / 265,216; middle-pivot; 14 parens, min 1360	78
Mock 4	8,390,656; penult; max 1117; Huffman 17/6; heap; 2753	81
Mock 5	tail-out; 3,145,728 / 6,144; $n = 31$; max 882, min 278	85